



LFS458

Kubernetes Administration

Version 1.35.2



© Copyright The Linux Foundation 2026 All rights reserved.

© Copyright The Linux Foundation 2026 All rights reserved.

The training materials provided or developed by The Linux Foundation in connection with the training services are protected by copyright and other intellectual property rights.

Open source code incorporated herein may have other copyright holders and is used pursuant to the applicable open source license.

The training materials are provided for individual use by participants in the form in which they are provided. They may not be copied, modified, distributed to non-participants or used to provide training to others without the prior written consent of The Linux Foundation.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without express prior written consent.

Published by:

the **Linux Foundation**

<https://www.linuxfoundation.org>

No representations or warranties are made with respect to the contents or use of this material, and any express or implied warranties of merchantability or fitness for any particular purpose or specifically disclaimed.

Although third-party application software packages may be referenced herein, this is for demonstration purposes only and shall not constitute an endorsement of any of these software applications.

Linux is a registered trademark of Linus Torvalds. Other trademarks within this course material are the property of their respective owners.

If there are any questions about proper and fair use of the material herein, please go to <https://trainingsupport.linuxfoundation.org>.

Nondisclosure of Confidential Information

“Confidential Information” shall not include any of the following, even if marked confidential or proprietary: (a) information that relates to the code base of any open source or open standards project (collectively, “Open Project”), including any existing or

future contribution thereto; (b) information generally relating or pertaining to the formation or operation of any Open Project; or (c) information relating to general business matters involving any Open Project.

This course does not include confidential information, nor should any confidential information be divulged in class.

Contents

1	Introduction	1
1.1	The Linux Foundation	2
1.2	The Linux Foundation Training	4
1.3	The Linux Foundation Certifications	8
1.4	The Linux Foundation Digital Badges	11
1.5	The Linux Foundation Subscriptions	12
1.6	Laboratory Exercises, Solutions and Resources	13
1.7	Things Change in Linux and Open Source Projects	15
1.8	E-Learning Course: LFS258	16
1.9	Platform Details	17
2	Basics of Kubernetes	19
2.1	Define Kubernetes	20
2.2	Cluster Structure	21
2.3	Adoption	22
2.4	Project Governance and CNCF	26
2.5	Labs	27
3	Installation and Configuration	29
3.1	Getting Started With Kubernetes	30

3.2	Minikube	33
3.3	kubeadm	34
3.4	More Installation Tools	37
3.5	Labs	41
4	Kubernetes Architecture	77
4.1	Kubernetes Architecture	78
4.2	Networking	93
4.3	Other Cluster Systems	99
4.4	Labs	100
5	APIs and Access	127
5.1	API Access	128
5.2	Annotations	132
5.3	Working with A Simple Pod	133
5.4	kubectl and API	134
5.5	Swagger and OpenAPI	141
5.6	Labs	143
6	API Objects	153
6.1	API Objects	154
6.2	The v1 Group	155
6.3	API Resources	157
6.4	RBAC APIs	162
6.5	Labs	163
7	Managing State With Deployments	181
7.1	Deployment Overview	182
7.2	Deployments and Replica Sets	183
7.3	DaemonSets	191
7.4	Labels	192
7.5	Labs	194

8	Helm and Kustomize	217
8.1	Helm Overview	218
8.2	Helm	219
8.3	Using Helm	222
8.4	Kustomize Overview	224
8.5	Labs	228
9	Volumes and Data	247
9.1	Volumes Overview	248
9.2	Volumes	249
9.3	Persistent Volumes	253
9.4	Rook	257
9.5	Passing Data To Pods	258
9.6	ConfigMaps	261
9.7	Labs	263
10	Services	301
10.1	Overview	302
10.2	Accessing Services	304
10.3	DNS	310
10.4	Labs	312
11	Advanced Traffic Management	337
11.1	Overview	338
11.2	Ingress Controller	339
11.3	Ingress Rules	344
11.4	Service Mesh	346
11.5	Limitations	347
11.6	Gateway API	348
11.7	Labs	353
12	Scheduling	381
12.1	Overview	382

12.2	Scheduler Settings	383
12.3	Pod Specification	388
12.4	Affinity Rules	390
12.5	Taints and Tolerations	395
12.6	Labs	398
13	Logging and Troubleshooting	413
13.1	Overview	414
13.2	Troubleshooting Flow	415
13.3	Basic Start Sequence	417
13.4	Monitoring	418
13.5	Plugins	419
13.6	Logging	422
13.7	Troubleshooting Resources	423
13.8	Labs	424
14	Custom Resource Definition	435
14.1	Overview	436
14.2	Custom Resource Definitions	437
14.3	Aggregated APIs	441
14.4	Labs	442
15	Security	449
15.1	Overview	450
15.2	Accessing the API	452
15.3	Authentication and Authorization	453
15.4	Admission Controller	457
15.5	Network Policies	459
15.6	Labs	463
16	High Availability	479
16.1	Overview	480
16.2	Stacked Database	481

16.3	External Database	482
16.4	Labs	483
17	Closing and Evaluation Survey	497
17.1	Evaluation Survey	498
	Appendices	501
A	Domain Review	501
A.1	CKA Exam	502
A.2	Exam Domain Review	503

List of Figures

2.1	K8s Official Logo	21
2.2	Kubernetes Architecture (image v2)	23
2.3	CNCF	26
3.1	External Access via Browser	74
4.1	Architectural Overview	78
4.2	K8s Architectural Review	91
4.3	Pod Network	93
4.4	Container/Services Networking	94
4.5	Services	95
4.6	Mesos Architecture	99
5.1	Swagger Screenshot	141
9.1	K8s Pod Volumes	249
10.1	Built in services	306
10.2	Service Traffic	307
10.3	Example of Cluster Networking	308

11.1	Ingress Controller for inbound connections	339
11.2	Istio Service Mesh	346
11.3	Main Linkerd Page	356
11.4	Now shows meshed	358
11.5	Five meshed pods	359
11.6	Ingress Traffic	367
11.7	Linkerd Top Metrics	370
13.1	External Access via Browser	430
13.2	External Access via Browser	432
13.3	External Access via Browser	434
15.1	Accessing the API from kubernetes.io	452
16.1	Initial HAProxy Status	487
16.2	Multiple HAProxy Status	492
16.3	HAProxy Down Status	495
17.1	Course Survey	498

Chapter 1

Introduction



1.1	The Linux Foundation	2
1.2	The Linux Foundation Training	4
1.3	The Linux Foundation Certifications	8
1.4	The Linux Foundation Digital Badges	11
1.5	The Linux Foundation Subscriptions	12
1.6	Laboratory Exercises, Solutions and Resources	13
1.7	Things Change in Linux and Open Source Projects	15
1.8	E-Learning Course: LFS258	16
1.9	Platform Details	17

1.1 The Linux Foundation

What is The Linux Foundation?

- A non-profit consortium, dedicated to fostering the growth of:
 - **Linux**
 - Many other **Open Source Software (OSS)** projects and communities
- Supports the creation of sustainable **OSS** ecosystems by providing:
 - Financial and intellectual resources and services
 - Training
 - Events
- Originally founded to protect, support and improve **Linux** development and sponsors the work of **Linux** creator Linus Torvalds
- Supported by leading technology companies and developers in a neutral collaborative environment
- See <https://linuxfoundation.org>.

The Linux Foundation Events

This is only a very partial list of **The Linux Foundation** events. Some are held in multiple locations yearly, such as North America, Europe and Asia.

- Open Source Summit
- Embedded Linux Conference North
- Open Networking & Edge Summit
- KubeCon + CloudNativeCon
- Automotive Linux Summit
- KVM Forum
- Linux Storage Filesystem and Memory Management Summit
- Linux Security Summit
- Linux Kernel Maintainer Summit
- The Linux Foundation Member Summit
- Open Compliance Summit
- And many more.

1.2 The Linux Foundation Training

Training Venues

The Linux Foundation offers several types of training:

- Physical Classroom (often On-Site)
- Online Virtual Classroom
- Individual Self-Paced E-learning over the Internet
- Events-Based

Training Offerings

Our current course offerings include:

- Linux Programming & Development Training
- Enterprise IT & Linux System Administration Courses
- Open Source Compliance Courses

For more information see <https://training.linuxfoundation.org>

The Linux Foundation also offers a wide range of free **MOOCs** (**M**assively **O**pen **O**nline **C**ourses) offered through **edX** at <https://edx.org>. These cover basic as well as rather advanced topics associated with open source.

To find them at the **edX** website, search on "The Linux Foundation"

Copyright

- The contents of this course and all its related materials, including hand-outs, are © Copyright The Linux Foundation 2026 All rights reserved.



Do not copy or distribute

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any way, including, but not limited to, photocopy, photograph, magnetic, electronic or other record, without the prior written permission of The Linux Foundation.

About Confidential Information



Nondisclosure of Confidential Information

“Confidential Information” shall not include any of the following, even if marked confidential or proprietary: (a) information that relates to the code base of any open source or open standards project (collectively, “Open Project”), including any existing or future contribution thereto; (b) information generally relating or pertaining to the formation or operation of any Open Project; or (c) information relating to general business matters involving any Open Project.

This course does not include confidential information, nor should any confidential information be divulged in class.

1.3 The Linux Foundation Certifications

The Linux Foundation Certification Exams

- **The Linux Foundation** offers comprehensive **Certification Programs**.
- Full details about this program can be found at <https://training.linuxfoundation.org/certification>. This information includes a thorough description of the **Domains** and **Competencies** covered by each exam.
- Besides the **LFCS (Linux Foundation Certified Sysadmin)** exam, **The Linux Foundation** provides certification exams for many other open source projects. The list is constantly expanding so please see <https://training.linuxfoundation.org/certification> for the current list.
- For additional information, including course descriptions, technical requirements and other logistics, see <https://training.linuxfoundation.org>.

Certification/Training Firewall

- **The Linux Foundation** has two separate training divisions:
 - Certification
 - Course Delivery
- These are separated by a **firewall**:
 - Enables third party organizations to develop and deliver **LF** certification preparation classes
 - Prevents using ***secret sauce*** in **LF** courses
 - Prevents ***teaching the test*** in **LF** courses
- Instructors (including today) are guided entirely by publicly available information

Preparation Resources

- Before doing anything else, download:
<https://training.linuxfoundation.org/download-free-certification-prep-guide>
- Sections on:
 - Domains and Competencies
 - Free Training Resources
 - Paid Training Resources
 - Taking the Exam
- Also get the **Candidate Handbook** at:
https://training.linuxfoundation.org/go/candidate_handbook
- Also read exam-specific information at:
<https://training.linuxfoundation.org/certification/lfcs>

1.4 The Linux Foundation Digital Badges

The Linux Foundation Digital Badges

- Digital Badges communicate abilities and credentials
- Can be used in email signatures, digital resumes, social media sites (**LinkedIn**, **Facebook**, **Twitter**, etc)
- Contain verified metadata describing qualifications and process that earned them.
- Available to students who successfully complete **The Linux Foundation** courses and certifications
- Details at <https://training.linuxfoundation.org/badges/>

1.5 The Linux Foundation Subscriptions

THRIVE-ONE Subscriptions

Empower your open source learning journey with one of The Linux Foundation subscriptions (<https://training.linuxfoundation.org/subscriptions/>):

- THRIVE-ONE Monthly: Flexible short-term access to all self-paced courses. Perfect for exploring new topics or sharpening specific skills. Earn **digital badges** for each course you complete.
- THRIVE-ONE Annual: Includes everything in the Monthly plan **plus** access to **microlearning modules**, **SkillCred assessments**, and **discounts** on certifications and instructor-led courses. Ideal for long-term, structured skill growth.

1.6 Laboratory Exercises, Solutions and Resources

Labs

- Hands-on exercises provided at the end of each session.
- Can be done on either virtual machines or bare-metal
 - Unless otherwise specified
- Some exercises marked as optional
- Solutions available at the end of each exercise.

Obtaining Course Solutions and Resources

- **If this course has such material**, lab exercise solutions, suggestions and other resources may be downloaded from: <https://cm.lf.training/LFS458>

- If you do not have a browser available you can obtain with:

```
$ wget --user=LFtraining --password=<ask instructor> \  
https://cm.lf.training/LFS458/LFS458_V1.35.2_SOLUTIONS.tar.xz  
and similarly for the RESOURCES file.
```

```
$ wget --user=LFtraining --password=<ask instructor> \  
https://cm.lf.training/LFS458/LFS458_V1.35.2_RESOURCES.tar
```

- Any errata, updated solutions, etc. will also be posted on that site
- These files may be unpacked with:

```
$ tar xvf LFS458_V1.35.2_SOLUTIONS.tar.xz  
$ tar xvf LFS458_V1.35.2_RESOURCES.tar
```

1.7 Things Change in Linux and Open Source Projects

Open Source Software Changes Often

- **The Linux Foundation** courses are constantly updated to synchronize with upstream versions of projects such as **Kubernetes** or the **Linux** kernel.
- There will be unavoidable breakage from time to time and deprecated features will disappear.
- We try to minimize problems, but often we have to respond to upstream changes we cannot control.
- The alternative is to have stale material which is lacking important features.
- Working with these shifting sources and targets is part of the *fun* of working with open source software!

1.8 E-Learning Course: LFS258

Companion E-Learning Course: LFS258

- **The Linux Foundation** offers a self-paced, e-learning closely related course:

LFS258: Kubernetes Fundamentals

- This course covers much of the same material and shares many of the laboratory exercises as this course.
- Access to materials online is available for one year and you can take as much time as you need to cover all the content and exercises.
- Note that this instructor-led version cannot give adequate time to all subjects either in breadth or depth due to time constraints, and some topics are marked as optional.
- You may have received a subscription to **LFS258** as part of your enrollment in this course. Otherwise you can contact separately through <https://training.linuxfoundation.org/>

1.9 Platform Details

Platform Details

Any modern operating system can be used:

- **Linux**
- **MacOS**
- **Windows**
- **Unix**

Required Software:

- Modern Web Browser
- Terminal Emulation Program (**ssh** or **PuTTY**)

Chapter 2

Basics of Kubernetes



2.1	Define Kubernetes	20
2.2	Cluster Structure	21
2.3	Adoption	22
2.4	Project Governance and CNCF	26
2.5	Labs	27

2.1 Define Kubernetes

What Is Kubernetes

- Orchestration
- <http://kubernetes.io>
“Open-source software for automating deployment, scaling, and management of containerized applications”
- Easy to run, potentially complex to integrate
- Built with lessons from **Google**
- Has become a common production solution
- Open and extensible
- From Greek κυβερνητης, pilot or Helmsman

2.2 Cluster Structure

Components of Kubernetes

- Microservices
- Decoupled
- Transient
- API call driven
- YAML to JSON, written in **Go**



Figure 2.1: K8s Official Logo

2.3 Adoption

Challenges

- Easy to build container images
- Simple to share images via registries
- Powerful tools to manage containers
- Network considerations
- Flexible storage

Kubernetes Architecture

- Control Planes
 - API server, Scheduler, operators, and etcd database
- Worker nodes

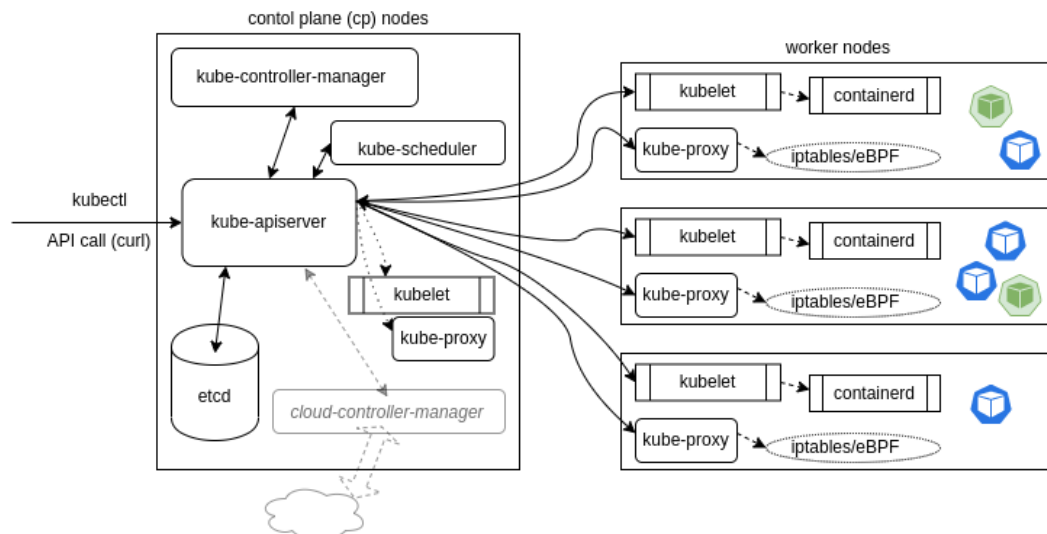


Figure 2.2: Kubernetes Architecture (image v2)

Terminology

- Node
- Pod
- Namespace
- Controllers
 - Deployment
 - ReplicaSet
 - DaemonSet
 - Service
- Label & Selectors
- Taints & Toleration

Innovation

- Seen as the default business solution
- Tens of thousands of contributors
- Hundreds of thousands of commits
- Tens of thousands on **Slack**
- Currently four month minor release cycle
- Patch releases every ten days or so
- Large ecosystem of projects which enhance functionality

2.4 Project Governance and CNCF

Cloud Native Computing Foundation



**CLOUD NATIVE
COMPUTING
FOUNDATION**

Figure 2.3: CNCF

2.5 Labs

Exercise 2.1: View Online Resources

Visit kubernetes.io

With such a fast changing project, it is important to keep track of updates. The main place to find documentation of the current version is <https://kubernetes.io/>.

1. Open a browser and visit the <https://kubernetes.io/> website.
2. In the upper right hand corner, use the drop down to view the versions available. It will say something like v1.30.
3. Select the top level link for **Documentation**. The links on the left of the page can be helpful in navigation.
4. As time permits navigate around other sub-pages such as **SETUP**, **CONCEPTS**, and **TASKS** to become familiar with the layout.

Track Kubernetes Issues

There are hundreds, perhaps thousands, working on Kubernetes every day. With that many people working in parallel there are good resources to see if others are experiencing a similar outage. Both the source code as well as feature and issue tracking are currently on github.com.

1. To view the main page use your browser to visit <https://github.com/kubernetes/kubernetes/>
2. Click on various sub-directories and view the basic information available.
3. Update your URL to point to <https://github.com/kubernetes/kubernetes/issues>. You should see a series of issues, feature requests, and support communication.

4. In the search box you probably see some existing text like `is:issue is:open:` which allows you to filter on the kind of information you would like to see. Append the search string to read: `is:issue is:open label:kind/bug:` then press enter.
5. You should now see bugs in descending date order. Across the top of the issues a menu area allows you to view entries by author, labels, projects, milestones, and assignee as well. Take a moment to view the various other selection criteria.
6. Some times you may want to exclude a kind of output. Update the URL again, but precede the label with a minus sign, like: `is:issue is:open -label:kind/bug`. Now you see everything except bug reports.

Chapter 3

Installation and Configuration



3.1	Getting Started With Kubernetes	30
3.2	Minikube	33
3.3	kubeadm	34
3.4	More Installation Tools	37
3.5	Labs	41

3.1 Getting Started With Kubernetes

Installation Tools

- Kubernetes Installation Tools
 - **kubeadm** community tool
 - Minikube
 - MicroK8s
 - Kubernetes Operations (kops)
- Managed Kubernetes Services
 - Google Kubernetes Engine (GKE)
 - Amazon Elastic Kubernetes Service (EKS)
 - Microsoft Azure Kubernetes Service (AKS)
 - DigitalOcean Kubernetes
- Use may need **kubectrl** or proprietary command

Installing kubectl

- Install or compile **kubectl**
- Main binary for working with objects
- Available for common distributions via dedicated repos
- Configuration file: `$HOME/.kube/config`
 - apiserver endpoints
 - User SSL keys
 - contexts

Using Google Kubernetes Engine (GKE)

- Create account on **GKE**
- Add method of payment
- Install and use **gcloud**
 - Vendor-specific command to manage GKE
- More details:
<https://console.cloud.google.com/getting-started>

3.2 Minikube

Using Minikube

- Open source project within GitHub Kubernetes
- Download from **Google**
- Assumes **VirtualBox** already installed
- Useful for developers
- Uses **Go** binary **localkube**
- Also uses **Docker**

3.3 kubeadm

Install With kubeadm

- Easy to use with strong community support
- Works with current versions of **Ubuntu** and **CentOS**
- Main steps
 - Run `kubeadm init` on the control plane node
 - Create a network for IP-per-Pod criteria
 - Run `kubeadm join` on workers or secondary cp nodes

kubeadm upgrade

- Allows validation prior to and eventual upgrade
- Upgrade to alpha, beta, release candidate, or stable release
- Sub-commands for various functions
 - plan
 - apply
 - diff
 - node

Install A Pod Network

- Only one pod network per cluster
- Several to choose from
 - **Calico**
 - **Canal**
 - **Flannel**
 - **Kube-router**
 - **Cilium**
- Can become complicated to manage
- Several add-ons available

3.4 More Installation Tools

More Installation Tools

- **kubespary**
- **kops**
- **kind**

Installation Considerations

- Which provider should I use?
 - Public or private cloud?
- Which operating system should I use?
- Which networking solution should I use?
 - Do I need an overlay?
- Where should I run my **etcd** cluster?
- Should I configure Highly-Available Control Plane nodes?

Main Deployment Configurations

- Single-node
- Single head node, multiple workers
- Multiple head nodes with HA, multiple workers
- HA etcd, HA head nodes, multiple workers
- Multi-cluster Federation also provides higher availability

Compiling from Source

- Configure **Golang** environment
- Clone source code
- May need to install other compiler and libraries

3.5 Labs

Exercise 3.1: Install Kubernetes

Overview

There are several Kubernetes installation tools provided by various vendors. In this lab we will learn to use **kubeadm**. As a community-supported independent tool, it is planned to become the primary manner to build a Kubernetes cluster.



Platforms: Digital Ocean, GCP, AWS, VirtualBox, etc

The labs were written using **Ubuntu 24.04** instances running on **Google Cloud Platform (GCP)**. They have been written to be vendor-agnostic so could run on AWS, local hardware, or inside of virtualization to give you the most flexibility and options. Each platform will have different access methods and considerations. As of v1.21.0 the minimum (as in barely works) size for **VirtualBox** is 3vCPU/4G memory/5G minimal OS for cp and 1vCPU/2G memory/5G minimal OS for worker node. Most other providers work with 2CPU/7.5G.

If using your own equipment you will have to disable swap on every node, and ensure there is only one network interface. Multiple interfaces are supported but require extra configuration. There may be other requirements which will be shown as warnings or errors when using the **kubeadm** command. While most commands are run as a regular user, there are some which require root privilege. You If you are accessing the nodes remotely, such as with **GCP** or **AWS**, you will need to use an SSH client such as a local terminal or **PuTTY** if not using **Linux** or a Mac. You can download **PuTTY** from www.putty.org. You would also require a **.pem** or **.ppk** file to access the nodes. Each cloud provider will have a process to download or create this file. If attending in-person instructor led training the file will be made available during class.



Very Important

Please disable any firewalls while learning Kubernetes. While there is a list of required ports for communication between components, the list may not be as complete as necessary. If using **GCP** you can add a rule to the project which allows all traffic to all ports. Should you be using **VirtualBox** be aware that inter-VM networking will need to be set to promiscuous mode.

In the following exercise we will install Kubernetes on a single node then grow the cluster, adding more compute resources. Both nodes used are the same size, providing 2 vCPUs and 7.5G of memory. Smaller nodes could be used, but would run slower, and may have strange errors.



YAML files and White Space

Various exercises will use YAML files, which are included in the text. You are encouraged to write some of the files as time permits, as the syntax of YAML has space indentation requirements that are important to learn. An important note, **do not** use tabs in your YAML files, **space only. Indentation matters.**

If using a PDF the use of copy and paste often does not paste the single quote correctly. It pastes as a back-quote instead. You will need to modify it by hand. The files mentioned in labs have also been made available as a compressed **tar** file. You can view the resources by navigating to this URL:

<https://cm.lf.training/LFS458>

To login use user: LFtraining and a password of: Penguin2014

Once you find the name and link of the current file, which will change as the course updates, use **wget** to download the file into your node from the command line then expand it like this:

```
$ wget https://cm.lf.training/LFS458/LFS458_V1.35.2.SOLUTIONS.tar.xz \
    --user=LFtraining --password=Penguin2014
```

```
$ tar -xvf LFS458_V1.35.2_SOLUTIONS.tar.xz
```

(**Note:** depending on your PDF viewer, if you are cutting and pasting the above instructions, the underscores may disappear and be replaced by spaces, so you may have to edit the command line by hand!)

Install Kubernetes

Log into your control plane (cp) and worker nodes. If attending in-person instructor led training the node IP addresses will be provided by the instructor. You will need to use a [.pem](#) or [.ppk](#) key for access, depending on if you are using **ssh** from a terminal or **PuTTY**. The instructor will provide this to you.

1. Open a terminal session on your first node. For example, connect via **PuTTY** or **SSH** session to the first **GCP** node. The user name may be different than the one shown, `student`. Create a non-root user if one is not present. The IP used in the example will be different than the one you will use. You may need to adjust the access mode of your pem or ppk key. The example shows how a Mac or Linux system would change mode. Windows may have a similar process.

```
[student@laptop ~]$ chmod 400 LFS458.pem  
[student@laptop ~]$ ssh -i LFS458.pem student@35.226.100.87
```

```
The authenticity of host '54.214.214.156 (35.226.100.87)' can't be established.  
ECDSA key fingerprint is SHA256:IPvznbkx93/Wc+ACwXrCcDDgvBwmvEXC9vmYhk2Wo1E.  
ECDSA key fingerprint is MD5:d8:c9:4b:b0:b0:82:d3:95:08:08:4a:74:1b:f6:e1:9f.  
Are you sure you want to continue connecting (yes/no)? yes  
Warning: Permanently added '35.226.100.87' (ECDSA) to the list of known hosts.  
<output_omitted>
```

2. Use the **wget** command above to download and extract the course tarball to your node. Again copy and paste won't always paste the underscore characters.

3. You are encouraged to type out commands, if using a PDF or eLearning, instead of copy and paste. By typing the commands you have a better chance to remember both the command and the concept. There are a few exceptions, such as when a long hash or output is much easier to copy over, and does not offer a learning opportunity.
4. Become `root` and update and upgrade the system. You may be asked a few questions. If so, allow restarts and keep the local version currently installed. Which would be a `yes` then a `2`.

```
student@cp:~$ sudo -i
```

```
root@cp:~# apt-get update && apt-get upgrade -y
```

```
<output_omitted>
```

5. The main choices for a container environment are **containerd**, **cri-o**, and **Docker** on older clusters. We suggest **containerd** for class, as it is easy to deploy and commonly used by cloud providers.

Please note, install one engine only. If more than one are installed the **kubeadm** init process search pattern will use Docker at the moment. Also be aware that engines other than **containerd** may show different output on some commands.

6. There are several packages we should install to ensure we have all dependencies take care of. Please note the backslash is not necessary and can be removed if typing on a single line.

```
root@cp:~# apt install apt-transport-https tree \
software-properties-common ca-certificates socat -y
```

```
<output-omitted>
```

7. Disable swap if not already done. Cloud providers disable swap on their images.

```
root@cp:~# swapoff -a
```


8. Load modules to ensure they are available for following steps.

```
root@cp:~# modprobe overlay
root@cp:~# modprobe br_netfilter
```

9. Update kernel networking to allow necessary traffic. Be aware the shell will add a greater than sign (>) to indicate the command continues after a carriage return.

```
root@cp:~# cat << EOF | tee /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

```
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```

10. Ensure the changes are used by the current kernel as well

```
root@cp:~# sysctl --system
```

```
* Applying /etc/sysctl.d/10-console-messages.conf ...
kernel.printk = 4 4 1 7
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
kernel.kptr_restrict = 1
<output_omitted>
```

11. Install the necessary key for the software to install

```
root@cp:~# mkdir -p /etc/apt/keyrings
root@cp:~# curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| gpg --dearmor -o /etc/apt/keyrings/docker.gpg

root@cp:~# echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

12. Install the containerd software.

```
root@cp:~# apt-get update && apt-get install containerd.io -y
root@cp:~# containerd config default | tee /etc/containerd/config.toml
root@cp:~# sed -e 's/SystemdCgroup = false/SystemdCgroup = true/g' -i /etc/containerd/config.toml
root@cp:~# systemctl restart containerd
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
<output_omitted>
```

13. Download the public signing key for the Kubernetes package repositories

```
root@cp:~# curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key \
| sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

14. Add the appropriate Kubernetes apt repository. Please note that this repository have packages only for Kubernetes 1.30; for other Kubernetes minor versions, you need to change the Kubernetes minor version in the URL to match your desired minor version

```
root@cp:~# echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" \
| sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]  
↪ https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /
```

15. Update with the new repo declared, which will download updated repo information.

```
root@cp:~# apt-get update
```

```
<output-omitted>
```

16. Install the Kubernetes software. There are regular releases, the newest of which can be used by omitting the equal sign and version information on the command line. Historically new versions have lots of changes and a good chance of a bug or five. As a result we will hold the software at the recent but stable version we install. In a later lab we will update the cluster to a newer version.

```
root@cp:~# apt-get install -y kubeadm=1.34.2-1.1 kubelet=1.34.2-1.1 kubectl=1.34.2-1.1
```

```
<output-omitted>
```

```
root@cp:~# apt-mark hold kubelet kubeadm kubectl
```

```
kubelet set on hold.  
kubeadm set on hold.  
kubectl set on hold.
```

17. Find the IP address of the primary interface of the cp server. The example below would be the `ens4` interface and an IP of `10.128.0.3`, yours may be different. There are two ways of looking at your IP addresses.

```
root@cp:~# hostname -i
```

```
10.128.0.3
```

```
root@cp:~# ip addr show
```

```
....
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc mq state UP group default qlen 1000
   link/ether 42:01:0a:80:00:18 brd ff:ff:ff:ff:ff:ff
   inet 10.128.0.3/32 brd 10.128.0.3 scope global ens4
       valid_lft forever preferred_lft forever
   inet6 fe80::4001:aff:fe80:18/64 scope link
       valid_lft forever preferred_lft forever
....
```

18. Add an local DNS alias for our cp server. Edit the `/etc/hosts` file and add the above IP address and assign a name `k8scp`.

```
root@cp:~# vim /etc/hosts
```

```
10.128.0.3 k8scp      #<-- Add this line
10.128.0.3 cp        #<-- Add this line
127.0.0.1 localhost
....
```

19. Create a configuration file for the cluster. There are many options we could include, and they differ for **containerd**, **Docker**, and **cri-o**. Use the file included in the course tarball. After our cluster is initialized we will view other default values used. Be sure to use the node alias we added to `/etc/hosts`, not the IP so the network certificates will continue to work when we deploy a load balancer in a future lab. The file is already included in the course tarball and doesn't require any changes.

```
root@cp:~# cp /home/student/LFS458/SOLUTIONS/s_03/kubeadm-config.yaml /root/
```

```
root@cp:~# vim kubeadm-config.yaml
```



kubeadm-config.yaml

```
1 apiVersion: kubeadm.k8s.io/v1beta4
2 kind: ClusterConfiguration
3 kubernetesVersion: 1.34.2
4 controlPlaneEndpoint: "k8scp:6443"      #<-- Use the alias we put in /etc/hosts not the IP
5 networking:
6   podSubnet: 192.168.0.0/16
7
```

20. Initialize the cp. Scan through the output. Expect the output to change as the software matures. At the end are configuration directions to run as a non-root user. The token is mentioned as well. This information can be found later with the **kubeadm token list** command. The output also directs you to create a pod network to the cluster, which will be our next step. Pass the network settings **Cilium** has in its configuration file. **Please note:** the output lists several commands which following exercise steps will complete.

```
root@cp:~# kubeadm init --config=kubeadm-config.yaml --upload-certs --node-name=cp \
| tee kubeadm-init.out      #<-- Save output for future review
```

```
[init] Using Kubernetes version: v1.34.2
```

```
[preflight] Running pre-flight checks
```

```
<output_omitted>
```

```
You can now join any number of the control-plane node
running the following command on each as root:
```

```
kubeadm join k8scp:6443 --token vapzqi.et2p9zbkzk29wwth \
--discovery-token-ca-cert-hash
↳ sha256:f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd \
```

```
--control-plane --certificate-key  
↪ 911d41fcada89a18210489afaa036cd8e192b1f122ebb1b79cce1818f642fab8
```

Please note that the certificate-key gives access to cluster sensitive data, keep it secret!

As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use

"kubeadm init phase upload-certs --upload-certs" to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join k8scp:6443 --token vapzqi.et2p9zbkzk29wwth \  
--discovery-token-ca-cert-hash  
↪ sha256:f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd
```

21. As suggested in the directions at the end of the previous output we will allow a non-root user admin level access to the cluster. Take a quick look at the configuration file once it has been copied and the permissions fixed.

```
root@cp:~# exit
```

```
logout
```

```
student@cp:~$ mkdir -p $HOME/.kube
```

```
student@cp:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
student@cp:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
student@cp:~$ less .kube/config
```

```
apiVersion: v1
clusters:
- cluster:
  <output_omitted>
```

22. Deciding which pod network to use for Container Networking Interface (**CNI**) should take into account the expected demands on the cluster. There can be only one pod network per cluster, although the **Multus CNI** project is trying to change this.

The network must allow container-to-container, pod-to-pod, pod-to-service, and external-to-service communications. We will use **Cilium** as a network plugin which will allow us to use Network Policies later in the course.

Cilium is generally installed using "cilium install" or using "helm install" commands. We have generated the cilium-cni.yaml file using the below commands for your convenience. **Note:** You don't need to execute the commands in this box, they are just for reference.

```
$ helm repo add cilium https://helm.cilium.io/
$ helm repo update
$ helm template cilium cilium/cilium --version 1.19.1 \
  --namespace kube-system > cilium.yaml
```

```
student@cp:~$ find $HOME -name cilium-cni.yaml
student@cp:~$ kubectl apply -f /home/student/LFS458/SOLUTIONS/s_03/cilium-cni.yaml
```

```
serviceaccount/cilium created
serviceaccount/cilium-operator created
secret/cilium-ca created
configmap/cilium-config created
<output_omitted>
```

23. While many objects have short names, a **kubectl** command can be a lot to type. We will enable **bash** auto-completion. Begin by adding the settings to the current shell. Then update the `$HOME/.bashrc` file to make it persistent. Ensure the `bash-completion` package is installed. If it was not installed, log out then back in for the shell completion to work.

```
student@cp:~$ sudo apt-get install bash-completion -y

<exit and log back in>

student@cp:~$ source <(kubectl completion bash)

student@cp:~$ echo "source <(kubectl completion bash)" >> $HOME/.bashrc
```

24. Test by describing the node again. Type the first three letters of the sub-command then type the **Tab** key. Auto-completion assumes the default namespace. Pass the namespace first to use auto-completion with a different namespace. By pressing **Tab** multiple times you will see a list of possible values. Continue typing until a unique name is used. First look at the current node (your node name may not start with `cp`), then look at pods in the `kube-system` namespace. If you see an error instead such as `-bash: _get_comp_words_by_ref: command not found` revisit the previous step, install the software, log out and back in.

```
student@cp:~$ kubectl des<Tab> no<Tab> cp<Tab>

student@cp:~$ kubectl -n kube-s<Tab> g<Tab> po<Tab>
```

25. Explore the **kubectl help** command. The output has been omitted from commands. Take a moment to review help topics.

```
student@cp:~$ kubectl help
student@cp:~$ kubectl help create
```

26. View other values we could have included in the `kubeadm-config.yaml` file when creating the cluster.

```
student@cp:~$ sudo kubeadm config print init-defaults
```

```
apiVersion: kubeadm.k8s.io/v1beta4
bootstrapTokens:
- groups:
```



```
- system:bootstrappers:kubeadm:default-node-token
token: abcdef.0123456789abcdef
ttl: 24h0m0s
usages:
- signing
- authentication
kind: InitConfiguration
<output_omitted>
```

Exercise 3.2: Grow the Cluster

Open another terminal and connect into a your second node. Install **containerd** and Kubernetes software. These are the many, but not all, of the steps we did on the cp node.

This book will use the **worker** prompt for the node being added to help keep track of the proper node for each command. Note that the prompt indicates both the user and system upon which run the command. It can be helpful to change the colors and fonts of your terminal session to keep track of the correct node.

1. Using the same process as before connect to a second node. If attending an instructor-led class session, use the same `.pem` key and a new IP provided by the instructor to access the new node. Giving a different title or color to the new terminal window is probably a good idea to keep track of the two systems. The prompts can look very similar.
2. `student@worker:~$ sudo -i`
3. `root@worker:~# apt-get update && apt-get upgrade -y`

<If asked allow services to restart and keep the local version of software>

4. Install the containerd engine, starting with dependent software.

```
root@worker:~# apt install apt-transport-https \
software-properties-common ca-certificates tree socat -y

root@worker:~# swapoff -a

root@worker:~# modprobe overlay

root@worker:~# modprobe br_netfilter

root@worker:~# cat << EOF | tee /etc/sysctl.d/kubernetes.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF

root@worker:~# sysctl --system

root@worker:~# mkdir -p /etc/apt/keyrings
root@worker:~# curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

root@worker:~# echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

root@worker:~# apt-get update && apt-get install containerd.io -y
root@worker:~# containerd config default | tee /etc/containerd/config.toml
root@worker:~# sed -e 's/SystemdCgroup = false/SystemdCgroup = true/g' -i /etc/containerd/config.toml
root@worker:~# systemctl restart containerd
```

5. Get the GPG key for the software

```
root@worker:~# curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key \
| sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

6. Add Kubernetes repo

```
root@worker:~# echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" \
| sudo tee /etc/apt/sources.list.d/kubernetes.list
```

7. Update repos then install the Kubernetes software. Be sure to match the version on the cp.

```
root@worker:~# apt-get update
```

8. `root@worker:~# apt-get install -y kubeadm=1.34.2-1.1 kubelet=1.34.2-1.1 kubect1=1.34.2-1.1`

9. Ensure the version remains if the system is updated.

```
root@worker:~# apt-mark hold kubeadm kubelet kubect1
```

10. Find the IP address of your **cp** server. The interface name will be different depending on where the node is running. Currently inside of **GCE** the primary interface for this node type is `ens4`. Your interfaces names may be different. From the output we know our cp node IP is `10.128.0.3`.

```
student@cp:~$ hostname -i
```

```
10.128.0.3
```

```
student@cp:~$ ip addr show ens4 | grep inet
```

```
inet 10.128.0.3/32 brd 10.128.0.3 scope global ens4
inet6 fe80::4001:aff:fe8e:2/64 scope link
```

11. At this point we could copy and paste the **join** command from the cp node. That command only works for 24 hours, so we will build our own **join** should we want to add nodes in the future. Find the token on the cp node. The token lasts 24 hours by default. If it has been longer, and no token is present you can generate a new one with the **sudo kubeadm token create** command, seen in the following command.

```
student@cp:~$ sudo kubeadm token create --print-join-command
```

```
kubeadm join k8scp:6443 --token kcu55w.7jso85i0e2dsn05y \  
--discovery-token-ca-cert-hash \  
↪ sha256:0fb62b3c47bfd3af3c15d21f2ab6082fad1f913b244d5980816f8147ce9936ef
```

12. On the **worker node** add a local DNS alias for the cp server. Edit the `/etc/hosts` file and add the cp IP address and assign the name k8scp. The entry should be exactly the same as the edit on the cp.

```
root@worker:~# vim /etc/hosts
```

```
10.128.0.3 k8scp      #<-- Add this line  
10.128.0.3 cp        #<-- Add this line  
127.0.0.1 localhost  
....
```

13. Use the token and hash, in this case as sha256:long-hash to join the cluster from the **second/worker** node. Use the **private** IP address of the cp server and port 6443. The output of the **kubeadm init** on the cp also has an example to use, should it still be available.

```
root@worker:~# kubeadm join \  
k8scp:6443 --token wcal99.hxv9v0gtznz42g6dr \  
--discovery-token-ca-cert-hash \  
sha256:0fb62b3c47bfd3af3c15d21f2ab6082fad1f913b244d5980816f8147ce9936ef --node-name=worker
```

```
[preflight] Running pre-flight checks
```

```
[preflight] Reading configuration from the cluster...
```

```
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm
↳ kubeadm-config -oyaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file
↳ "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Activating the kubelet service
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...
```

This node has joined the cluster:

- * Certificate signing request was sent to apiserver and a response was received.
- * The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

14. Try to run the **kubectl** command on the secondary system. It should fail. You do not have the cluster or authentication keys in your local `.kube/config` file.

```
root@worker:~# exit
```

```
student@worker:~$ kubectl get nodes
```

```
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

```
student@worker:~$ ls -l .kube
```

```
ls: cannot access '.kube': No such file or directory
```

Exercise 3.3: Finish Cluster Setup

1. View the available nodes of the cluster. It can take a minute or two for the status to change from NotReady to Ready.

The NAME field can be used to look at the details. Your node name may be different, use YOUR control-plane name in future commands, if different than the book.

```
student@cp:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready	control-plane	28m	v1.34.2
worker	Ready	<none>	50s	v1.34.2

2. Look at the details of the node. Work line by line to view the resources and their current status. Notice the status of Taints. The cp won't allow non-infrastructure pods by default for security and resource contention reasons. Take a moment to read each line of output, some appear to be an error until you notice the status shows False.

```
student@cp:~$ kubectl describe node cp
```

```
Name:                cp
Roles:               control-plane
Labels:              beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=cp
                    kubernetes.io/os=linux
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
Annotations:         kubeadm.alpha.kubernetes.io/cri-socket:
↳  unix:///var/run/containerd/containerd.sock
                    node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
Taints:              node-role.kubernetes.io/control-plane:NoSchedule
<output_omitted>
```

3. Allow the cp server to run non-infrastructure pods. The cp node begins tainted for security and performance reasons.

We will allow usage of the node in the training environment, but this step may be skipped in a production environment. Note the **minus sign (-)** at the end, which is the syntax to remove a taint. As the second node does not have the taint you will get a `not found` error. There may be more than one taint. Keep checking and removing them until all are removed.

```
student@cp:~$ kubectl describe node | grep -i taint
```

```
Taints:          node-role.kubernetes.io/control-plane:NoSchedule
Taints:          <none>
```

```
student@cp:~$ kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

```
node/cp untainted
error: taint "node-role.kubernetes.io/control-plane" not found
```

```
student@cp:~$ kubectl describe node | grep -i taint
```

```
Taints:          <none>
Taints:          <none>
```

4. Determine if the DNS and Cilium pods are ready for use. They should all show a status of `Running`. It may take a minute or two to transition from `Pending`.

```
student@cp:~$ kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	cilium-operator-788c7d7585-tnsph	1/1	Running	0	95m
kube-system	cilium-swjsj	1/1	Running	0	95m
kube-system	coredns-5d78c9869d-dwds8	1/1	Running	0	100m
kube-system	coredns-5d78c9869d-t24p5	1/1	Running	0	100m

```
<output_omitted>
```

5. **Only if** you notice the `coredns` pods are stuck in `ContainerCreating` status you may have to delete them, causing new ones to be generated. Delete both pods and check to see they show a `Running` state. Your pod names will be different.

```
student@cp:~$ kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	cilium-swjsj	2/2	Running	0	12m
kube-system	coredns-576cbf47c7-rn6v4	0/1	ContainerCreating	0	3s
kube-system	coredns-576cbf47c7-vq5dz	0/1	ContainerCreating	0	94m

```
<output_omitted>
```

```
student@cp:~$ kubectl -n kube-system delete \
  pod coredns-576cbf47c7-vq5dz coredns-576cbf47c7-rn6v4
```

```
pod "coredns-576cbf47c7-vq5dz" deleted
pod "coredns-576cbf47c7-rn6v4" deleted
```

6. When it finished you should see more interfaces will be created . It may take up to a minute to be created. You will notice interfaces such as `cilium` interfaces when you deploy pods, as shown in the output below.

```
student@cp:~$ ip a
```

```
<output_omitted>
3: cilium_net@cilium_host: <BROADCAST,MULTICAST,NOARP,UP,LOWER_UP> mtu 1460 qdisc noqueue state UP
↪ group default qlen 1000
   link/ether be:19:22:da:62:ac brd ff:ff:ff:ff:ff:ff
   inet6 fe80::bc19:22ff:feda:62ac/64 scope link
```



```

    valid_lft forever preferred_lft forever
5: cilium_vxlan: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc noqueue state UNKNOWN group
↪ default qlen 1000
    link/ether ca:22:e7:23:42:89 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::c822:e7ff:fe23:4289/64 scope link
        valid_lft forever preferred_lft forever
<output_omitted>

```

7. Containerd may still be using an out of date notation for the runtime-endpoint. You may see errors about an undeclared resource type such as `unix://`. We will update the **crictl** configuration. There are many possible configuration options. We will set one, and view the configuration file that is created. We will also set this configuration on worker node as well for our convenience.

```

student@cp:~$ sudo crictl config --set \
runtime-endpoint=unix:///run/containerd/containerd.sock \
--set image-endpoint=unix:///run/containerd/containerd.sock

```

```

student@worker:~$ sudo crictl config --set \
runtime-endpoint=unix:///run/containerd/containerd.sock \
--set image-endpoint=unix:///run/containerd/containerd.sock

```

```

student@cp:~$ sudo cat /etc/crictl.yaml

```

```

runtime-endpoint: "unix:///run/containerd/containerd.sock"
image-endpoint: "unix:///run/containerd/containerd.sock"
timeout: 0
debug: false
pull-image-on-create: false
disable-pull-on-run: false

```

Exercise 3.4: Deploy A Simple Application

We will test to see if we can deploy a simple application, in this case the **nginx** web server.

1. Create a new deployment, which is a Kubernetes object, which will deploy an application in a container. Verify it is running and the desired number of containers matches the available.

```
student@cp:~$ kubectl create deployment nginx --image=nginx
```

```
deployment.apps/nginx created
```

```
student@cp:~$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	8s

2. View the details of the deployment. Remember auto-completion will work for sub-commands and resources as well.

```
student@cp:~$ kubectl describe deployment nginx
```

```
Name:                nginx
Namespace:           default
Labels:              app=nginx
Annotations:         deployment.kubernetes.io/revision: 1
Selector:            app=nginx
Replicas:            1 desired | 1 updated | 1 total | 1 ava...
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
<output_omitted>
```

3. View the basic steps the cluster took in order to pull and deploy the new application. You should see several lines of

output. The first column shows the age of each message, note that due to JSON lack of order the time LAST SEEN time does not print out chronologically. Eventually older messages will be removed.

```
student@cp:~$ kubectl get events
```

```
<output_omitted>
```

4. You can also view the output in **yaml** format, which could be used to create this deployment again or new deployments. Get the information but change the output to yaml. Note that halfway down there is status information of the current deployment.

```
student@cp:~$ kubectl get deployment nginx -o yaml
```

YAML

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   annotations:
5     deployment.kubernetes.io/revision: "1"
6   <output_omitted>
7
```

5. Run the command again and redirect the output to a file. Then edit the file. Remove the `creationTimestamp`, `resourceVersion`, and `uid` lines. Also remove all the lines including and after `status:`, which should be somewhere around line 120, if others have already been removed.

```
student@cp:~$ kubectl get deployment nginx -o yaml > first.yaml
```

```
student@cp:~$ vim first.yaml
```

```
<Remove the lines mentioned above>
```

6. Delete the existing deployment.

```
student@cp:~$ kubectl delete deployment nginx
```

```
deployment.apps "nginx" deleted
```

7. Create the deployment again this time using the file.

```
student@cp:~$ kubectl create -f first.yaml
```

```
deployment.apps/nginx created
```

8. Look at the yaml output of this iteration and compare it against the first. The creation time stamp, resource version and unique ID we had deleted are in the new file. These are generated for each resource we create, so we may need to delete them from yaml files to avoid conflicts or false information. You may notice some time stamp differences as well. The status should not be hard-coded either.

```
student@cp:~$ kubectl get deployment nginx -o yaml > second.yaml
```

```
student@cp:~$ diff first.yaml second.yaml
```

```
<output_omitted>
```

9. Now that we have worked with the raw output we will explore two other ways of generating useful YAML or JSON. Use the `--dry-run` option and verify no object was created. Only the prior `nginx` deployment should be found. The output lacks the unique information we removed before, but does have the same essential values.

```
student@cp:~$ kubectl create deployment two --image=nginx --dry-run=client -o yaml
```

YAML

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   creationTimestamp: null
5   labels:
6     app: two
7     name: two
8 spec:
9   <output_omitted>
10
```

```
student@cp:~$ kubectl get deployment
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	7m

10. Existing objects can be viewed in a ready to use YAML output. Take a look at the existing **nginx** deployment.

```
student@cp:~$ kubectl get deployments nginx -o yaml
```

YAML

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   annotations:
5     deployment.kubernetes.io/revision: "1"
6   creationTimestamp: null
7   generation: 1
8   labels:
9     run: nginx
```



```
<output_omitted>
```

```
11
```

11. The output can also be viewed in JSON output.

```
student@cp:~$ kubectl get deployment nginx -o json
```



```
1 {
2   "apiVersion": "apps/v1",
3   "kind": "Deployment",
4   "metadata": {
5     "annotations": {
6       "deployment.kubernetes.io/revision": "1"
7     },
8   <output_omitted>
9 }
```

12. The newly deployed **nginx** container is a light weight web server. We will need to create a service to view the default welcome page. Begin by looking at the help output. Note that there are several examples given, about halfway through the output.

```
student@cp:~$ kubectl expose -h
```

```
<output_omitted>
```

13. Now try to gain access to the web server. As we have not declared a port to use you will receive an error.

```
student@cp:~$ kubectl expose deployment/nginx
```

```
error: couldn't find port via --port flag or introspection
See 'kubectl expose -h' for help and examples.
```

14. To change an object configuration one can use subcommands `apply`, `edit` or `patch` for non-disruptive updates. The `apply` command does a three-way diff of previous, current, and supplied input to determine modifications to make. Fields not mentioned are unaffected. The `edit` function performs a `get`, opens an editor, then an `apply`. You can update API objects in place with `JSON patch` and `merge patch` or `strategic merge patch` functionality.

If the configuration has resource fields which cannot be updated once initialized then a disruptive update could be done using the `replace --force` option. This deletes first then re-creates a resource.

Edit the file. Find the container name, somewhere around line 31 and add the port information as shown below.

```
student@cp:~$ vim first.yaml
```

YAML**first.yaml**

```
1  ....
2  spec:
3    containers:
4      - image: nginx
5        imagePullPolicy: Always
6        name: nginx
7        ports:                                # Add these
8          - containerPort: 80                  # three
9            protocol: TCP                      # lines
10       resources: {}
11  ....
12
```

15. Due to how the object was created we will need to use `replace` to terminate and create a new deployment.

```
student@cp:~$ kubectl replace -f first.yaml --force
```

```
deployment.apps/nginx replaced
```

16. View the Pod and Deployment. Note the AGE shows the Pod was re-created.

```
student@cp:~$ kubectl get deploy,pod
```

NAME		READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx	1/1	1	1	2m4s	

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-7db75b8b78-qjffm	1/1	Running	0	8s

17. Try to expose the resource again. This time it should work.

```
student@cp:~$ kubectl expose deployment/nginx
```

```
service/nginx exposed
```

18. Verify the service configuration. First look at the service, then the endpoint information. Note the ClusterIP is not the current endpoint. Cilium provides the ClusterIP. The Endpoint is provided by kubelet and kube-proxy. Take note of the current endpoint IP. In the example below it is 192.168.1.5:80. We will use this information in a few steps.

```
student@cp:~$ kubectl get svc nginx
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx	ClusterIP	10.100.61.122	<none>	80/TCP	3m


```
student@cp:~$ kubectl get ep nginx
```

NAME	ENDPOINTS	AGE
nginx	192.168.1.5:80	26s

19. Test access to the Cluster IP, port 80. You should see the generic nginx installed and working page. The output should be the same when you look at the ENDPOINTS IP address. If the **curl** command times out the pod may be running on the other node. Run the same command on that node and it should work.

```
student@cp:~$ curl 10.100.61.122:80
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
<output_omitted>
```

```
student@cp:~$ curl 192.168.1.5:80
```

20. Now scale up the deployment from one to three web servers.

```
student@cp:~$ kubectl get deployment nginx
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	12m

```
student@cp:~$ kubectl scale deployment nginx --replicas=3
```

```
deployment.apps/nginx scaled
```

```
student@cp:~$ kubectl get deployment nginx
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	12m

21. View the current endpoints. There now should be three. If the UP-TO-DATE above said three, but AVAILABLE said two wait a few seconds and try again, it could be slow to fully deploy.

```
student@cp:~$ kubectl get ep nginx
```

NAME	ENDPOINTS	AGE
nginx	192.168.0.3:80,192.168.1.5:80,192.168.1.6:80	7m40s

22. Find the oldest pod of the **nginx** deployment and delete it. The Tab key can be helpful for the long names. Use the AGE field to determine which was running the longest. You may notice activity in the other terminal where **tcpdump** is running, when you delete the pod. The pods with 192.168.0 addresses are probably on the cp and the 192.168.1 addresses are probably on the worker

```
student@cp:~$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
nginx-1423793266-7f1qw	1/1	Running	0	14m	192.168.1.5
nginx-1423793266-8w2nk	1/1	Running	0	86s	192.168.1.6
nginx-1423793266-fbt4b	1/1	Running	0	86s	192.168.0.3

```
student@cp:~$ kubectl delete pod nginx-1423793266-7f1qw
```

```
pod "nginx-1423793266-7f1qw" deleted
```

23. Wait a minute or two then view the pods again. One should be newer than the others. In the following example nine seconds instead of four minutes. If your **tcpdump** was using the `veth` interface of that container it will error out. Also note we are using a short name for the object.

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-1423793266-13p69	1/1	Running	0	9s
nginx-1423793266-8w2nk	1/1	Running	0	4m1s
nginx-1423793266-fbt4b	1/1	Running	0	4m1s

24. View the endpoints again. The original endpoint IP is no longer in use. You can delete any of the pods and the service will forward traffic to the existing backend pods.

```
student@cp:~$ kubectl get ep nginx
```

NAME	ENDPOINTS	AGE
nginx	192.168.0.3:80,192.168.1.6:80,192.168.1.7:80	12m

Exercise 3.5: Access from Outside the Cluster

You can access a Service from outside the cluster using a DNS add-on or environment variables. We will use environment variables to gain access to a Pod.

1. Begin by getting a list of the pods.

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-1423793266-13p69	1/1	Running	0	4m10s
nginx-1423793266-8w2nk	1/1	Running	0	8m2s
nginx-1423793266-fbt4b	1/1	Running	0	8m2s

2. Choose one of the pods and use the `exec` command to run **printenv** inside the pod. The following example uses the first pod listed above.

```
student@cp:~$ kubectl exec nginx-1423793266-13p69 \
-- printenv |grep KUBERNETES
```

```
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
<output_omitted>
```

3. Find and then delete the existing service for **nginx**.

```
student@cp:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4h
nginx	ClusterIP	10.100.61.122	<none>	80/TCP	17m

4. Delete the service.

```
student@cp:~$ kubectl delete svc nginx
```

```
service "nginx" deleted
```

5. Create the service again, but this time pass the LoadBalancer type. Check to see the status and note the external ports mentioned. The output will show the External-IP as pending. Unless a provider responds with a load balancer it will continue to show as pending.

```
student@cp:~$ kubectl expose deployment nginx --type=LoadBalancer
```

```
service/nginx exposed
```

```
student@cp:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4h
nginx	LoadBalancer	10.104.249.102	<pending>	80:32753/TCP	6s

6. Open a browser on your local system, not the lab exercise node, and use the public IP of your node and node port 32753, shown in the output above. If running the labs on remote nodes like **AWS** or **GCE** use the public IP you used with PuTTY or SSH to gain access. You may be able to find the IP address using **curl**.

```
student@cp:~$ curl ifconfig.io
```

```
54.214.214.156
```



Figure 3.1: **External Access via Browser**

7. Scale the deployment to zero replicas. Then test the web page again. Once all pods have finished terminating accessing the web page should fail.

```
student@cp:~$ kubectl scale deployment nginx --replicas=0
```

```
deployment.apps/nginx scaled
```

```
student@cp:~$ kubectl get po
```

```
No resources found in default namespace.
```

8. Scale the deployment up to two replicas. The web page should work again.

```
student@cp:~$ kubectl scale deployment nginx --replicas=2
```

```
deployment.apps/nginx scaled
```

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-1423793266-7x181	1/1	Running	0	6s
nginx-1423793266-s6vcz	1/1	Running	0	6s

9. Delete the deployment to recover system resources. Note that deleting a deployment does not delete the endpoints or services.

```
student@cp:~$ kubectl delete deployments nginx
```

```
deployment.apps "nginx" deleted
```

```
student@cp:~$ kubectl delete svc nginx
```

```
service "nginx" deleted
```


Chapter 4

Kubernetes Architecture



4.1	Kubernetes Architecture	78
4.2	Networking	93
4.3	Other Cluster Systems	99
4.4	Labs	100

4.1 Kubernetes Architecture

Main Components

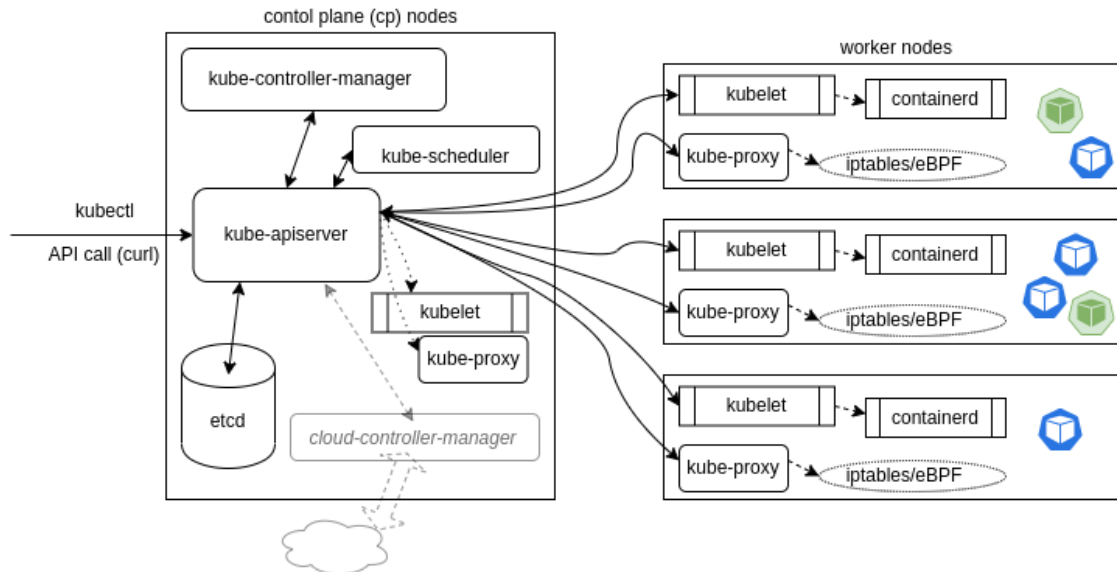


Figure 4.1: Architectural Overview

Control Plane Node

- **kube-apiserver**
- **kube-scheduler**
- **etcd**
- **kube-controller-manager**
- **cloud-controller-manager**
- **CoreDNS**
- Network plugin
- Add-ons
 - Dashboard - Web UI
 - Cluster-level resource monitoring
 - cluster-level logging

kube-apiserver

- Front-end of cluster's shared state
- Control Plane for the cluster
- All components work through it
- Validates and configures data for API objects
- Services REST operations
- Only component to connect to **etcd** database

kube-scheduler

- Uses algorithm to determine Pod placement
- Checks quota restrictions
- Custom scheduling policies possible
- Affinity rules to place pods on specific nodes
- Taints can be used to repel pods
- Pod bindings can force particular scheduling

etcd Database

- Multiversion persistent b+tree key-value store
- Append only, regular compaction
- Shed oldest version of superseded data
- Works with **curl** and other HTTP libraries
- Provides reliable watch queries
- Distributed consensus protocol for leadership

Other Agents

- **kube-controller-manager**
 - Daemon which embeds core control-loops
 - Watches state of cluster
 - Works to make current state match desired state
- **cloud-controller-manager (ccm)**
 - Interacts with outside cloud managers
 - Allows features to be developed outside of core release cycle
 - Each **kubelet** must use `--cloud-provider-external`
 - Handles tasks once part of kube-controller-manager
 - This is an optional agent which takes a few steps to enable
- Network plugins
- **CoreDNS**

Worker Nodes

- kubelet
- kube-proxy
- Docker engine, cri-o, containerd, etc.
- Network plugin pods
- Fluentd
- Prometheus

kubelet

- systemd process on each node
- Uses PodSpec
- Mounts volumes to Pod
- Downloads secrets
- Passes request to local container engine
- Reports status of Pods and node to cluster

Operators

- Watch based control loop monitoring delta
- Informer / SharedInformer
- Workqueue
- Shipped operators
 - Deployment operator
 - replicaSet operator
 - Service operator
 - endpoints operator
 - namespace operator
 - serviceaccounts operator

Service Operator

- Connect Pods together
- Expose Pods to Internet
- Decouple settings
- Define Pod access policy
- Operator monitoring a different operator

Pods

- One or more containers
- Smallest unit to work with
- Only one, shared IP address per Pod

Containers

- Not worked with directly
- Usage limits passed to container engine

```
resources:  
  limits:  
    cpu: "1"  
    memory: "4Gi"  
  requests:  
    cpu: "0.5"  
    memory: "500Mi"
```

- ResourceQuota
- PriorityClass

Init Containers

- Block app containers until precondition met
- Can contain code or utilities not in an app
- Independent security from app container

spec:

containers:

- name: main-app
image: databaseD

initContainers:

- name: wait-database
image: busybox
command: ['sh', '-c', 'until ls /db/dir ; do sleep 5; done; ']

Component Review

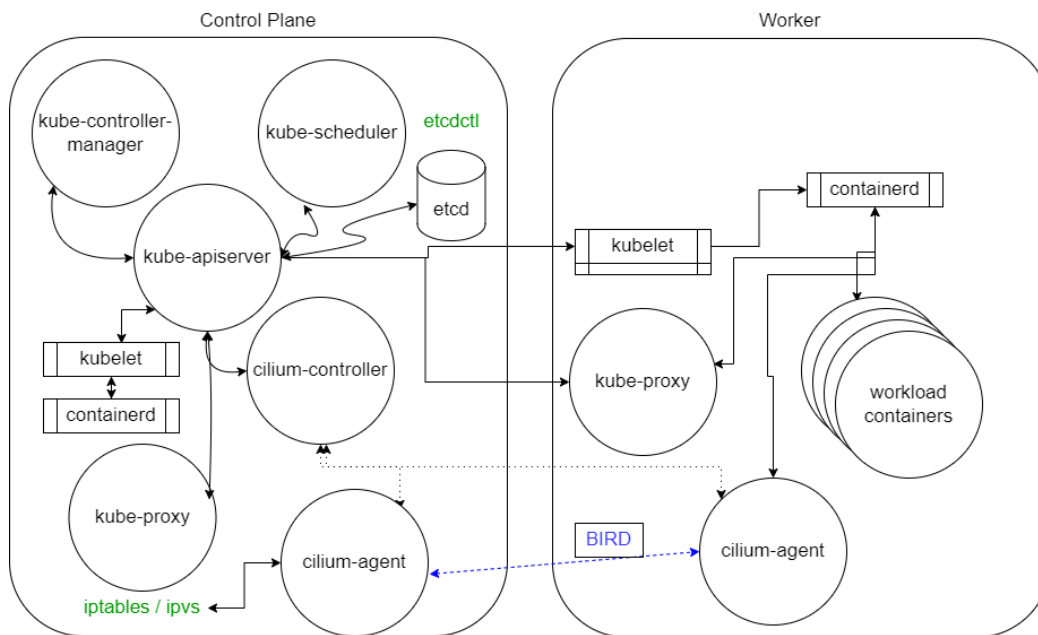


Figure 4.2: K8s Architectural Review

Node

- Created outside of cluster
- **NodeStatus**
- **NodeLease**
- View resource usage with **kubectl describe node**

4.2 Networking

Single IP per Pod

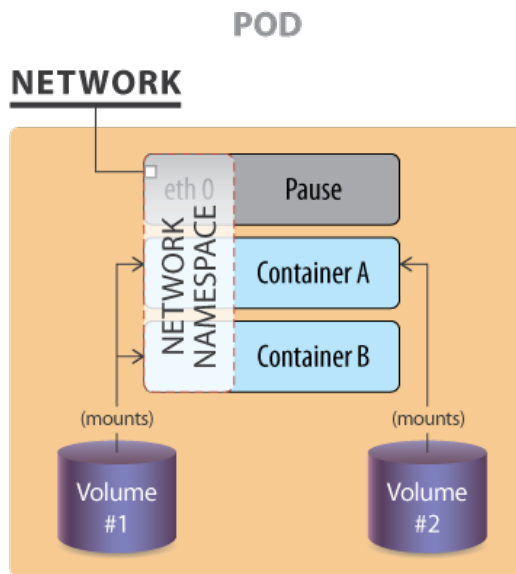


Figure 4.3: Pod Network

Container to Outside Path

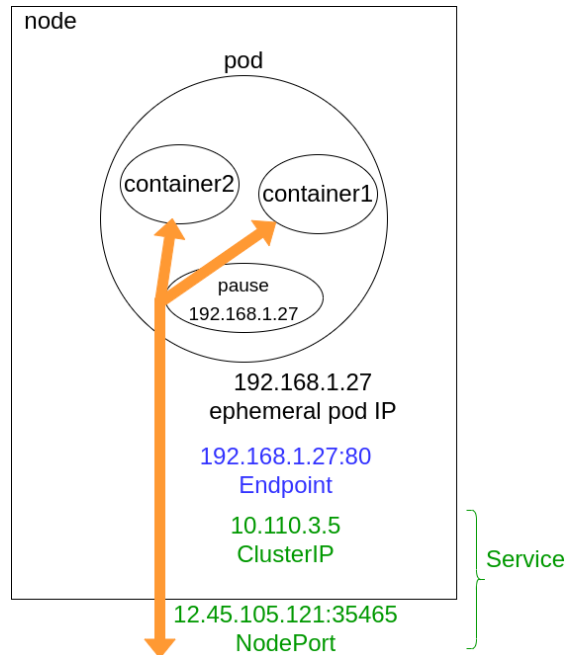


Figure 4.4: **Container/Services Networking**

Services

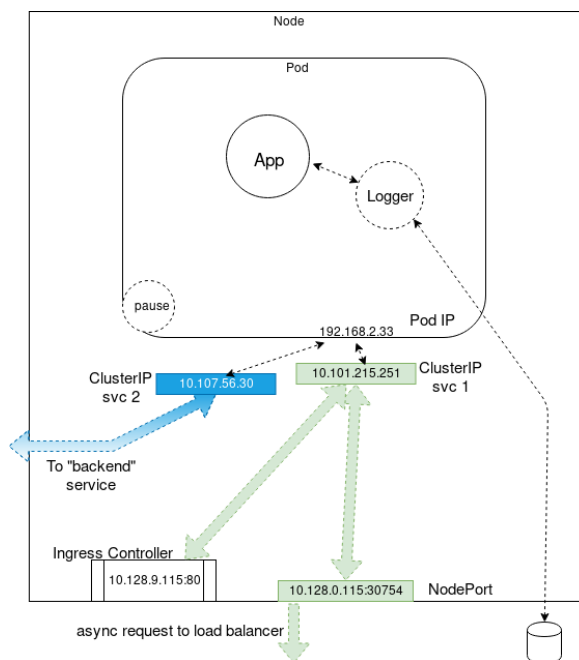


Figure 4.5: Services

Networking Setup

- Main networking challenges:
 - Coupled container-to-container communications
 - Pod-to-pod communications
 - External-to-pod communications (solved by the services concept, to be discussed later)
- Admin configuration required
- IP assigned to pod, not container

CNI Network Configuration File

```
{
  "cniVersion": "1.3.0",
  "name": "mynet",
  "type": "bridge",
  "bridge": "cni0",
  "isGateway": true,
  "ipMasq": true,
  "ipam": {
    "type": "host-local",
    "subnet": "10.22.0.0/16",
    "routes": [
      { "dst": "0.0.0.0/0" }
    ]
  }
}
```

Pod-to-Pod Communication

- All pods can communicate with each other across nodes.
- All nodes can communicate with all pods.
- No Network Address Translation (NAT).

4.3 Other Cluster Systems

Mesos

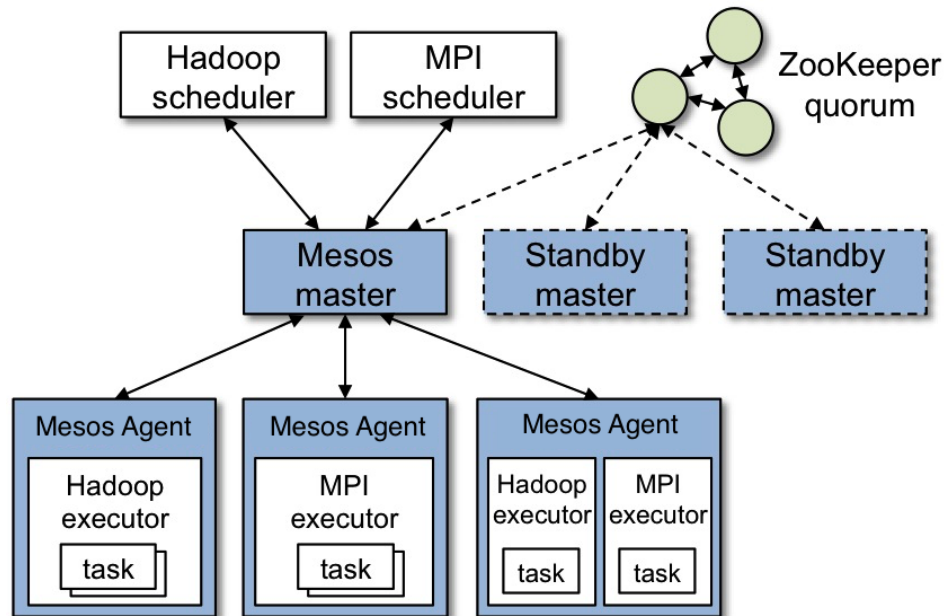


Figure 4.6: Mesos Architecture

4.4 Labs

Exercise 4.1: Basic Node Maintenance

In this section we will backup the **etcd** database then update the version of Kubernetes used on control plane nodes and worker nodes.

Backup The etcd Database

While the upgrade process has become stable, it remains a good idea to backup the cluster state prior to upgrading. There are many tools available in the market to backup and manage etcd, each with a distinct backup and restore process. We will use the included snapshot command, but be aware the exact steps to restore will depend on the tools used, the version of the cluster, and the nature of the disaster being recovered from.

1. Find the data directory of the **etcd** daemon. All of the settings for the pod can be found in the manifest.

```
student@cp:~$ sudo grep data-dir /etc/kubernetes/manifests/etcd.yaml
```

```
- --data-dir=/var/lib/etcd
```

2. Execute the etcdctl commands from the container and look at the options **etcdctl** provides. Use tab to complete the container name, which has the node name appended to it.

```
student@cp:~$ kubectl -n kube-system exec -it etcd-<Tab> -- etcdctl -h
```

```
NAME:
    etcdctl - A simple command line client for etcd3.

USAGE:
    etcdctl [flags]

<output_omitted>
```


3. Check the health of the database using the loopback IP and port 2379. You will need to pass then peer cert and key as well as the Certificate Authority as environmental variables. The command is commented, you do not need to type out the comments or the backslashes.

```
student@cp:~$ kubectl -n kube-system exec -it etcd-cp -- \
etcdctl --cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
--endpoints=https://127.0.0.1:2379 \
endpoint health
```

```
https://127.0.0.1:2379 is healthy: successfully committed proposal: took = 11.942936ms
```

4. Determine how many databases are part of the cluster. Three and five are common in a production environment to provide 50%+1 for quorum. In our current exercise environment we will only see one database. Remember you can use up-arrow to return to the previous command and edit the command without having to type the whole command again.

```
student@cp:~$ kubectl -n kube-system exec -it etcd-cp -- \
etcdctl \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
--endpoints=https://127.0.0.1:2379 \
member list
```

```
fb50b7ddbf4930ba, started, cp, https://10.128.0.35:2380,
https://10.128.0.35:2379, false
```

5. You can also view the status of the cluster in a table format, among others passed with the **-w** option. Again, up-arrow allows you to edit just the last part of the long string easily.

```
student@cp:~$ kubectl -n kube-system exec -it etcd-cp -- \
etcdctl \
```

```
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
--endpoints=https://127.0.0.1:2379 \
member list -w table
```

```
+-----+-----+-----+-----+-----+-----+
|      ID      | STATUS | NAME |      PEER ADDRS      |      CLIENT ADDRS      | IS |
+-----+-----+-----+-----+-----+-----+
| LEARNER |
+-----+-----+-----+-----+-----+-----+
| 802d78549985d5a8 | started | cp | https://10.128.0.15:2380 | https://10.128.0.15:2379 |
| false |
+-----+-----+-----+-----+-----+-----+
```

6. Now that we know how many etcd databases are in the cluster, and their health, we can back it up. Use the snapshot argument to save the snapshot into the container data directory `/var/lib/etcd/`

```
student@cp:~$ kubectl -n kube-system exec -it etcd-cp -- \
etcdctl \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
--endpoints=https://127.0.0.1:2379 \
snapshot save /var/lib/etcd/snapshot.db
```

```
{"level":"info","ts":"2026-03-30T13:26:33.928034Z","caller":"snapshot/v3_snapshot.go:83","
  msg":"created temporary db file","path":"/var/lib/etcd/snapshot.db.part"}
{"level":"info","ts":"2026-03-30T13:26:33.936817Z","logger":"client","caller":"v3@v3.6.5/
  maintenance.go:236","msg":"opened snapshot stream; downloading"}
.
.
.
```

```
Snapshot saved at /var/lib/etcd/snapshot.db  
Server version 3.6.0
```

7. Verify the snapshot exists from the node perspective, the file date should have been moments earlier.

```
student@cp:~$ sudo ls -l /var/lib/etcd/
```

```
total 3888  
drwx----- 4 root root    4096 Aug 25 11:22 member  
-rw----- 1 root root 3973152 Aug 25 18:42 snapshot.db
```

8. Backup the snapshot as well as other information used to create the cluster both locally as well as another system in case the node becomes unavailable. Remember to create snapshots on a regular basis, perhaps using a cronjob to ensure a timely restore. When using the `snapshot restore` it's important the database not be in use. An HA cluster would remove and replace the control plane node, and not need a restore.

```
student@cp:~$ mkdir $HOME/backup  
student@cp:~$ sudo cp /var/lib/etcd/snapshot.db $HOME/backup/snapshot.db-$(date +%m-%d-%y)  
student@cp:~$ sudo cp /root/kubeadm-config.yaml $HOME/backup/  
student@cp:~$ sudo cp -r /etc/kubernetes/pki/etcd $HOME/backup/
```

9. Any mistakes during restore may render the cluster unusable. Instead of issues, and having to rebuild the cluster, please attempt a database restore after the final lab exercise of the course. More on the restore process can be found here: <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/#restoring-an-etcd-cluster>

Upgrade the Cluster

1. Begin by updating the package metadata for **APT**.

```
student@cp:~$ sudo apt update
```

```
Hit:1 http://us-central1.gce.archive.ubuntu.com/ubuntu bionic InRelease
Get:2 http://us-central1.gce.archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
Get:3 http://us-central1.gce.archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
Get:5 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
<output_omitted>
```



Introducing Kubernetes Community-Owned Package Repositories

Note: The legacy package repositories (apt.kubernetes.io and yum.kubernetes.io) have been deprecated and frozen starting from September 13, 2023. Using the new package repositories hosted at pkgs.k8s.io is strongly recommended and required in order to install Kubernetes versions released after September 13, 2023. The deprecated legacy repositories, and their contents, might be removed at any time in the future and without a further notice period. The new package repositories provide downloads for Kubernetes versions starting with v1.24.0. For further information please refer the URL <https://kubernetes.io/blog/2023/08/15/pkgs-k8s-io-introduction/>

2. Replace the apt repository definition so that apt points to the new repository instead of the Google-hosted repository. Make sure to replace the Kubernetes minor version in the command below with the minor version that you are going to upgrade.

```
student@cp:~$ sudo sed -i 's/34/35/g' /etc/apt/sources.list.d/kubernetes.list
```

```
student@cp:~$ sudo apt-get update
```

3. View the available packages.

```
student@cp:~$ sudo apt-cache madison kubeadm
```

```
kubeadm | 1.35.2-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
kubeadm | 1.35.1-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
kubeadm | 1.35.0-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
<output_omitted>
```

4. Remove the hold on **kubeadm** and update the package. Remember to update to the next major release's update 1.

```
student@cp:~$ sudo apt-mark unhold kubeadm
```

```
Canceled hold on kubeadm.
```

```
student@cp:~$ sudo apt-get install -y kubeadm=1.35.2-1.1
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
<output_omitted>
```

5. Hold the package again to prevent updates along with other software.

```
student@cp:~$ sudo apt-mark hold kubeadm
```

```
kubeadm set on hold.
```

6. Verify the version of **Kubeadm**. It should indicate the new version you just installed.

```
student@cp:~$ sudo kubeadm version
```

```
kubeadm version: &version.Info{Major:"1", Minor:"35", EmulationMajor:"", EmulationMinor:"",
↳ MinCompatibilityMajor:"", MinCompatibilityMinor:"", GitVersion:"v1.35.2",
↳ GitCommit:"fdc9d74cbf2da6754ebf81d56f80ae2948cd6425", GitTreeState:"clean",
↳ BuildDate:"2026-02-26T20:04:53Z", GoVersion:"go1.25.7", Compiler:"gc",
↳ Platform:"linux/amd64"}
```

7. To prepare the cp node for update we first need to evict as many pods as possible. The nature of daemonsets is to have them on every node, and some such as Cilium must remain. Change the node name to your node's name, and add the option to ignore the daemonsets.

```
student@cp:~$ kubectl drain cp --ignore-daemonsets
```

```
node/cp cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/cilium-5tv9d, kube-system/kube-proxy-8x9c5
evicting pod kube-system/coredns-5d78c9869d-z5ngb
evicting pod kube-system/cilium-operator-788c7d7585-wnb5b
evicting pod kube-system/coredns-5d78c9869d-4h2bs
pod/cilium-operator-788c7d7585-wnb5b evicted
pod/coredns-5d78c9869d-z5ngb evicted
pod/coredns-5d78c9869d-4h2bs evicted
node/cp drained
```

8. Use the **upgrade plan** argument to check the existing cluster and then update the software. You may notice that there are versions available later than the .1 update in use. If you initialized the cluster in a previous lab using 1.34.1, use upgrade to 1.35.1. Read through the output and get a feel for what would be changed in an upgrade.

```
student@cp:~$ sudo kubeadm upgrade plan
```

```
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the "kubeadm-config" ConfigMap in namespace
↳ "kube-system"...
[upgrade/config] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to
↳ re-upload it.
[upgrade] Running cluster health checks
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: 1.34.2
[upgrade/versions] kubeadm version: v1.35.2
```

```
[upgrade/versions] Target version: v1.35.3
[upgrade/versions] Latest version in the v1.34 series: v1.34.6

<output_omitted>
```

9. We are now ready to actually upgrade the software. There will be a lot of output. Be aware the command will ask if you want to proceed with the upgrade, answer **y** for yes. Take a moment and look for any errors or suggestions, such as upgrading the version of etcd, or some other package. Again, Use the next minor release, update 1 from the version used previous lab which initialized the cluster. The process will take several minutes to complete.

```
student@cp:~$ sudo kubeadm upgrade apply v1.35.2
```

```
[preflight] Running pre-flight checks.
[upgrade/config] Reading configuration from the cluster...
[upgrade/config] FYI: You can look at this config file with 'kubectl -n kube-system get cm
↳ kubeadm-config -o yaml'
[upgrade] Running cluster health checks
[upgrade/version] You have chosen to change the cluster version to "v1.35.2"
[upgrade/versions] Cluster version: v1.34.2
[upgrade/versions] kubeadm version: v1.35.2
[upgrade] Are you sure you want to proceed? [y/N]: y
[upgrade/prepull] Pulling images required for setting up a Kubernetes cluster
[upgrade/prepull] This might take a minute or two, depending on the speed of your internet
↳ connection
[upgrade/prepull] You can also perform this action beforehand using 'kubeadm config images pull'
[upgrade/apply] Upgrading your Static Pod-hosted control plane to version "v1.35.2" (timeout:
↳ 5m0s)...

<output_omitted>
```

10. Check the status of the nodes. The cp should show scheduling disabled. Also as we have not updated all the software and restarted the daemons it will show the previous version.

```
student@cp:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready,SchedulingDisabled	control-plane	109m	v1.34.2
worker	Ready	<none>	61m	v1.34.2

11. Release the hold on **kubelet** and **kubect1**.

```
student@cp:~$ sudo apt-mark unhold kubelet kubect1
```

```
Canceled hold on kubelet.  
Canceled hold on kubect1.
```

12. Upgrade both packages to the same version as **kubeadm**.

```
student@cp:~$ sudo apt-get install -y kubelet=1.35.2-1.1 kubect1=1.35.2-1.1
```

```
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
<output_omitted>
```

13. Again add the hold so other updates don't update the Kubernetes software.

```
student@cp:~$ sudo apt-mark hold kubelet kubect1
```

```
kubelet set on hold.  
kubect1 set on hold.
```

14. Restart the daemons.


```
student@cp:~$ sudo systemctl daemon-reload
```

```
student@cp:~$ sudo systemctl restart kubelet
```

15. Verify the cp node has been updated to the new version. Then update other cp nodes, if you should have them, using the same process except **sudo kubeadm upgrade node** instead of **sudo kubeadm upgrade apply**.

```
student@cp:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready,SchedulingDisabled	control-plane	113m	v1.35.2
worker	Ready	<none>	65m	v1.34.2

16. Now make the cp available for the scheduler, again change the name to match the cluster node name on your control plane.

```
student@cp:~$ kubectl uncordon cp
```

```
node/cp uncordoned
```

17. Verify the cp now shows a Ready status.

```
student@cp:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready	control-plane	114m	v1.35.2
worker	Ready	<none>	66m	v1.34.2

18. Now update the worker node(s) of the cluster. **Open a second terminal session to the worker.** Note that you will need to run a couple commands on the cp as well, having two sessions open may be helpful. Begin by allowing the software

to update on the worker.

```
student@worker:~$ sudo apt-mark unhold kubeadm
```

Canceled hold on kubeadm.



Introducing Kubernetes Community-Owned Package Repositories

Note: The legacy package repositories (apt.kubernetes.io and yum.kubernetes.io) have been deprecated and frozen starting from September 13, 2023. Using the new package repositories hosted at pkgs.k8s.io is strongly recommended and required in order to install Kubernetes versions released after September 13, 2023. The deprecated legacy repositories, and their contents, might be removed at any time in the future and without a further notice period. The new package repositories provide downloads for Kubernetes versions starting with v1.24.0. <https://kubernetes.io/blog/2023/08/15/pkgs-k8s-io-introduction/>

19. Replace the apt repository definition so that apt points to the new repository instead of the Google-hosted repository. Make sure to replace the Kubernetes minor version in the command below with the minor version that you are going to upgrade.

```
student@worker:~$ sudo sed -i 's/34/35/g' /etc/apt/sources.list.d/kubernetes.list
```

```
student@worker:~$ sudo apt-get update
```

20. View the available packages.

```
student@worker:~$ sudo apt-cache madison kubeadm
```

```
kubeadm | 1.35.2-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
kubeadm | 1.35.1-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
kubeadm | 1.35.0-1.1 | https://pkgs.k8s.io/core:stable/v1.35/deb Packages
<output_omitted>
```

21. Update the **kubeadm** package to the same version as the cp node.

```
student@worker:~$ sudo apt-get update && sudo apt-get install -y kubeadm=1.35.2-1.1
```

```
<output_omitted>  
Setting up kubeadm (1.35.2-1.1) ...
```

22. Hold the package again.

```
student@worker:~$ sudo apt-mark hold kubeadm
```

```
kubeadm set on hold.
```

23. Back on the **cp terminal session** drain the worker node, but allow the daemonsets to remain.

```
student@cp:~$ kubectl drain worker --ignore-daemonsets
```

```
node/worker cordoned  
Warning: ignoring DaemonSet-managed Pods: kube-system/cilium-gzdk6, kube-system/kube-proxy-lpsmq  
evicting pod kube-system/cilium-operator-788c7d7585-hc9wf  
evicting pod kube-system/coredns-5d78c9869d-h4p7v  
evicting pod kube-system/coredns-5d78c9869d-d4nv8  
pod/cilium-operator-788c7d7585-hc9wf evicted  
pod/coredns-5d78c9869d-h4p7v evicted  
pod/coredns-5d78c9869d-d4nv8 evicted  
node/worker drained
```

24. Return to the **worker** node and download the updated node configuration.

```
student@worker:~$ sudo kubeadm upgrade node
```

```
[upgrade] Reading configuration from the cluster...  
[upgrade] FYI: You can look at this config file with 'kubectl -n kube-system get cm  
kubeadm-config -o yaml'
```

```
[preflight] Running pre-flight checks
[preflight] Skipping prepull. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[upgrade] The configuration for this node was successfully updated!
[upgrade] Now you should go ahead and upgrade the kubelet package using your package manager.
```

25. Remove the hold on the software then update to the same version as set on the cp.

```
student@worker:~$ sudo apt-mark unhold kubelet kubect1
```

```
Canceled hold on kubelet.
Canceled hold on kubect1.
```

```
student@worker:~$ sudo apt-get install -y kubelet=1.35.2-1.1 kubect1=1.35.2-1.1
```

```
Reading package lists... Done
Building dependency tree
<output_omitted>
Setting up kubect1 (1.35.2-1.1) ...
Setting up kubelet (1.35.2-1.1) ...
```

26. Ensure the packages don't get updated when along with regular updates.

```
student@worker:~$ sudo apt-mark hold kubelet kubect1
```

```
kubelet set on hold.
kubect1 set on hold.
```

27. Restart daemon processes for the software to take effect.

```
student@worker:~$ sudo systemctl daemon-reload
```

```
student@worker:~$ sudo systemctl restart kubelet
```

28. Return to the cp node. View the status of the nodes. Notice the worker status.

```
student@cp:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready	control-plane	118m	v1.35.2
worker	Ready,SchedulingDisabled	<none>	70m	v1.35.2

29. Allow pods to be deployed to the worker node. Remember to use YOUR worker name. TAB can be helpful to enter the name if command line completion is enabled.

```
student@cp:~$ kubectl uncordon worker
```

```
node/worker uncordoned
```

30. Verify the nodes both show a Ready status and the same upgraded version.

```
student@cp:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready	control-plane	119m	v1.35.2
worker	Ready	<none>	71m	v1.35.2

Exercise 4.2: Working with CPU and Memory Constraints

Overview

We will continue working with our cluster, which we built in the previous lab. We will work with `resource limits`, more with `namespaces` and then a complex deployment which you can explore to further understand the architecture and relationships.

Use **SSH** or **PuTTY** to connect to the nodes you installed in the previous exercise. We will deploy an application called **stress** inside a container, and then use `resource limits` to constrain the resources the application has access to use.

1. Use a container called `stress`, in a deployment which we will name `hog`, to generate load. Verify you have the container running.

```
student@cp:~$ kubectl create deployment hog --image vish/stress
```

```
deployment.apps/hog created
```

```
student@cp:~$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hog	1/1	1	1	13s

2. Use the `describe` argument to view details, then view the output in YAML format. Note there are no settings limiting resource usage. Instead, there are empty curly brackets.

```
student@cp:~$ kubectl describe deployment hog
```

```
Name:                hog
Namespace:           default
Labels:              app=hog
Annotations:         deployment.kubernetes.io/revision: 1
<output_omitted>
```

```
student@cp:~$ kubectl get deployment hog -o yaml
```

```
apiVersion: apps/v1
kind: Deployment
Metadata:

<output_omitted>

  template:
    metadata:
      creationTimestamp: null
      labels:
        app: hog
    spec:
      containers:
      - image: vish/stress
        imagePullPolicy: Always
        name: stress
        resources: {}
        terminationMessagePath: /dev/termination-log
    <output_omitted>
```

3. We will use the YAML output to create our own configuration file.

```
student@cp:~$ kubectl get deployment hog -o yaml > hog.yaml
```

4. Probably good to remove the status output, creationTimestamp and other settings. We will also add in memory limits found below.

```
student@cp:~$ vim hog.yaml
```



hog.yaml

```
1  ....
2      imagePullPolicy: Always
3      name: hog
4      resources:                                # Edit to remove {}
5          limits:                                # Add these 4 lines
6              memory: "4Gi"
7          requests:
8              memory: "2500Mi"
9      terminationMessagePath: /dev/termination-log
10     terminationMessagePolicy: File
11  ....
12
```

5. Replace the deployment using the newly edited file.

```
student@cp:~$ kubectl replace -f hog.yaml
```

```
deployment.apps/hog replaced
```

6. Verify the change has been made. The deployment should now show resource limits.

```
student@cp:~$ kubectl get deployment hog -o yaml
```

```
....
  resources:
    limits:
      memory: 4Gi
    requests:
      memory: 2500Mi
  terminationMessagePath: /dev/termination-log
```



```
....
```

7. View the `stdio` of the `hog` container. Note how much memory has been allocated.

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
hog-64cbfcc7cf-lwq66	1/1	Running	0	2m

```
student@cp:~$ kubectl logs hog-64cbfcc7cf-lwq66
```

```
I1102 16:16:42.638972      1 main.go:26] Allocating "0" memory, in
    "4Ki" chunks, with a 1ms sleep between allocations
I1102 16:16:42.639064      1 main.go:29] Allocated "0" memory
```

8. Open a second and third terminal to access both `cp` and second nodes. Run **top** to view resource usage. You should not see unusual resource usage at this point. The **containerd** and **top** processes should be using about the same amount of resources. The **stress** command should not be using enough resources to show up. Use the **kubectl get events** to see if any pods are evicted when too many resources are in use.
9. Edit the `hog` configuration file and add arguments for **stress** to consume CPU and memory. The `args:` entry should be indented the same number of spaces as `resources:`.

```
student@cp:~$ vim hog.yaml
```

`hog.yaml`

```
1  ....
2      resources:
3          limits:
4              cpu: "1"
5              memory: "4Gi"
6          requests:
7              cpu: "0.5"
8              memory: "500Mi"
9          args:
10             - -cpus
11             - "2"
12             - -mem-total
13             - "950Mi"
14             - -mem-alloc-size
15             - "100Mi"
16             - -mem-alloc-sleep
17             - "1s"
18  ....
19
```

10. Delete and recreate the deployment. You should see increased CPU usage almost immediately and memory allocation happen in 100M chunks, allocated to the **stress** program via the running **top** command. Check both nodes as the container could be deployed to either. Be aware that nodes with a small amount of memory or CPU may encounter issues. Symptoms include cp node infrastructure pods failing. Adjust the amount of resources used to allow standard pods to run without error.

```
student@cp:~$ kubectl delete deployment hog
```

```
deployment.apps "hog" deleted
```

```
student@cp:~$ kubectl create -f hog.yaml
```

```
deployment.apps/hog created
```



Only if top does not show high usage

Should the resources not show increased use, there may have been an issue inside of the container. Kubernetes may show it as running, but the actual workload has failed. Or the container may have failed; for example if you were missing a parameter the container may panic.

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
hog-1985182137-5bz2w	0/1	Error	1	5s

```
student@cp:~$ kubectl logs hog-1985182137-5bz2w
```

```
panic: cannot parse '150mi': unable to parse quantity's suffix

goroutine 1 [running]:
panic(0x5ff9a0, 0xc820014cb0)
    /usr/local/go/src/runtime/panic.go:481 +0x3e6
k8s.io/kubernetes/pkg/api/resource.MustParse(0x7ffe460c0e69, 0x5, 0x0, 0x0, 0x0, 0x0, 0x0,
    ↪ 0x0, 0x0)
    /usr/local/google/home/vishnuk/go/src/k8s.io/kubernetes/pkg/api/resource/quantity.go:134
    ↪ +0x287
main.main()
    /usr/local/google/home/vishnuk/go/src/github.com/vishh/stress/main.go:24 +0x43
```



Here is an example of an improper parameter. The container is running, but not allocating memory. It should show the usage requested from the YAML file.

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
hog-1603763060-x3vnn	1/1	Running	0	8s

```
student@cp:~$ kubectl logs hog-1603763060-x3vnn
```

```
I0927 21:09:23.514921      1 main.go:26] Allocating "0" memory, in "4ki" chunks, with a 1ms
→ sleep \
                        between allocations
I0927 21:09:23.514984      1 main.go:39] Spawning a thread to consume CPU
I0927 21:09:23.514991      1 main.go:39] Spawning a thread to consume CPU
I0927 21:09:23.514997      1 main.go:29] Allocated "0" memory
```

Exercise 4.3: Resource Limits for a Namespace

The previous steps set limits for that particular deployment. You can also set limits on an entire namespace. We will create a new namespace and configure another hog deployment to run within. When set hog should not be able to use the previous amount of resources.

1. Begin by creating a new namespace called `low-usage-limit` and verify it exists.

```
student@cp:~$ kubectl create namespace low-usage-limit
```

```
namespace/low-usage-limit created
```

```
student@cp:~$ kubectl get namespace
```

NAME	STATUS	AGE
default	Active	1h
kube-node-lease	Active	1h
kube-public	Active	1h
kube-system	Active	1h
low-usage-limit	Active	42s

2. Create a YAML file which limits CPU and memory usage. The kind to use is `LimitRange`. Remember the file may be found in the example tarball.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_04/low-resource-range.yaml .
```

```
student@cp:~$ vim low-resource-range.yaml
```

YAML**low-resource-range.yaml**

```
1 apiVersion: v1
2 kind: LimitRange
3 metadata:
4   name: low-resource-range
5 spec:
6   limits:
7   - default:
8       cpu: 1
9       memory: 500Mi
```



```
10     defaultRequest:
11         cpu: 0.5
12         memory: 100Mi
13     type: Container
```

3. Create the LimitRange object and assign it to the newly created namespace low-usage-limit. You can use `--namespace` or `-n` to declare the namespace.

```
student@cp:~$ kubectl create -f low-resource-range.yaml -n low-usage-limit
```

```
limitrange/low-resource-range created
```

4. Verify it works. Remember that every command needs a namespace and context to work. Defaults are used if not provided.

```
student@cp:~$ kubectl get LimitRange
```

```
No resources found in default namespace.
```

```
student@cp:~$ kubectl get LimitRange --all-namespaces
```

NAMESPACE	NAME	CREATED AT
low-usage-limit	low-resource-range	2026-03-19T10:23:57Z

5. Create a new deployment in the namespace.

```
student@cp:~$ kubectl -n low-usage-limit \
create deployment limited-hog --image vish/stress
```

```
deployment.apps/limited-hog created
```

6. List the current deployments. Note hog continues to run in the default namespace. If you chose to use the **Cilium** network policy you may see a couple more than what is listed below.

```
student@cp:~$ kubectl get deployments --all-namespaces
```

NAMESPACE	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
default	hog	1/1	1	1	7m57s
kube-system	cilium-operator	1/1	1	1	2d10h
kube-system	coredns	2/2	2	2	2d10h
low-usage-limit	limited-hog	1/1	1	1	9s

7. View all pods within the namespace. Remember you can use the **tab** key to complete the namespace. You may want to type the namespace first so that tab-completion is appropriate to that namespace instead of the default namespace.

```
student@cp:~$ kubectl -n low-usage-limit get pods
```

NAME	READY	STATUS	RESTARTS	AGE
limited-hog-2556092078-wnpnv	1/1	Running	0	2m11s

8. Look at the details of the pod. You will note it has the settings inherited from the entire namespace. The use of shell completion should work if you declare the namespace first.

```
student@cp:~$ kubectl -n low-usage-limit \
  get pod limited-hog-2556092078-wnpnv -o yaml
```

```
<output_omitted>
spec:
  containers:
  - image: vish/stress
```

```
imagePullPolicy: Always
name: stress
resources:
  limits:
    cpu: "1"
    memory: 500Mi
  requests:
    cpu: 500m
    memory: 100Mi
terminationMessagePath: /dev/termination-log
<output_omitted>
```

9. Copy and edit the config file for the original hog file. Add the namespace: line so that a new deployment would be in the low-usage-limit namespace. Delete the selflink line, if it exists.

```
student@cp:~$ cp hog.yaml hog2.yaml
```

```
student@cp:~$ vim hog2.yaml
```

YA
ML

hog2.yaml

```
1  ....
2  labels:
3    app: hog
4    name: hog
5    namespace: low-usage-limit      #<<--- Add this line
6    selfLink: /apis/apps/v1/namespaces/default/deployments/hog
7  spec:
8  ....
9
```


10. Open up extra terminal sessions so you can have **top** running in each. When the new deployment is created it will probably be scheduled on the node not yet under any stress.

Create the deployment.

```
student@cp:~$ kubectl create -f hog2.yaml
```

```
deployment.apps/hog created
```

11. View the deployments. Note there are two with the same name, `hog` but in different namespaces. You may also find the `cilium` deployment has no pods, nor has any requested. Our small cluster does not need to add **Cilium** pods via this autoscaler.

```
student@cp:~$ kubectl get deployments --all-namespaces
```

NAMESPACE	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
default	hog	1/1	1	1	24m
kube-system	cilium-operator	1/1	0	0	4h
kube-system	coredns	2/2	2	2	4h
low-usage-limit	hog	1/1	1	1	26s
low-usage-limit	limited-hog	1/1	1	1	5m11s

12. Look at the **top** output running in other terminals. You should find that both `hog` deployments are using about the same amount of resources, once the memory is fully allocated. Per-deployment settings override the global namespace settings. You should see something like the following lines one from each node, which indicates use of one processor and about 12 percent of your memory, were you on a system with 8G total.

```
25128 root    20   0 958532 954672  3180 R 100.0 11.7   0:52.27 stress
24875 root    20   0 958532 954800  3180 R 100.3 11.7  41:04.97 stress
```

13. Delete the `hog` deployments to recover system resources.

```
student@cp:~$ kubectl -n low-usage-limit delete deployment hog limited-hog
```

```
deployment.apps "hog" deleted  
deployment.apps "limited-hog" deleted
```

```
student@cp:~$ kubectl delete deployment hog
```

```
deployment.apps "hog" deleted
```

Chapter 5

APIs and Access



5.1	API Access	128
5.2	Annotations	132
5.3	Working with A Simple Pod	133
5.4	kubectl and API	134
5.5	Swagger and OpenAPI	141
5.6	Labs	143

5.1 API Access

API Access

- API driven architecture
- API groups
- RESTful style
- Standard HTTP verbs
- Deprecation process now being honored

RESTful

- Responds to typical HTTP verbs (GET, POST, DELETE ...)
- Allows easy interaction with other ecosystems
- Scripting and interaction with deployment tools
- User impersonation headers

Checking Access

- Several ways to authenticate
- `auth can-i` subcommand to query authorization
- Accepts `can-i` and `reconcile` arguments
- More in Security chapter to follow.

Optimistic Concurrency

- Currently leverage JSON
- `resourceVersion`
- Clients must handle 409 CONFLICT Errors

5.2 Annotations

Using Annotations

- Distinct from Labels
- Non-identifying metadata
- Key/value maps
- Metadata otherwise held in exterior databases
- Useful for third-party automation

5.3 Working with A Simple Pod

Simple Pod

- Lowest compute unit of K8s
- Typically multiple containers grouped together
- Created from PodSpec
- Few required, many optional
 - `apiVersion`
Must match existing API group
 - `kind`
The type of object to create
 - `metadata`
At least a name
 - `spec`
What to create and parameters

5.4 kubectl and API

Manage API Resources with kubectl

- API exposed via RESTful interface
- Use **curl** to access and test
- Use verbose mode
- Leverages HTTP verbs

Access From Outside The Cluster

- Can use **curl** from outside cluster
- Must use SSL/TLS for secure access
- Information found in `~/.kube/config`
- View server information via **kubectl config view**

~/.kube/config

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdF.....
    server: https://10.128.0.3:6443
  name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: LS0tLS1CRUdJTib.....
    client-key-data: LS0tLS1CRUdJTi....
```

Namespaces

- **Linux** kernel feature
 - Segregates system resources
 - Core functionality of containers
- API Object
 - Four namespaces to begin with:
 - * default
 - * kube-node-lease
 - * kube-public
 - * kube-system
 - - **all-namespaces**

Working with Namespaces

```
$ kubectl get ns  
$ kubectl create ns linuxcon  
$ kubectl describe ns linuxcon  
$ kubectl get ns/linuxcon -o yaml  
$ kubectl delete ns/linuxcon
```

API Resources with kubectl

- All available via **kubectl**
- `kubectl [command] [type] [Name] [flag]`
- `kubectl help` for more information
- Abbreviated names

Additional Resource Methods

- Various Endpoints
- CLI `--help`
- Online documentation

5.5 Swagger and OpenAPI

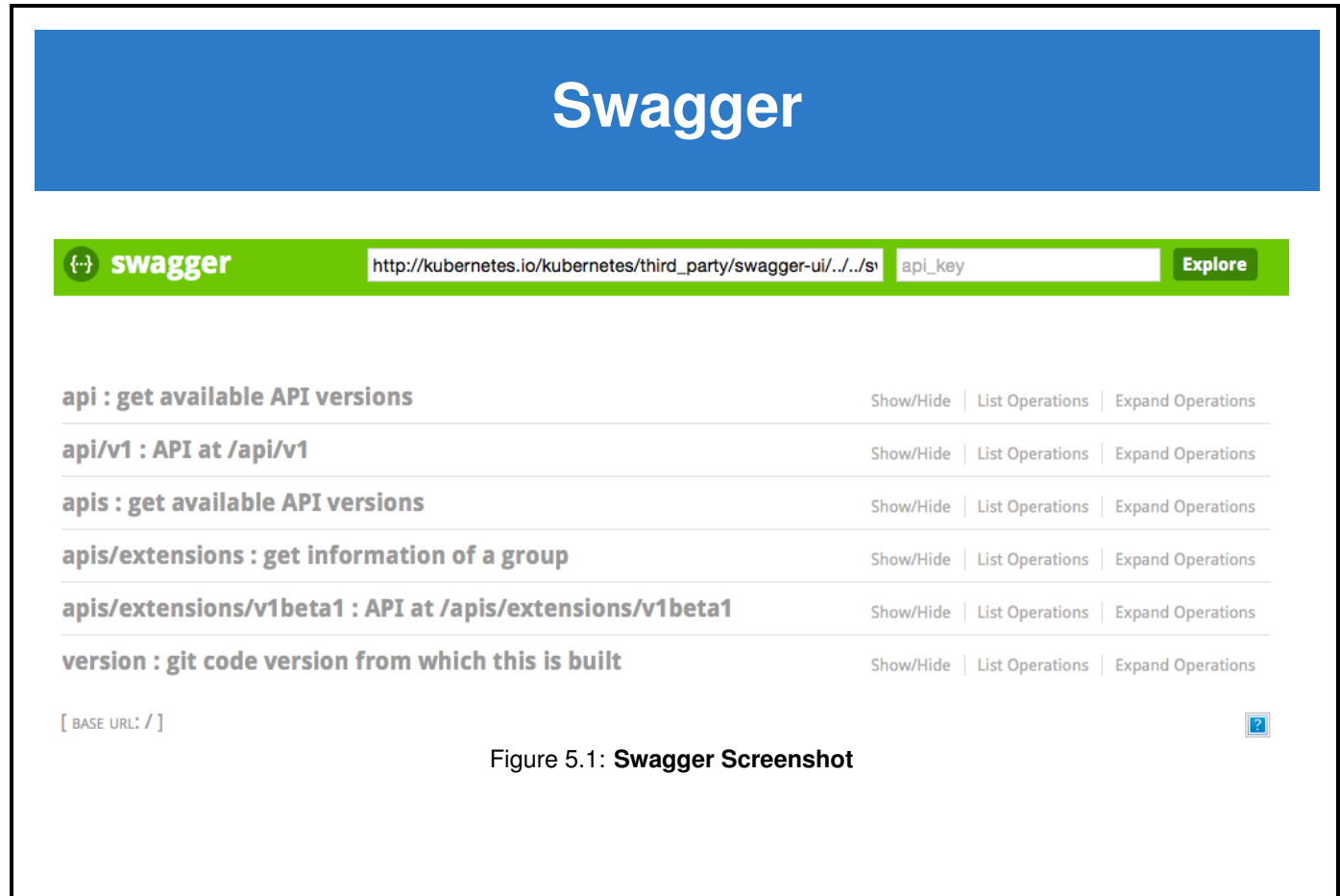


Figure 5.1: Swagger Screenshot

API Maturity

- Versioning of API levels for easier growth
- Not directly tied to software versioning
- Versions imply level of support
 - Alpha
 - Beta
 - Stable

5.6 Labs

Exercise 5.1: Configuring TLS Access

Overview

Using the Kubernetes API, **kubectl** makes API calls for you. With the appropriate TLS keys you could run **curl** as well use a **golang** client. Calls to the `kube-apiserver` get or set a `PodSpec`, or desired state. If the request represents a new state the **Kubernetes Control Plane** will update the cluster until the current state matches the specified state. Some end states may require multiple requests. For example, to delete a `ReplicaSet`, you would first set the number of replicas to zero, then delete the `ReplicaSet`.

An API request must pass information as JSON. **kubectl** converts `.yaml` to JSON when making an API request on your behalf. The API request has many settings, but must include `apiVersion`, `kind` and `metadata`, and `spec` settings to declare what kind of container to deploy. The `spec` fields depend on the object being created.

We will begin by configuring remote access to the `kube-apiserver` then explore more of the API.

1. Begin by reviewing the **kubectl** configuration file. We will use the three certificates and the API server address.

```
student@cp:~$ less $HOME/.kube/config
```

```
<output_omitted>
```

2. We will create a variables using certificate information. You may want to double-check each parameter as you set it. Begin with setting the `client-certificate-data` key.

```
student@cp:~$ export client=$(grep client-cert $HOME/.kube/config |cut -d" " -f 6)
```

```
student@cp:~$ echo $client
```

```
LSOtLS1CRUdJTiBDRVJUSUZJQ0FURSOtLSOtCk1JSUM4akNDQWRxZ0F3SUJ  
BZ0lJRy9wbC9rWEpNdm3RFFZSk1vWklodmNOQVFFTEJRQXdGVEVUTUJFRO  
ExVUUKQXhNS2EzVmlaWEp1WlhSbGN6QWVGdzB4TnpFeU1UTXh0elEyTXpKY  
UZ3MHhPREV5TVRNeE56UTJNe1JhTURReApGekFWQmd0VkJBb1REbk41YzNS  
<output_omitted>
```

3. Almost the same command, but this time collect the `client-key-data` as the `key` variable.

```
student@cp:~$ export key=$(grep client-key-data $HOME/.kube/config | cut -d " " -f 6)  
  
student@cp:~$ echo $key
```

```
<output_omitted>
```

4. Finally set the `auth` variable with the `certificate-authority-data` key.

```
student@cp:~$ export auth=$(grep certificate-authority-data $HOME/.kube/config | cut -d " " -f 6)  
  
student@cp:~$ echo $auth
```

```
<output_omitted>
```

5. Now encode the keys for use with **curl**.

```
student@cp:~$ echo $client | base64 -d - > ./client.pem  
  
student@cp:~$ echo $key | base64 -d - > ./client-key.pem  
  
student@cp:~$ echo $auth | base64 -d - > ./ca.pem
```

6. Pull the API server URL from the config file. Your hostname or IP address may be different.

```
student@cp:~$ kubectl config view |grep server
```

```
server: https://k8scp:6443
```

7. Use **curl** command and the encoded keys to connect to the API server. Use your hostname, or IP, found in the previous command, which may be different than the example below.

```
student@cp:~$ curl --cert ./client.pem \
--key ./client-key.pem \
--cacert ./ca.pem \
https://k8scp:6443/api/v1/pods
```

```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods",
    "resourceVersion": "239414"
  },
  <output_omitted>
```

8. If the previous command was successful, create a JSON file to create a new pod. Remember to use **find** and search for this file in the tarball output, it can save you some typing.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_05/curlpod.json .
```

```
student@cp:~$ vim curlpod.json
```

```
{
  "kind": "Pod",
```

```

    "apiVersion": "v1",
    "metadata": {
      "name": "curlpod",
      "namespace": "default",
      "labels": {
        "name": "examplepod"
      }
    },
    "spec": {
      "containers": [{
        "name": "nginx",
        "image": "nginx",
        "ports": [{"containerPort": 80}]
      }]
    }
  }
}

```

9. The previous **curl** command can be used to build a XPOST API call. There will be a lot of output, including the scheduler and taints involved. Read through the output. In the last few lines the phase will probably show Pending, as it's near the beginning of the creation process.

```

student@cp:~$ curl --cert ./client.pem \
  --key ./client-key.pem --cacert ./ca.pem \
  https://k8scp:6443/api/v1/namespaces/default/pods \
  -XPOST -H'Content-Type: application/json' \
  -d@curlpod.json

```

```

{
  "kind": "Pod",
  "apiVersion": "v1",
  "metadata": {
    "name": "curlpod",
    <output_omitted>
  }
}

```

10. Verify the new pod exists and shows a Running status.

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
curlpod	1/1	Running	0	45s

Exercise 5.2: Explore API Calls

1. One way to view what a command does on your behalf is to use **strace**. In this case, we will look for the current endpoints, or targets of our API calls. Install the tool, if not present.

```
student@cp:~$ sudo apt-get install -y strace
```

```
student@cp:~$ kubectl get endpoints
```

NAME	ENDPOINTS	AGE
kubernetes	10.128.0.3:6443	3h

2. Run this command again, preceded by **strace**. You will get a lot of output. Near the end you will note several **openat** functions to a local directory, `/home/student/.kube/cache/discovery/k8scp_6443`. If you cannot find the lines, you may want to redirect all output to a file and **grep** for them. This information is cached, so you may see some differences should you run the command multiple times. As well your IP address may be different.

```
student@cp:~$ strace kubectl get endpoints
```

```
execve("/usr/bin/kubectl", ["kubectl", "get", "endpoints"], [/*....  
....  
openat(AT_FDCWD, "/home/student/.kube/cache/discovery/k8scp_6443..  
<output_omitted>
```

3. Change to the parent directory and explore. Your endpoint IP will be different, so replace the following with one suited to your system.

```
student@cp:~$ cd /home/student/.kube/cache/discovery/
```

```
student@cp:~/.kube/cache/discovery$ ls
```

```
k8scp_6443
```

```
student@cp:~/.kube/cache/discovery$ cd k8scp_6443/
```

4. View the contents. You will find there are directories with various configuration information for kubernetes.

```
student@cp:~/.kube/cache/discovery/k8scp_6443$ ls
```

```
admissionregistration.k8s.io  certificates.k8s.io      node.k8s.io
apiextensions.k8s.io          coordination.k8s.io      policy
apiregistration.k8s.io       cilium.io               rbac.authorization.k8s.io
apps                          discovery.k8s.io         scheduling.k8s.io
authentication.k8s.io        events.k8s.io           servergroups.json
authorization.k8s.io         extensions               storage.k8s.io
autoscaling                  flowcontrol.apiserver.k8s.io v1
batch                        networking.k8s.io
```

5. Use the find command to list out the subfiles. The prompt has been modified to look better on this page.

```
student@cp:./k8scp_6443$ find .
```

```
./storage.k8s.io
./storage.k8s.io/v1beta1
./storage.k8s.io/v1beta1/serverresources.json
./storage.k8s.io/v1
```



```
./storage.k8s.io/v1/serverresources.json
./rbac.authorization.k8s.io
<output_omitted>
```

6. View the objects available in version 1 of the API. For each object, or kind:, you can view the verbs or actions for that object, such as create seen in the following example. Note the prompt has been truncated for the command to fit on one line. Some are HTTP verbs, such as GET, others are product specific options, not standard HTTP verbs. The command may be **python**, depending on what version is installed.

```
student@cp:.$ python3 -m json.tool v1/serverresources.json
```

A small teal square icon with the word "JSON" in white text.

serverresources.json

```
1 {
2   "apiVersion": "v1",
3   "groupVersion": "v1",
4   "kind": "APIResourceList",
5   "resources": [
6     {
7       "kind": "Binding",
8       "name": "bindings",
9       "namespaced": true,
10      "singularName": "",
11      "verbs": [
12        "create"
13      ]
14    },
15    <output_omitted>
16  ]
}
```

7. Some of the objects have shortNames, which makes using them on the command line much easier. Locate the

shortName for endpoints.

```
student@cp:.$ python3 -m json.tool v1/serverresources.json | less
```

JSON

serverresources.json

```
1  ....
2  {
3    "kind": "Endpoints",
4    "name": "endpoints",
5    "namespaced": true,
6    "shortNames": [
7      "ep"
8    ],
9    "singularName": "",
10   "verbs": [
11     "create",
12     "delete",
13     ....
14
```

8. Use the shortName to view the endpoints. It should match the output from the previous command.

```
student@cp:.$ kubectl get EndpointSlice
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
kubernetes	IPv4	6443	10.2.0.31	3h

9. We can see there are 37 objects in version 1 file.

```
student@cp:.$ python3 -m json.tool v1/serverresources.json | grep kind
```

```
"kind": "APIResourceList",  
  "kind": "Binding",  
  "kind": "ComponentStatus",  
  "kind": "ConfigMap",  
  "kind": "Endpoints",  
  "kind": "Event",  
<output_omitted>
```

10. Looking at another file we find nine more.

```
student@cp:$ python3 -m json.tool apps/v1/serverresources.json | grep kind
```

```
"kind": "APIResourceList",  
  "kind": "ControllerRevision",  
  "kind": "DaemonSet",  
  "kind": "DaemonSet",  
  "kind": "Deployment",  
<output_omitted>
```

11. Delete the curlpod to recoup system resources.

```
student@cp:$ kubectl delete po curlpod
```

```
pod "curlpod" deleted
```

12. Take a look around the other files in this directory as time permits.

Chapter 6

API Objects



6.1	API Objects	154
6.2	The v1 Group	155
6.3	API Resources	157
6.4	RBAC APIs	162
6.5	Labs	163

6.1 API Objects

Overview

- Ongoing growth in API objects
- Track release notes to find new objects
- In this chapter we will introduce common API objects:
 - Deployment the typical object used
 - DaemonSets, ReplicaSets now apps/v1
 - StatefulSets (once called PetSets) part of apps/v1 since v1.9
 - Jobs and CronJob now batch/v1
 - RBAC moved from v1alpha1 all the way to v1 in one release
- Explain which API group contains these new API objects.
- Find additional resources to start using the new API objects.

6.2 The v1 Group

v1 API Group

- Pod
- Node
- Service Account
- Resource Quota
- Endpoint
- More added with each release

Discovering API Groups

```
$ curl https://localhost:6443/apis \
  --header "Authorization: Bearer $token" -k

{
  "kind": "APIGroupList",
  "apiVersion": "v1",
  "groups": [
    {
      "name": "apiregistration.k8s.io",
      "versions": [
        {
          "groupVersion": "apiregistration.k8s.io/v1",
          "version": "v1"
        }
      ],
      "preferredVersion": {
        "groupVersion": "apiregistration.k8s.io/v1",
        "version": "v1"
      }
    }
  ]
}
```


6.3 API Resources

Deploying an Application

- Deployment
- ReplicaSet
- Pod

DaemonSets

- Ensures every node runs a single pod
- Similar to ReplicaSet
- Often used for logging, metrics and security pods.
- Can be configured to avoid nodes

StatefulSet

- Similar to Deployment
- Ensures unique pods
- Guarantee ordering

Autoscaling

- Agents which add or remove resources from the cluster.
- Horizontal Pod Autoscaling (HPA)
 - Scale based on current CPU usage, or custom metric
 - Must have **Metrics Server** or custom component running
- Vertical Pod Autoscaler
- Cluster Autoscaler (CA)
 - Add or remove nodes based on utilization
 - Makes request to cloud provider
 - Pods which cannot be evicted prevent scale-down

Jobs

- Part of Batch API group
- Jobs run Pod until number of completions reached
 - Batch processing or one-off Pods
 - Ensure specified number of pods successfully terminate
 - Can run multiple Pods in parallel
- Cronjob to run Job on regular basis
 - Creates a Job about once per executing time
 - Some issues, job should be idempotent
 - Can run in serial or parallel
 - Same time syntax as Linux cron job

6.4 RBAC APIs

RBAC

- `rbac.authorization.k8s.io`
- Provide resources
 - `ClusterRole`
 - `ClusterRoleBinding`
 - `RoleBinding`
 - `Role`
- Often combined with quotas for typical production deployments

6.5 Labs

Exercise 6.1: RESTful API Access

Overview

We will continue to explore ways of accessing the control plane of our cluster. In the security chapter we will discuss there are several authentication methods, one of which is use of a Bearer token We will work with one then deploy a local proxy server for application-level access to the Kubernetes API.

We will use the **curl** command to make API requests to the cluster, in an insecure manner. Once we know the IP address and port, then the token we can retrieve cluster data in a RESTful manner. By default most of the information is restricted, but changes to authentication policy could allow more access.

1. First we need to know the IP and port of a node running a replica of the API server. The cp system will typically have one running. Use **kubectl config view** to get overall cluster configuration, and find the server entry. This will give us both the IP and the port.

```
student@cp:~$ kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://k8scp:6443
    name: kubernetes
  <output_omitted>
```

2. The creation of token secrets by Kubernetes is no longer automatic in recent releases. It then falls on the user to design them as desired. Use the command shown below to create the token.

```
student@cp:~$ export token=$(kubectl create token default)
```

3. Test to see if you can get basic API information from your cluster. We will pass it the server name and port, the token and use the **-k** option to avoid using a cert.

```
student@cp:~$ curl https://k8scp:6443/apis --header "Authorization: Bearer $token" -k
```

```
{
  "kind": "APIGroupList",
  "apiVersion": "v1",
  "groups": [
    {
      "name": "apiregistration.k8s.io",
      "versions": [
        {
          "groupVersion": "apiregistration.k8s.io/v1",
          "version": "v1"
        }
      ]
    }
  ]
}
<output_omitted>
```

4. Try the same command, but look at API v1. Note that the path has changed to api.

```
student@cp:~$ curl https://k8scp:6443/api/v1 --header "Authorization: Bearer $token" -k
```

```
<output_omitted>
```

5. Now try to get a list of namespaces. This should return an error. It shows our request is being seen as `systemserviceaccount`, which does not have the RBAC authorization to list all namespaces in the cluster.

```
student@cp:~$ curl \
https://k8scp:6443/api/v1/namespaces --header "Authorization: Bearer $token" -k
```

```
<output_omitted>
"message": "namespaces is forbidden: User \"system:serviceaccount:default...
<output_omitted>
```


Exercise 6.2: Using the Proxy

Another way to interact with the API is via a **proxy**. The proxy can be run from a node or from within a Pod through the use of a sidecar. In the following steps we will deploy a proxy listening to the loopback address. We will use **curl** to access the API server. If the **curl** request works, but does not from outside the cluster, we have narrowed down the issue to authentication and authorization instead of issues further along the API ingestion process.

1. Begin by starting the proxy. It will start in the foreground by default. There are several options you could pass. Begin by reviewing the help output.

```
student@cp:~$ kubectl proxy -h
```

```
Creates a proxy server or application-level gateway between localhost
and the Kubernetes API Server. It also allows serving static content
over specified HTTP path. All incoming data enters through one port
and gets forwarded to the remote kubernetes API Server port, except
for the path matching the static content path.
```

Examples:

```
# To proxy all of the kubernetes api and nothing else, use:
```

```
$ kubectl proxy --api-prefix=/
<output_omitted>
```

2. Start the proxy while setting the API prefix, and put it in the background. You may need to use **enter** to view the prompt. Take note of the process ID, 225000 in the example below, we'll use it to kill the process when we are done.

```
student@cp:~$ kubectl proxy --api-prefix=/ &
```

```
[1] 22500
Starting to serve on 127.0.0.1:8001
```

3. Now use the same **curl** command, but point toward the IP and port shown by the proxy. The output should be the same as without the proxy, but may be formatted differently.

```
student@cp:~$ curl http://127.0.0.1:8001/api/
```

```
<output_omitted>
```

4. Make an API call to retrieve the namespaces. The command did not work in the previous section due to permissions, but should work now as the proxy is making the request on your behalf.

```
student@cp:~$ curl http://127.0.0.1:8001/api/v1/namespaces
```

```
{
  "kind": "NamespaceList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/namespaces",
    "resourceVersion": "86902"
  }
}<output_omitted>
```

5. Stop the proxy service as we won't need it any more. Use the process ID from a previous step. Your process ID may be different.

```
student@cp:~$ kill 22500
```

Exercise 6.3: Working with Jobs

While most API objects are deployed such that they continue to be available there are some which we may want to run a particular number of times called a `Job`, and others on a regular basis called a `CronJob`

Create A Job

1. Create a job which will run a container which sleeps for three seconds then stops.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_06/job.yaml .
```

```
student@cp:~$ vim job.yaml
```

Y
A
M
L

job.yaml

```
1  apiVersion: batch/v1
2  kind: Job
3  metadata:
4    name: sleepy
5  spec:
6    template:
7      spec:
8        containers:
9          - name: resting
10            image: busybox
11            command: ["/bin/sleep"]
12            args: ["3"]
13            restartPolicy: Never
14
```

2. Create the job, then verify and view the details. The example shows checking the job three seconds in and then again after it has completed. You may see different output depending on how fast you type.

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get job
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	0/1	3s	3s

```
student@cp:~$ kubectl describe jobs.batch sleepy
```

```
Name:          sleepy
Namespace:     default
Selector:      controller-uid=24c91245-d0fb-11e8-947a-42010a800002
Labels:        controller-uid=24c91245-d0fb-11e8-947a-42010a800002
               job-name=sleepy
Annotations:   <none>
Parallelism:   1
Completions:   1
Start Time:    Thu, 19 Mar 2026 10:47:53 +0000
Completed At:  Thu, 19 Mar 2026 10:48:00 +0000
Duration:      5s
Pods Statuses: 0 Running / 1 Succeeded / 0 Failed
<output_omitted>
```

```
student@cp:~$ kubectl get job
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	1/1	5s	17s

3. View the configuration information of the job. There are three parameters we can use to affect how the job runs. Use **-o yaml** to see these parameters. We can see that `backoffLimit`, `completions`, and the `parallelism`. We'll add these parameters next.

```
student@cp:~$ kubectl get jobs.batch sleepy -o yaml
```

```
<output_omitted>
  uid: c2c3a80d-d0fc-11e8-947a-42010a800002
spec:
  backoffLimit: 6
  completions: 1
  parallelism: 1
  selector:
    matchLabels:
  <output_omitted>
```

4. As the job continues to AGE in a completion state, delete the job.

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

5. Edit the YAML and add the completions: parameter and set it to 5.

```
student@cp:~$ vim job.yaml
```

YA
ML

job.yaml

```
1 <output_omitted>
2 metadata:
3   name: sleepy
4 spec:
5   completions: 5  #<--Add this line
6   template:
```



```

7     spec:
8       containers:
9     <output_omitted>
10

```

6. Create the job again. As you view the job note that COMPLETIONS begins as zero of 5.

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get jobs.batch
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	0/5	5s	5s

7. View the pods that running. Again the output may be different depending on the speed of typing.

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
sleepy-z5tnh	0/1	Completed	0	8s
sleepy-zd692	1/1	Running	0	3s
<output_omitted>				

8. Eventually all the jobs will have completed. Verify then delete the job.

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	5/5	26s	10m

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

9. Edit the YAML again. This time add in the `parallelism:` parameter. Set it to **2** such that two pods at a time will be deployed.

```
student@cp:~$ vim job.yaml
```

YAML

job.yaml

```
1 <output_omitted>
2   name: sleepy
3 spec:
4   completions: 5
5   parallelism: 2  #<-- Add this line
6   template:
7     spec:
8 <output_omitted>
9
```

10. Create the job again. You should see the pods deployed two at a time until all five have completed.

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
sleepy-8xwpc	1/1	Running	0	5s
sleepy-xjqnf	1/1	Running	0	5s
<output_omitted>				

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	3/5	11s	11s

11. Add a parameter which will stop the job after a certain number of seconds. Set the `activeDeadlineSeconds`: to 15. The job and all pods will end once it runs for 15 seconds. We will also increase the sleep argument to five, just to be sure does not expire by itself.

```
student@cp:~$ vim job.yaml
```

YAML

```
<output_omitted>
1  completions: 5
2  parallelism: 2
3  activeDeadlineSeconds: 15  #<-- Add this line
4  template:
5    spec:
6      containers:
7        - name: resting
8
```




```
9      image: busybox
10     command: ["/bin/sleep"]
11     args: ["5"]           #<-- Edit this line
12 <output_omitted>
13
```

12. Delete and recreate the job again. It should run for 15 seconds, usually 3/5, then continue to age without further completions.

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	1/5	6s	6s

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy	3/5	16s	16s

13. View the message: entry in the Status section of the object YAML output.

```
student@cp:~$ kubectl get job sleepy -o yaml
```

```
<output_omitted>
status:
  conditions:
  - lastProbeTime: 2026-03-19T10:48:00Z
    lastTransitionTime: 2026-03-19T10:48:00Z
    message: Job was active longer than specified deadline
    reason: DeadlineExceeded
    status: "True"
    type: Failed
  failed: 2
  startTime: 2026-03-19T10:48:00Z
  succeeded: 3
```

14. Delete the job.

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

Create a CronJob

A CronJob creates a watch loop which will create a batch job on your behalf when the time becomes true. We Will use our existing Job file to start.

1. Copy the yaml file from the tarball.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_06/cronjob.yaml .
```

2. Verify the file to look like the annotated file shown below. Edit the lines mentioned below if needed. The three parameters we added will need to be removed. Other lines will need to be further indented if needed.

```
student@cp:~$ vim cronjob.yaml
```

YAML

```
1 apiVersion: batch/v1
2 kind: CronJob                                #<-- Update this line to CronJob
3 metadata:
4   name: sleepy
5 spec:
6   schedule: "*/2 * * * *"                  #<-- Add Linux style cronjob syntax
7   jobTemplate:                             #<-- New jobTemplate and spec move
8     spec:
9       template:                             #<-- This and following lines move
10         spec:                               #<-- four spaces to the right
11           containers:
12             - name: resting
13               image: busybox
14               command: ["/bin/sleep"]
15               args: ["5"]
16             restartPolicy: Never
17
```

3. Create the new CronJob. View the jobs. It will take two minutes for the CronJob to run and generate a new batch Job.

```
student@cp:~$ kubectl create -f cronjob.yaml
```

```
cronjob.batch/sleepy created
```

```
student@cp:~$ kubectl get cronjobs.batch
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
sleepy	*/* * * * *	False	0	<none>	8s

```
student@cp:~$ kubectl get jobs.batch
```

No resources found.

- After two minutes you should see jobs start to run.

```
student@cp:~$ kubectl get cronjobs.batch
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
sleepy	*/* * * * *	False	0	21s	2m1s

```
student@cp:~$ kubectl get jobs.batch
```

NAME	COMPLETIONS	DURATION	AGE
sleepy-1539722040	1/1	5s	18s

```
student@cp:~$ kubectl get jobs.batch
```

NAME	COMPLETIONS	DURATION	AGE
sleepy-1539722040	1/1	5s	5m17s
sleepy-1539722160	1/1	6s	3m17s
sleepy-1539722280	1/1	6s	77s

5. Ensure that if the job continues for more than 10 seconds it is terminated. We will first edit the **sleep** command to run for 30 seconds then add the `activeDeadlineSeconds`: entry to the container.

```
student@cp:~$ vim cronjob.yaml
```

YAML

```
1  ....
2  jobTemplate:
3    spec:
4      activeDeadlineSeconds: 10  #<-- Add this line
5      template:
6        spec:
7          containers:
8            - name: resting
9  ....
10     command: ["/bin/sleep"]
11     args: ["30"]                #<-- Edit this line
12     restartPolicy: Never
13  ....
14
```

6. Delete and recreate the CronJob. It may take a couple of minutes for the batch Job to be created and terminate due to the timer.

```
student@cp:~$ kubectl delete cronjobs.batch sleepy
```

```
cronjob.batch "sleepy" deleted
```

```
student@cp:~$ kubectl create -f cronjob.yaml
```

```
cronjob.batch/sleepy created
```

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy-1539723240	0/1	61s	61s

```
student@cp:~$ kubectl get cronjobs.batch
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
sleepy	*/* * * * *	False	1	72s	94s

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy-1539723240	0/1	75s	75s

```
student@cp:~$ kubectl get jobs
```

NAME	COMPLETIONS	DURATION	AGE
sleepy-1539723240	0/1	2m19s	2m19s
sleepy-1539723360	0/1	19s	19s

```
student@cp:~$ kubectl get cronjobs.batch
```

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
sleepy	*/* * * * *	False	2	31s	2m53s

7. Clean up by deleting the CronJob.

```
student@cp:~$ kubectl delete cronjobs.batch sleepy
```

```
cronjob.batch "sleepy" deleted
```


Chapter 7

Managing State With Deployments



7.1	Deployment Overview	182
7.2	Deployments and Replica Sets	183
7.3	DaemonSets	191
7.4	Labels	192
7.5	Labs	194

7.1 Deployment Overview

Overview

- Deployments
- Application Updates
- Labels
- ReplicaSet

7.2 Deployments and Replica Sets

Deployment Details

- Generate Deployment

```
$ kubectl create deployment dev-web --image=nginx:1.21
```

```
deployment "dev-web" created
```

- Generate YAML of newly created objects

```
kubectl get deployments,rs,pods -o yaml
```

- Sometimes JSON output can make it more clear

```
kubectl get deployments,rs,pods -o json
```

```
apiVersion: v1
items:
- apiVersion: apps/v1
  kind: Deployment
```

Deployment Configuration Metadata

```
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: 2026-03-19T13:57:07Z
  generation: 1
  labels:
    app: dev-web
  name: dev-web
  namespace: default
  resourceVersion: "774003"
  uid: d52d3a63-e656-11e7-9319-42010a800003
```

Deployment Configuration Spec

```
spec:
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: dev-web
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
```

Deployment Configuration Pod Template

```
template:
  metadata:
    creationTimestamp: null
    labels:
      app: dev-web
  spec:
    containers:
      - image: nginx:1.17.7-alpine
        imagePullPolicy: IfNotPresent
        name: dev-web
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
    dnsPolicy: ClusterFirst
    restartPolicy: Always
    schedulerName: default-scheduler
    securityContext: {}
    terminationGracePeriodSeconds: 30
```

Deployment Configuration Status

```
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: 2026-03-19T13:57:07Z
    lastUpdateTime: 2026-03-19T13:57:07Z
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2026-03-19T06:00:24Z"
    lastUpdateTime: "2026-03-19T06:00:33Z"
    message: ReplicaSet "test-5f6778868d" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  observedGeneration: 2
  readyReplicas: 2
  replicas: 2
  updatedReplicas: 2
```

Scaling and Rolling Updates

- Deployment configuration can be dynamically updated Controllers
- Use `set` argument to create new Replica Set
- Also use `edit` to trigger update
- Update YAML file and use `kubectl apply`

Deployment Rollbacks

- Previous ReplicaSets retained for rollback
- Can pause and resume
- Deployment edits replica counts decrementing old and incrementing new `replicaSet`

Deployment Rollbacks (cont.)

```
$ kubectl rollout pause deployment/ghost
```

```
$ kubectl rollout resume deployment/ghost
```

7.3 DaemonSets

Using DaemonSets

- Runs on every node
- Same image on each
- Added and removed dynamically
- Use kind: `DaemonSet`

7.4 Labels

Labels

- Can exist in every resource metadata
- Hash label created by default
- Immutable as of API version apps/v1

Labels (cont.)

- During creation or on the fly
- Easy to query or select

7.5 Labs

Exercise 7.1: Working with ReplicaSets

Overview

Understanding and managing the state of containers is a core Kubernetes task. In this lab we will first explore the API objects used to manage groups of containers. The objects available have changed as Kubernetes has matured, so the Kubernetes version in use will determine which are available. Our first object will be a `ReplicaSet`, which does not include newer management features found with `Deployments`. A `Deployment` operator manages `ReplicaSet` operators for you. We will also work with another object and watch loop called a `DaemonSet` which ensures a container is running on newly added node.

Then we will update the software in a container, view the revision history, and roll-back to a previous version.

A `ReplicaSet` is a next-generation of a `Replication Controller`, which differs only in the selectors supported. The only reason to use a `ReplicaSet` anymore is if you have no need for updating container software or require update orchestration which won't work with the typical process.

1. View any current `ReplicaSets`. If you deleted resources at the end of a previous lab, you should have none reported in the default namespace.

```
student@cp:~$ kubectl get rs
```

```
No resources found in default namespace.
```

2. Create a YAML file for a simple `ReplicaSet`. The `apiVersion` setting depends on the version of Kubernetes you are using. The object is stable using the `apps/v1` `apiVersion`. We will use an older version of **nginx** then update to a newer version later in the exercise.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_07/rs.yaml .
```

```
student@cp:~$ vim rs.yaml
```

**YA
ML****rs.yaml**

```
1  apiVersion: apps/v1
2  kind: ReplicaSet
3  metadata:
4    name: rs-one
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        system: ReplicaOne
10   template:
11     metadata:
12       labels:
13         system: ReplicaOne
14     spec:
15       containers:
16       - name: nginx
17         image: nginx:1.22.1
18         ports:
19         - containerPort: 80
```

3. Create the ReplicaSet:

```
student@cp:~$ kubectl create -f rs.yaml
```

```
replicaset.apps/rs-one created
```

4. View the newly created ReplicaSet:

```
student@cp:~$ kubectl describe rs rs-one
```

```
Name:          rs-one
Namespace:     default
Selector:      system=ReplicaOne
Labels:        <none>
Annotations:   <none>
Replicas:      2 current / 2 desired
Pods Status:   2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:      system=ReplicaOne
  Containers:
    nginx:
      Image:      nginx:1.22.1
      Port:       80/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:     <none>
      Volumes:    <none>
  Events:      <none>
```

5. View the Pods created with the ReplicaSet. From the yaml file created there should be two Pods. You may see a Completed busybox which will be cleared out eventually.

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
rs-one-2p9x4	1/1	Running	0	5m4s
rs-one-3c6pb	1/1	Running	0	5m4s

6. Now we will delete the ReplicaSet, but not the Pods it controls.

```
student@cp:~$ kubectl delete rs rs-one --cascade=orphan
```

```
replicaset.apps "rs-one" deleted
```

7. View the ReplicaSet and Pods again:

```
student@cp:~$ kubectl describe rs rs-one
```

```
Error from server (NotFound): replicaset.apps "rs-one" not found
```

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
rs-one-2p9x4	1/1	Running	0	7m
rs-one-3c6pb	1/1	Running	0	7m

8. Create the ReplicaSet again. As long as we do not change the selector field, the new ReplicaSet should take ownership. Pod software versions cannot be updated this way.

```
student@cp:~$ kubectl create -f rs.yaml
```

```
replicaset.apps/rs-one created
```

9. View the age of the ReplicaSet and then the Pods within:

```
student@cp:~$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
rs-one	2	2	2	46s

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
rs-one-2p9x4	1/1	Running	0	8m
rs-one-3c6pb	1/1	Running	0	8m

10. We will now isolate a Pod from its ReplicaSet. Begin by editing the label of a Pod. We will change the `system:` parameter to be `IsolatedPod`.

```
student@cp:~$ kubectl edit pod rs-one-3c6pb
```

```
....
  labels:
    system: IsolatedPod  #<-- Change from ReplicaOne
managedFields:
....
```

11. View the number of pods within the ReplicaSet. You should see two running.

```
student@cp:~$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
rs-one	2	2	2	4m

12. Now view the pods with the label key of `system`. You should note that there are three, with one being newer than others. The ReplicaSet made sure to keep two replicas, replacing the Pod which was isolated.

```
student@cp:~$ kubectl get po -L system
```

NAME	READY	STATUS	RESTARTS	AGE	SYSTEM
rs-one-3c6pb	1/1	Running	0	10m	IsolatedPod
rs-one-2p9x4	1/1	Running	0	10m	ReplicaOne
rs-one-dq5xd	1/1	Running	0	30s	ReplicaOne

13. Delete the ReplicaSet, then view any remaining Pods.

```
student@cp:~$ kubectl delete rs rs-one
```

```
replicaset.apps "rs-one" deleted
```

```
student@cp:~$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
rs-one-3c6pb	1/1	Running	0	14m
rs-one-dq5xd	0/1	Terminating	0	4m

14. In the above example the Pods had not finished termination. Wait for a bit and check again. There should be no ReplicaSets, but one Pod.

```
student@cp:~$ kubectl get rs
```

```
No resources found in default namespaces.
```

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
rs-one-3c6pb	1/1	Running	0	16m

15. Delete the remaining Pod using the label.

```
student@cp:~$ kubectl delete pod -l system=IsolatedPod
```

```
pod "rs-one-3c6pb" deleted
```

Exercise 7.2: Working with Deployments

A Deployment is a watch loop object which we have been working with in the previous labs. A Deployment provides a declarative update to Pods and ReplicaSets and ensure a particular number of pods are created in general, several could be on a single node. Deployment is a high-level resource object that is used to manage the rollout and scaling of containerized applications. A Deployment describes the desired state of the application, such as the number of replicas, and the container image to use. When a Deployment is created, Kubernetes will automatically create and manage the necessary replica sets, which in turn will create and manage the necessary pods to ensure that the desired state of the application is met. Deployment also provide rolling updates, which allow for updating an application to a new version without downtime by gradually replacing the old replicas with new ones. Using Deployment in Kubernetes makes it easy to manage and scale containerized applications while ensuring high availability and reliability.

1. We begin by creating a yaml file. In this case the kind would be set to deployment. We can generate the yaml file using the imperative method

```
student@cp:~$ kubectl create deploy webserver --image nginx:1.22.1 --replicas=2 \
--dry-run=client -o yaml | tee dep.yaml
```



dep.yaml

```
1 ....
2 kind: Deployment
3 ....
4 name: webserver
5 ....
6 replicas: 2
7 ....
8     app: webserver
9 ....
10
```

2. Create and verify the newly formed Deployment. There should be two replicas of Pods created in the cluster.

```
student@cp:~$ kubectl create -f dep.yaml
```

```
deployment.apps/webserver created
```

```
student@cp:~$ kubectl get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
webserver	2/2	2	2	14s

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
webserver-6cbc654ddc-lssbm	1/1	Running	0	42s
webserver-6cbc654ddc-xpmtl	1/1	Running	0	42s

3. Verify the image running inside the Pods. We will use this information in the next section.

```
student@cp:~$ kubectl describe pod webserver-6cbc654ddc-lssbm | grep Image:
```

```
Image:          nginx:1.22.1
```

Exercise 7.3: Rollout and Rollback using Deployment

One of the advantages of micro-services is the ability to replace and upgrade a container while continuing to respond to client requests. We will use the `recreate` setting that upgrades a container when the predecessor is deleted, then the use the `RollingUpdate` feature as well, which begins a rolling update immediately.



nginx versions

The **nginx** software updates on a distinct timeline from Kubernetes. If the lab shows an older version please use the current default, and then a newer version. Versions can be verified on the repositories on the registry

1. Begin by viewing the current strategy setting for the Deployment created in the previous section.

```
student@cp:~$ kubectl get deploy webserver -o yaml | grep -A 4 strategy
```

```
strategy:
rollingUpdate:
  maxSurge: 25%
  maxUnavailable: 25%
type: RollingUpdate
```

2. Edit the object to use the `Recreate` update strategy. This would allow the manual termination of some of the pods, resulting in an updated image when they are recreated.

```
student@cp:~$ kubectl edit deploy webserver
```

```
....
strategy:
  rollingUpdate:          # <-- remove this line
    maxSurge: 25%         # <-- remove this line
    maxUnavailable: 25%   # <-- remove this line
  type: Recreate          # <-- Edit this line
:q....
```

3. Update the Deployment to use a newer version of the **nginx** server. This time use the **set** command instead of **edit**. Set the version to be `1.23.1-alpine`.

```
student@cp:~$ kubectl set image deploy webserver nginx=nginx:1.23.1-alpine --record
```

```
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/webserver image updated
```

4. Verify that the `Image:` parameter for the Pod checked in the previous section is unchanged.

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
webserver-6cf9cd5c74-qjph4	1/1	Running	0	35s
webserver-6cf9cd5c74-zc6x9	1/1	Running	0	35s

```
student@cp:~$ kubectl describe po webserver-6cf9cd5c74-qjph4 |grep Image:
```

```
Image:          nginx:1.23.1-alpine
```

5. View the history of changes for the Deployment. You should see two revisions listed. As we did not add the `change-cause` annotation we didn't see why the object updated.

```
student@cp:~$ kubectl rollout history deploy webserver
```

```
deployment.apps/webserver
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl set image deploy webserver nginx=nginx:1.23.1-alpine --record=true
```

6. View the settings for the various versions of the Deployment. The `Image:` line should be the only difference between the two outputs.

```
student@cp:~$ kubectl rollout history deploy webserver --revision=1
```

```
deployment.apps/webserver with revision #1
Pod Template:
  Labels:          app=webserver
                  pod-template-hash=6cbc654ddc
  Containers:
    nginx:
      Image:        nginx:1.22.1
      Port:         <none>
      Host Port:    <none>
      Environment:  <none>
      Mounts:       <none>
      Volumes:      <none>
```



```
student@cp:~$ kubectl rollout history deploy webserver --revision=2
```

```
....  
Image:      nginx:1.23.1-alpine  
.....
```

7. Use `kubectl rollout undo` to change the Deployment back to previous version.

```
student@cp:~$ kubectl rollout undo deploy webserver
```

```
deployment.apps/webserver rolled back
```

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
webserver-6cbc654ddc-7wb5q	1/1	Running	0	37s
webserver-6cbc654ddc-svbtj	1/1	Running	0	37s

```
student@cp:~$ kubectl describe pod webserver-6cbc654ddc-7wb5q |grep Image:
```

```
Image:      nginx:1.22.1
```

8. Let's try the "RollingUpdate" strategy next. First, open the deployment configuration file and change the update strategy to "RollingUpdate." Then, just as you did before, update the container image to a new version (for example, set it to

nginx:1.26-alpine). After making these changes, apply the update and observe how the rollout is executed, ensuring that the new version is deployed gradually.

9. Clean up the system by removing the Deployment.

```
student@cp:~$ kubectl delete deploy webserver
```

```
deployment.apps "webserver" deleted
```

Exercise 7.4: Working with DaemonSets

A DaemonSet is a watch loop object like a Deployment which we have been working with in the rest of the labs. The DaemonSet ensures that when a node is added to a cluster, a pod will be created on that node. A Deployment would only ensure a particular number of pods are created in general, several could be on a single node. Using a DaemonSet can be helpful to ensure applications are on each node, helpful for things like metrics and logging especially in large clusters where hardware may be swapped out often. Should a node be removed from a cluster the DaemonSet would ensure the Pods are garbage collected before removal. Starting with Kubernetes v1.12 the scheduler handles DaemonSet deployment which means we can now configure certain nodes to not have a particular DaemonSet pods.

This extra step of automation can be useful for using with products like **ceph** where storage is often added or removed, but perhaps among a subset of hardware. They allow for complex deployments when used with declared resources like memory, CPU or volumes.

1. We begin by creating a yaml file. In this case the kind would be set to DaemonSet. For ease of use we will copy the previously created `rs.yaml` file and make a couple edits. Remove the Replicas: 2 line.

```
student@cp:~$ cp rs.yaml ds.yaml
```

```
student@cp:~$ vim ds.yaml
```



ds.yaml

```

1 ....
2 kind: DaemonSet #<<<----change the kind
3 ....
4   name: ds-one #<<<----change the name
5 ....
6   replicas: 2 #<<<----Remove this line
7 ....
8     system: DaemonSetOne #<<-- Edit both references
9 ....
10

```

2. Create and verify the newly formed DaemonSet. There should be one Pod per node in the cluster.

```
student@cp:~$ kubectl create -f ds.yaml
```

```
daemonset.apps/ds-one created
```

```
student@cp:~$ kubectl get ds
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE-SELECTOR	AGE
ds-one	2	2	2	2	2	<none>	1m

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ds-one-b1dcv	1/1	Running	0	2m
ds-one-z31r4	1/1	Running	0	2m

3. Verify the image running inside the Pods. We will use this information in the next section.

```
student@cp:~$ kubectl describe pod ds-one-b1dcv | grep Image:
```

```
Image:          nginx:1.22.1
```

Exercise 7.5: Rollout and Rollback using DaemonSet

One of the advantages of micro-services is the ability to replace and upgrade a container while continuing to respond to client requests. We will use the `OnDelete` setting that upgrades a container when the predecessor is deleted.



nginx versions

The **nginx** software updates on a distinct timeline from Kubernetes. If the lab shows an older version please use the current default, and then a newer version. Versions can be seen with this command: **sudo docker image ls nginx**

1. Begin by viewing the current `updateStrategy` setting for the `DaemonSet` created in the previous section.

```
student@cp:~$ kubectl get ds ds-one -o yaml | grep -A 4 Strategy
```

```
updateStrategy:
  rollingUpdate:
    maxSurge: 0
    maxUnavailable: 1
  type: RollingUpdate
```

2. Edit the object to use the `OnDelete` update strategy. This would allow the manual termination of some of the pods, resulting in an updated image when they are recreated.

```
student@cp:~$ kubectl edit ds ds-one
```

```
....
  updateStrategy:
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1
      type: OnDelete          #<-- Edit to be this line
status:
....
```

3. Update the DaemonSet to use a newer version of the **nginx** server. This time use the **set** command instead of **edit**. Set the version to be 1.26.1-alpine.

```
student@cp:~$ kubectl set image ds ds-one nginx=nginx:1.26-alpine --record
```

```
Flag --record has been deprecated, --record will be removed in the future
daemonset.apps/ds-one image updated
```

4. Verify that the Image: parameter for the Pod checked in the previous section is unchanged.

```
student@cp:~$ kubectl describe po ds-one-b1dcv |grep Image:
```

```
Image:                nginx:1.22.1
```

5. Delete the Pod. Wait until the replacement Pod is running and check the version.

```
student@cp:~$ kubectl delete po ds-one-b1dcv
```

```
pod "ds-one-b1dcv" deleted
```

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ds-one-xc86w	1/1	Running	0	19s
ds-one-z31r4	1/1	Running	0	4m8s

```
student@cp:~$ kubectl describe pod ds-one-xc86w |grep Image:
```

```
Image:          nginx:1.26-alpine
```

6. View the image running on the older Pod. It should still show version 1.22.1.

```
student@cp:~$ kubectl describe pod ds-one-z31r4 |grep Image:
```

```
Image:          nginx:1.22.1
```

7. View the history of changes for the DaemonSet. You should see two revisions listed. As we did not add the the `change-cause` annotation we didn't see why the object updated.

```
student@cp:~$ kubectl rollout history ds ds-one
```

```
daemonset.apps/ds-one
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl set image ds ds-one nginx=nginx:1.26-alpine --record=true
```

8. View the settings for the various versions of the DaemonSet. The `Image:` line should be the only difference between the two outputs.

```
student@cp:~$ kubectl rollout history ds ds-one --revision=1
```

```
daemonsets "ds-one" with revision #1
Pod Template:
  Labels:      system=DaemonSetOne
  Containers:
    nginx:
      Image:    nginx:1.22.1
      Port:     80/TCP
      Environment:  <none>
      Mounts:    <none>
      Volumes:    <none>
```

```
student@cp:~$ kubectl rollout history ds ds-one --revision=2
```

```
....
  Image:      nginx:1.26-alpine
.....
```

9. Use `kubectl rollout undo` to change the DaemonSet back to an earlier version. As we are still using the `OnDelete` strategy there should be no change to the Pods.

```
student@cp:~$ kubectl rollout undo ds ds-one --to-revision=1
```

```
daemonset.apps/ds-one rolled back
```

```
student@cp:~$ kubectl describe pod ds-one-xc86w |grep Image:
```

```
Image:          nginx:1.26-alpine
```

10. Delete the Pod, wait for the replacement to spawn then check the image version again.

```
student@cp:~$ kubectl delete pod ds-one-xc86w
```

```
pod "ds-one-xc86w" deleted
```

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ds-one-qc72k	1/1	Running	0	10s
ds-one-xc86w	0/1	Terminating	0	12m
ds-one-z31r4	1/1	Running	0	28m

```
student@cp:~$ kubectl describe po ds-one-qc72k |grep Image:
```

```
Image:          nginx:1.22.1
```

11. View the details of the DaemonSet. The Image should be v1.22.1 in the output.

```
student@cp:~$ kubectl describe ds |grep Image:
```

```
Image:          nginx:1.22.1
```

12. View the current configuration for the DaemonSet in YAML output. Look for the `updateStrategy`: the the type:

```
student@cp:~$ kubectl get ds ds-one -o yaml
```



```

apiVersion: apps/v1
kind: DaemonSet
.....
  terminationGracePeriodSeconds: 30
  updateStrategy:
    type: OnDelete
status:
  currentNumberScheduled: 2
.....

```

13. Create a new DaemonSet, this time setting the update policy to RollingUpdate. Begin by generating a new config file.

```
student@cp:~$ kubectl get ds ds-one -o yaml > ds2.yaml
```

14. Edit the file. Change the name and the update strategy, back to the default RollingUpdate.

```
student@cp:~$ vim ds2.yaml
```

```

....
  name: ds-two
....
  type: RollingUpdate

```

15. Create the new DaemonSet and verify the **nginx** version in the new pods.

```
student@cp:~$ kubectl create -f ds2.yaml
```

```
daemonset.apps/ds-two created
```

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ds-one-qc72k	1/1	Running	0	28m
ds-one-z31r4	1/1	Running	0	57m

```
ds-two-10khc      1/1      Running    0          5m
ds-two-kzp9g      1/1      Running    0          5m
```

```
student@cp:~$ kubectl describe po ds-two-10khc |grep Image:
```

```
Image:                nginx:1.22.1
```

16. Edit the configuration file and set the image to a newer version such as 1.26-alpine.

```
student@cp:~$ kubectl edit ds ds-two
```

```
....
- image: nginx:1.26-alpine
.....
```

17. View the age of the DaemonSets. It should be around ten minutes old, depending on how fast you type.

```
student@cp:~$ kubectl get ds ds-two
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE-SELECTOR	AGE
ds-two	2	2	2	2	2	<none>	10m

18. Now view the age of the Pods. Two should be much younger than the DaemonSet. They are also a few seconds apart due to the nature of the rolling update where one then the other pod was terminated and recreated.

```
student@cp:~$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
ds-one-qc72k	1/1	Running	0	36m
ds-one-z31r4	1/1	Running	0	1h

ds-two-2p8vz	1/1	Running	0	34s
ds-two-8lx7k	1/1	Running	0	32s

19. Verify the Pods are using the new version of the software.

```
student@cp:~$ kubectl describe po ds-two-8lx7k |grep Image:
```

```
Image:          nginx:1.26-alpine
```

20. View the rollout status and the history of the DaemonSets.

```
student@cp:~$ kubectl rollout status ds ds-two
```

```
daemon set "ds-two" successfully rolled out
```

21. Clean up the system by removing the DaemonSets.

```
student@cp:~$ kubectl delete ds ds-one ds-two
```


Chapter 8

Helm and Kustomize



8.1	Helm Overview	218
8.2	Helm	219
8.3	Using Helm	222
8.4	Kustomize Overview	224
8.5	Labs	228

8.1 Helm Overview

Deploying Complex Applications

- Package your Kubernetes application via **chart** template
- **Helm** client to request install of **chart**
- **Helm** creates cluster resources according to **chart**
- Version 4 was major update.

8.2 Helm

Benefits of Using Helm

- Simplifies Deployment
- Reusable and Shareable Packages
- Version Control
- Easy Rollback and Recovery
- Parameterization and Customization

Chart Contents

```
|-- Chart.yaml
|-- README.md
|-- templates
|   |-- NOTES.txt
|   |-- _helpers.tpl
|   |-- configmap.yaml
|   |-- deployment.yaml
|   |-- pvc.yaml
|   |-- secrets.yaml
|   |-- svc.yaml
|-- values.yaml
```


Templates

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ template "fullname" . }}
  labels:
    app: {{ template "fullname" . }}
    chart: "{{ .Chart.Name }}-{{ .Chart.Version }}"
    release: "{{ .Release.Name }}"
    heritage: "{{ .Release.Service }}"
type: Opaque
data:
  mariadb-root-password: {{ default "" .Values.mariadbRootPassword | b64enc\
                                                                    | quote }}
  mariadb-password: {{ default "" .Values.mariadbPassword | b64enc | quote }}
```

8.3 Using Helm

Chart Repositories and Hub

- Search for charts
- Add a new repository
- Simple HTTP servers with index file and tarball of charts
- **helm repo**
- ArtifactHub to replace Docker hub

Deploying a Chart

```
$ helm fetch bitnami/apache --untar  
$ cd apache/  
$ ls
```

```
Chart.lock Chart.yaml README.md charts ci files templates  
values.schema.json values.yaml
```

```
$ helm install anotherweb .
```

8.4 Kustomize Overview

Overview of Kustomize

- Declarative Configuration Management
- Overlay System for Customization
- No Templating Required
- Integrated with kubectl
- Resource Generation

Concepts of Kustomize

- Central Kustomization File - kustomization.yaml
- Bases and Overlays
- Resources
- Patches and Transformers

Installation and Setup

- Integrated with kubectl
- Standalone kustomize tool
- Directory Structure

kustomization.yaml

- Centralized Configuration
- Resource Aggregation
- Common Metadata Injection
- Name Modification
- Dynamic Resource Generation
- Patching and Overrides

8.5 Labs

Exercise 8.1: Working with Helm and Charts

Overview

helm allows for easy deployment of complex configurations. This could be handy for a vendor to deploy a multi-part application in a single step. Through the use of a Chart, or template file, the required components and their relationships are declared. Local agents like **Tiller** use the API to create objects on your behalf. Effectively its orchestration for orchestration.

There are a few ways to install **Helm**. The newest version may require building from source code. We will download a recent, stable version. Once installed we will deploy a Chart, which will configure **MariaDB** on our cluster.

Install Helm

1. On the cp node use **wget** to download the compressed tar file. Various versions can be found here: <https://github.com/helm/helm/releases/>

```
student@cp:~$ wget https://get.helm.sh/helm-v3.19.0-linux-amd64.tar.gz
```

```
<output_omitted>
helm-v3.19.0-linux-amd64.tar.gz 100%[=====] 17.18M --.-KB/s in 0.1s
2025-10-18 13:38:09 (101 MB/s) - 'helm-v3.19.0-linux-amd64.tar.gz' saved [16624839/16624839]
```

2. Uncompress and expand the file.

```
student@cp:~$ tar -xvf helm-v3.19.0-linux-amd64.tar.gz
```

```
linux-amd64/
linux-amd64/helm
```



```
linux-amd64/README.md
linux-amd64/LICENSE
```

3. Copy the **helm** binary to the `/usr/local/bin/` directory, so it is usable via the shell search path.

```
student@cp:~$ sudo cp linux-amd64/helm /usr/local/bin/helm
```

4. A Chart is a collection of files to deploy an application. There is a good starting repo available on <https://github.com/kubernetes/charts/tree/master/stable>, provided by vendors, or you can make your own. Search the current Charts in the Helm Hub or an instance of Monocular for available stable databases. Repos change often, so the following output may be different from what you see.

```
student@cp:~$ helm search hub database
```

URL	APP VERSION	DESCRIPTION	CHART VERSION
https://artifacthub.io/packages/helm/drycc/data...			1.0.2
		A PostgreSQL database used by Drycc Workflow.	
https://artifacthub.io/packages/helm/drycc-cana...			1.0.0
		A PostgreSQL database used by Drycc	
		↳ Workflow.	
https://artifacthub.io/packages/helm/camptocamp...			0.0.6
	1.0	Expose services and secret to access postgres	
	↳ d...		
https://artifacthub.io/packages/helm/cnieg/h2-d...			1.0.3
	1.4.199	A helm chart to deploy h2-database	
<output_omitted>			

5. You can also add repositories from various vendors, often found by searching artifacthub.io such as `ea1enn`, who has an echo program.

```
student@cp:~$ helm repo add ealenn https://ealenn.github.io/charts
```

```
"ealenn" has been added to your repositories
```

```
student@cp:~$ helm repo update
```

```
Hang tight while we grab the latest from your chart repositories...  
...Successfully got an update from the "ealenn" chart repository  
Update Complete. Happy Helming!
```

6. We will install the **tester** tool. The **-debug** option will create a lot of output. The output will typically suggest ways to access the software.

```
student@cp:~$ helm upgrade -i tester ealenn/echo-server --debug
```

```
history.go:56: [debug] getting history for release tester  
Release "tester" does not exist. Installing it now.  
install.go:173: [debug] Original chart version: ""  
install.go:190: [debug] CHART PATH: /home/student/.cache/helm/repository/echo-server-0.5.0.tgz  
  
client.go:122: [debug] creating 4 resource(s)  
NAME: tester  
<output_omitted>
```

7. Ensure the newly created `tester-echo-server` pod is running. Fix any issues, if not.
8. Look for the newly created service. Send a **curl** to the ClusterIP. You should get a lot of information returned.

```
student@cp:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	26h
tester-echo-server	ClusterIP	10.98.252.11	<none>	80/TCP	11m

```
student@cp:~$ curl 10.98.252.11
```

```
{"host":{"hostname":"10.98.252.11","ip":"","ffff:192.168.74.128","ips":
[]},"http":{"method":"GET","baseUrl":"","originalUrl":"/","protocol":
"http"},"request":{"params":{"0":"/"},"query":{"},"cookies":{"},"body":
{},"headers":{"host":"10.98.252.11","user-agent":"curl/7.58.0","accept":
"*//*"},"environment":{"PATH":"/usr/local/sbin:/usr/local/bin:/usr/sbin:
/usr/bin:/sbin:/bin","TERM":"xterm","HOSTNAME":"tester-echo-server-
786768d9f4-4zsz9"},"ENABLE__HOST":"true","ENABLE__HTTP":"true","ENABLE__
<output_omitted>
```

- View the Chart history on the system. The use of the **-a** option will show all Charts including deleted and failed attempts.

```
student@cp:~$ helm list
```

NAME	NAMESPACE	REVISION	UPDATED
STATUS	CHART	APP	VERSION
tester	default	1	2025-10-19 13:42:38.262327888 +0000 UTC
deployed	echo-server-0.5.0	0.6.0	

- Delete the **tester** Chart. No releases of tester should be found.

```
student@cp:~$ helm uninstall tester
```

```
release "tester" uninstalled
```

```
student@cp:~$ helm list
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
------	-----------	----------	---------	--------	-------	-------------

- Find the downloaded chart. It should be a compressed tarball under the user's home directory. Your **echo** version may be slightly different.

```
student@cp:~$ find $HOME -name *echo*
```

```
/home/student/.cache/helm/repository/echo-server-0.5.0.tgz
```

- Move to the archive directory and extract the tarball. Take a look at the files within.

```
student@cp:~$ cd $HOME/.cache/helm/repository ; tar -xvf echo-server-*
```

```
echo-server/Chart.yaml
echo-server/values.yaml
echo-server/templates/_helpers.tpl
echo-server/templates/configmap.yaml
echo-server/templates/deployment.yaml
<output_omitted>
```

- Examine the `values.yaml` file to see some of the values that could have been set.

```
student@cp:~/.cache/helm/repository$ cat echo-server/values.yaml
```

```
<output_omitted>
```

14. You can also download the **values** file to review or modify it before proceeding with the installation. Once ready, add an additional repository and download the **Metrics Server** chart.

```
student@cp:~$ helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
```

```
student@cp:~$ helm fetch metrics-server/metrics-server --untar
```

```
student@cp:~$ cd metrics-server/
```

15. Take a look at the chart. You'll note it looks similar to the previous. Read through the `:values.yaml`. Make changes to `values.yaml` file to include the `--kubelet-insecure-tls` argument before deploying.

```
student@cp:~$ ls
```

```
Chart.lock  Chart.yaml  README.md  charts  ci  files  templates
values.schema.json  values.yaml
```

```
student@cp:~$ less values.yaml
```

```
# Default values for metrics-server.
# This is a YAML-formatted file.
# Declare variables to be passed into your templates.
image:
  repository: registry.k8s.io/metrics-server/metrics-server
##
defaultArgs:
  - --cert-dir=/tmp
  - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
  - --kubelet-use-node-status-port
  - --metric-resolution=15s
  - --kubelet-insecure-tls      # <---- add this line

<output_omitted>
```

16. Use the `values.yaml` file to install the chart. Take a look at the output and ensure the pod is running.

```
student@cp:~$ helm install metrics-server . -n kube-system
```

```
NAME: metrics-server
LAST DEPLOYED: Tue Oct 21 11:09:16 2025
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
* Metrics Server
<output_omitted>
```

17. Verify the installed metrics server is working fine and able to display the metrics.

```
student@cp:~$ kubectl get pods -n kube-system | grep metrics
```

```
metrics-server-799f7ccf68-b9t8d    1/1    Running    0    2m38s
```

```
student@cp:~$ kubectl top nodes
```

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
cp	202m	10%	1156Mi	14%
worker	41m	2%	717Mi	9%

Exercise 8.2: Horizontal Pod Autoscaler (HPA)

1. In this exercise, you will explore how Kubernetes automatically scales applications based on real-time resource usage.

The Horizontal Pod Autoscaler (HPA) adjusts the number of pod replicas to maintain target CPU utilization. You already have the `metrics-server` installed in the previous lab, which is required for HPA to collect performance metrics.

2. Deploy a lightweight web application that consumes measurable CPU resources. The image `registry.k8s.io/hpa-example` is a small HTTP server that performs CPU work on each request. By setting explicit `resources.requests` and `resources.limits`, you define the CPU boundaries that the HPA will monitor. The limit of 10m (10 millicores) ensures that even a small load triggers noticeable scaling events.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_08/hpa-deploy.yaml .
```

```
student@cp:~$ vim hpa-deploy.yaml
```

YAML

rs.yaml

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: hpa-app
5 spec:
6   replicas: 2
7   selector:
8     matchLabels:
9       app: hpa-app
10  template:
11    metadata:
12      labels:
13        app: hpa-app
14    spec:
15      containers:
16      - name: web
17        image: registry.k8s.io/hpa-example
18        ports:
```



```
19     - containerPort: 80
20     resources:
21       requests:
22         cpu: 5m
23       limits:
24         cpu: 10m
25 ---
26 apiVersion: v1
27 kind: Service
28 metadata:
29   name: hpa-app
30 spec:
31   type: ClusterIP
32   selector:
33     app: hpa-app
34   ports:
35     - port: 80
36       targetPort: 80
37
```

3. Create the Deployment:

```
student@cp:~$ kubectl create -f hpa-deploy.yaml
```

```
deployment.apps/hpa-app created
service/hpa-app created
```

4. Verify that your deployment is running as expected. The `kubectl get` command will show your Deployment, ReplicaSet, Pods, and Service. Each Pod should be Running on one of your worker nodes, indicating successful scheduling and networking.


```
student@cp:~$ kubectl get deploy,rs,pods,svc -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES
↪ SELECTOR						
deployment.apps/hpa-app	2/2	2	2	2m6s	web	
↪ registry.k8s.io/hpa-example		app=hpa-app				

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES
↪ SELECTOR						
replicaset.apps/hpa-app-844785cc56	2	2	2	2m6s	web	
↪ registry.k8s.io/hpa-example		app=hpa-app,pod-template-hash=844785cc56				

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
↪ NOMINATED NODE						
pod/hpa-app-844785cc56-djrtq	1/1	Running	0	2m6s	192.168.0.1	cp
↪ <none>						<none>
pod/hpa-app-844785cc56-zbk1c	1/1	Running	0	2m6s	192.168.1.68	worker
↪ <none>						<none>

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/hpa-app	ClusterIP	10.107.208.86	<none>	80/TCP	2m6s	app=hpa-app
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	20m	<none>

- Before creating an HPA resource, you can generate its YAML file using the `--dry-run=client` flag. This is a safe and powerful way to preview what Kubernetes will create without making actual changes. The generated YAML can be version-controlled or customized later.

```
student@cp:~$ kubectl autoscale deployment hpa-app \
--cpu=50% --min=2 --max=10 \
--dry-run=client -o yaml > hpa.yaml
```

```
student@cp:~$ cat hpa.yaml
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
...
```

Here, the HPA targets 50% CPU utilization: when average CPU usage across Pods exceeds 50% of their requested value, Kubernetes will add replicas; when usage falls below, it will gradually reduce replicas back down.

6. Apply the HPA definition and confirm it's being tracked by the cluster. You should immediately see an entry in the HPA list, though the TARGETS column will remain 0% until load is detected.

```
student@cp:~$ kubectl apply -f hpa.yaml
student@cp:~$ kubectl get hpa
```

```
horizontalpodautoscaler.autoscaling/hpa-app created
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hpa-app	Deployment/hpa-app	cpu: 20%/50%	2	10	2	33s

At this point, your HPA is active and continuously monitoring metrics-server data.

7. In a separate terminal, use the `siege` tool to simulate concurrent HTTP traffic to your service of deployment `hpa-app`. Each request triggers CPU work inside the container, causing CPU utilization to spike. Run the test for a couple of minutes to allow HPA metrics to stabilize and trigger scaling.

```
student@cp:~$ sudo apt-get update && sudo apt-get install -y siege
student@cp:~$ siege -q -c 5 -t 2m http://10.107.208.86
```

```
New configuration template added to /home/student/.siege
Run siege -C to view the current settings in that file
```

The spike in requests will push average CPU usage well above 50%, forcing the HPA to scale up pods automatically.

- While the load test is running, observe HPA behavior in real time. The TARGETS column will show CPU utilization percentage and you'll see the number of replicas gradually increase.

```
student@cp:~$ watch kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hpa-app	Deployment/hpa-app	210%/50%	2	10	6	3m

The HPA controller polls metrics every 15 seconds. It scales conservatively to avoid thrashing, so scaling might appear delayed.

- You can also watch Pods scale dynamically. The new Pods will be distributed across both nodes, confirming the scheduler is balancing workloads properly.

```
student@cp:~$ watch kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	NODE
hpa-app-6d7d9c7c9c-hbw9t	1/1	Running	0	6m	worker
hpa-app-6d7d9c7c9c-8qj9d	1/1	Running	0	2m	cp
hpa-app-6d7d9c7c9c-7k2h4	1/1	Running	0	1m	worker
...					

- Once you stop the load test, the average CPU usage will drop, and the HPA will gradually reduce the number of replicas. Scaling down is intentionally slower to ensure stability and avoid oscillation.

```
student@cp:~$ # Press Ctrl+C in siege window
student@cp:~$ watch kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hpa-app	Deployment/hpa-app	cpu: 20%/50%	2	10	10	7m20s

The Deployment eventually stabilizes back to 2 pods when idle.

11. Troubleshooting Tips:

- If HPA shows <unknown> metrics, ensure metrics-server is healthy and running in kube-system.
- Verify that the container defines a `cpu request`; HPA relies on requests, not limits, to compute utilization.
- Check `kubectl top pods` to view live CPU usage.
- Scaling behavior can be tuned in HPA v2 using custom metrics and stabilization windows.

12. Finally, clean up your resources to free up the cluster for other labs.

```
student@cp:~$ kubectl delete -f hpa.yaml -f hpa-deploy.yaml
```

```
horizontalpodautoscaler.autoscaling "hpa-app" deleted from default namespace
deployment.apps "hpa-app" deleted from default namespace
service "hpa-app" deleted from default namespace
```

Exercise 8.3: Working with Kustomize

1. Kustomize is a tool for customizing Kubernetes configurations. Its built into kubectl CLI.

```
student@cp:~$ kubectl kustomize --help
```

```
Build a set of KRM resources using a 'kustomization.yaml' file. The DIR argument must be a path to
↳ a directory
containing 'kustomization.yaml', or a git repository URL with a path suffix specifying same with
↳ respect to the
repository root. If DIR is omitted, '.' is assumed.
```

```
Examples:
  # Build the current working directory
  kubectl kustomize
...
<output_omitted>
```

2. Create a directory structure and copy the resource files in appropriate directory.

```
student@cp:~$ mkdir -p myapp/base myapp/overlays/dev myapp/overlays/prod
student@cp:~$ tree myapp
```

```
myapp
|--base
|-- overlays
    |-- dev
    |-- prod
```

3. Copy appropriate resource yaml files from the Solutions directory to directory structure created above.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_08/*.yaml-base myapp/base/
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_08/*.yaml-dev myapp/overlays/dev
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_08/*.yaml-prod myapp/overlays/prod
```

```
student@cp:~$ tree myapp
```

```
myapp
|-- base
|   |-- deployment.yaml-base
```

```
| |-- kustomization.yaml-base
| |-- service.yaml-base
|-- overlays
|   |-- dev
|   |   |-- deployment-patch.yaml-dev
|   |   |-- kustomization.yaml-dev
|   |   |-- service-patch.yaml-dev
|   |-- prod
|       |-- deployment-patch.yaml-prod
|       |-- kustomization.yaml-prod
|       |-- service-patch.yaml-prod
```

5 directories, 9 files

4. Rename the manifest files in the base directory.

```
student@cp:~$ cd myapp/base
student@cp:~/myapp/base$ for file in *.yaml-base; do mv "$file" "${file/-base/}"; done
student@cp:~/myapp/base$ ls *.yaml
```

```
deployment.yaml kustomization.yaml service.yaml
```

5. Rename the manifest files in the overlays/dev directory.

```
student@cp:~/myapp/base$ cd ../overlays/dev/
student@cp:~/myapp/overlays/dev$ for file in *.yaml-dev; do mv "$file" "${file/-dev/}"; done
student@cp:~/myapp/overlays/dev$ ls *.yaml
```

```
deployment-patch.yaml kustomization.yaml service-patch.yaml
```

6. Rename the manifest files in the overlays/prod directory.

```
student@cp:~/myapp/overlays/dev$ cd ../prod
student@cp:~/myapp/overlays/prod$ for file in *.yaml-prod; do mv "$file" "${file/-prod/}"; done
student@cp:~/myapp/overlays/prod$ ls *.yaml
```

```
deployment-patch.yaml  kustomization.yaml  service-patch.yaml
```

7. Verify to see if the files and directory structure match the below output.

```
student@cp:~/myapp/overlays/prod$ cd
student@cp:~$ tree myapp
```

```
myapp
|-- base
|   |-- deployment.yaml
|   |-- kustomization.yaml
|   |-- service.yaml
|-- overlays
|   |-- dev
|       |-- deployment-patch.yaml
|       |-- kustomization.yaml
|       |-- service-patch.yaml
|   |-- prod
|       |-- deployment-patch.yaml
|       |-- kustomization.yaml
|       |-- service-patch.yaml
```

```
5 directories, 9 files
```

8. The kustomization.yaml manifest file in the base directory has the details of the resources needs to be created along with additional metadata injection and name modification.

```
student@cp:~$ vim myapp/base/kustomization.yaml
```

YAML

```
1 apiVersion: kustomize.config.k8s.io/v1beta1
2 kind: Kustomization
3 namePrefix: lf-
4 resources:
5 - deployment.yaml
6 - service.yaml
7 labels:
8 - includeSelectors: true
9   pairs:
10     company: linux-foundation
11
12
```

9. Build the configuration and preview the output before applying it to the cluster. The base resource configurations are used as a foundation, and then environment-specific modifications are applied when building for a particular environment.

```
student@cp:~$ kubectl kustomize myapp/base
student@cp:~$ kubectl kustomize myapp/overlays/dev
student@cp:~$ kubectl kustomize myapp/overlays/prod
```

10. Once Verified, the configuration can be applied to the cluster using the -k option along with the kubectl

```
student@cp:~$ kubectl apply -k myapp/base/
```

```
service/lf-myapp created
deployment.apps/lf-myapp created
```


11. Verify the resources have been deployed correctly and the metadata has been injected as per kustomization.yaml

```
student@cp:~$ kubectl get all -l company=linux-foundation
```

```
NAME                                READY   STATUS    RESTARTS   AGE
pod/lf-myapp-5b68c7d779-ngsq4      1/1     Running   0           3m23s
```



```
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
service/lf-myapp ClusterIP      10.104.21.181 <none>       80/TCP     3m23s
```



```
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/lf-myapp 1/1     1             1           3m23s
```



```
NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/lf-myapp-5b68c7d779 1          1         1       3m23s
```

12. Likewise, the remaining configurations can also be patched

```
student@cp:~$ kubectl apply -k myapp/overlays/dev/
```

```
service/dev-lf-myapp created
deployment.apps/dev-lf-myapp created
```

13. Verify if the resources have been configured as per the manifest files present in the overlays/dev directory.
14. Clean up by deleting the resources.

```
student@cp:~$ kubectl delete -k myapp/overlays/dev/
```

```
service "dev-lf-myapp" deleted from default namespace  
deployment.apps "dev-lf-myapp" deleted from default namespace
```

Chapter 9

Volumes and Data



9.1	Volumes Overview	248
9.2	Volumes	249
9.3	Persistent Volumes	253
9.4	Rook	257
9.5	Passing Data To Pods	258
9.6	ConfigMaps	261
9.7	Labs	263

9.1 Volumes Overview

Overview

- Object to save data longer than container lifetime
- Many `Volume` types to choose from
- Define Persistent Volumes (PV)
- Define Persistent Volume Claims (PVC)
- Create `Secrets`
- Create `ConfigMaps`

9.2 Volumes

Introducing Volumes

A POD

Single IP

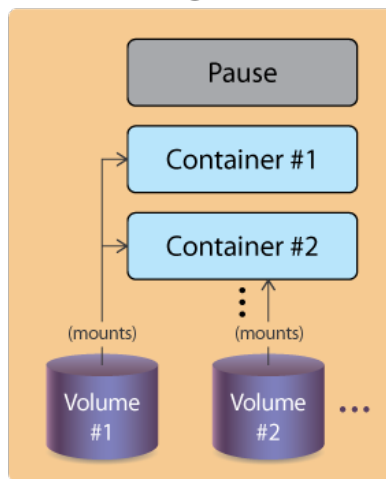


Figure 9.1: K8s Pod Volumes

Volume Spec

```
apiVersion: v1
kind: Pod
metadata:
  name: ford Pinto
  namespace: default
spec:
  containers:
  - image: simpleapp
    name: gastank
    command:
      - sleep
      - "3600"
    volumeMounts:
      - mountPath: /scratch
        name: scratch-volume
  volumes:
  - name: scratch-volume
    emptyDir: {}
```

Volume Types

- Several types possible, more being added

- | | | |
|------------------------|---------------------|-----------------------------|
| – awsElasticBlockStore | – gcePersistentDisk | – quobyte |
| – azureDisk | – gitRepo | – rbd |
| – azureFile | – glusterfs | – scaleIO |
| – cephfs | – hostPath | – secret |
| – csi | – iscsi | – storageos |
| – downwardAPI | – local | – vsphereVolume |
| – emptyDir | – nfs | – persistentVolumeClaim |
| – fc (fibre channel) | – projected | – CSIPersistentVolumeSource |
| – flocker | – portworxVolume | |

Shared Volume Example

```
....
  containers:
    - name: alphacont
      image: busybox
      volumeMounts:
        - mountPath: /alphadir
          name: sharevol
    - name: betacont
      image: busybox
      volumeMounts:
        - mountPath: /betadir
          name: sharevol
  volumes:
    - name: sharevol
      emptyDir: {}
```


9.3 Persistent Volumes

Persistent Volumes and Claims

- Useful for porting data
- Resources managed via API
- Storage abstraction
- Several phases

```
$ kubectl get pv  
$ kubectl get pvc
```

Persistent Volume

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: 10Gpv01
  labels:
    type: local
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/somepath/data01"
```

Persistent Volume Claim

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
```

In the Pod:

```
spec:
  containers:
  ....
  volumes:
    - name: test-volume
      persistentVolumeClaim:
        claimName: myclaim
```

Dynamic Provisioning

- Claim filled via auto-provisioning
- No need for admin to pre-create PVs
- Uses the `StorageClass` API object
- `StorageClass` defines volume plugin, or provisioner to use
- Single, default class possible via annotation

9.4 Rook

Using Rook for Storage Orchestration

- File, block and object storage
- Multiple storage providers
- Hyper-scale or hyper-converge
- Leverages CRDs and a rook operator

9.5 Passing Data To Pods

Secrets

- Leverages base64 encoding
- Is **not** encryption unless further configured
- Conversion to generic data, acceptable everywhere
- Encoded manually or via **kubect1 create secret**

```
$ kubect1 create secret generic mysql --from-literal=password=root
```

Using Secrets via Environment Variables

```
...
spec:
  containers:
  - image: mysql:5.5
    name: dbpod
    env:
    - name: MYSQL_ROOT_PASSWORD
      valueFrom:
        secretKeyRef:
          name: mysql
          key: password
```

Mounting Secrets as Volumes

```
...
spec:
  containers:
    - image: busybox
      command:
        - sleep
        - "3600"
      volumeMounts:
        - mountPath: /mysqlpassword
          name: mysql
      name: busy
  volumes:
    - name: mysql
      secret:
        secretName: mysql
```


9.6 ConfigMaps

Portable Data With ConfigMaps

- Decouple configuration data from container image
- Not encoded or encrypted
- Can be created from various sources
 - Multiple files in same directory
 - Individual files
 - Literal values
- Can be consumed in various ways
 - Container environmental variables
 - Use ConfigMap values in Pod commands
 - Populate Volume from ConfigMap
 - Add ConfigMap data to specific path in Volume
 - Set file names and access mode in Volume from ConfigMap data
 - Can be used by system components and controllers

Using ConfigMaps

```
env:
- name: SPECIAL_LEVEL_KEY
  valueFrom:
    configMapKeyRef:
      name: special-config
      key: special.how

volumes:
- name: config-volume
  configMap:
    name: special-config
```

9.7 Labs

Exercise 9.1: Create a ConfigMap

Overview

Container files are ephemeral, which can be problematic for some applications. Should a container be restarted the files will be lost. In addition, we need a method to share files between containers inside a Pod.

A **Volume** is a directory accessible to containers in a Pod. Cloud providers offer volumes which persist further than the life of the Pod, such that AWS or GCE volumes could be pre-populated and offered to Pods, or transferred from one Pod to another. **Ceph** is also another popular solution for dynamic, persistent volumes.

Unlike current **Docker** volumes a Kubernetes volume has the lifetime of the Pod, not the containers within. You can also use different types of volumes in the same Pod simultaneously, but Volumes cannot mount in a nested fashion. Each must have their own mount point. Volumes are declared with `spec.volumes` and mount points with `spec.containers.volumeMounts` parameters. Each particular volume type, may have other restrictions. <https://kubernetes.io/docs/concepts/storage/volumes/#types-of-volumes>

We will also work with a **ConfigMap**, which is basically a set of key-value pairs. This data can be made available so that a Pod can read the data as environment variables or configuration data. A **ConfigMap** is similar to a **Secret**, except they are not base64 byte encoded arrays. They are stored as strings and can be read in serialized form.

There are three different ways a **ConfigMap** can ingest data, from a literal value, from a file or from a directory of files.

1. We will create a **ConfigMap** containing primary colors. We will create a series of files to ingest into the **ConfigMap**. First, we create a directory `primary` and populate it with four files. Then we create a file in our home directory with our favorite color.

```
student@cp:~$ mkdir primary
```

```
student@cp:~$ echo c > primary/cyan
```

```
student@cp:~$ echo m > primary/magenta

student@cp:~$ echo y > primary/yellow

student@cp:~$ echo k > primary/black

student@cp:~$ echo "known as key" >> primary/black

student@cp:~$ echo blue > favorite
```

2. Now we will create the ConfigMap and populate it with the files we created as well as a literal value from the command line.

```
student@cp:~$ kubectl create configmap colors \
  --from-literal=text=black \
  --from-file=./favorite \
  --from-file=./primary/
```

```
configmap/colors created
```

3. View how the data is organized inside the cluster. Use the `yaml` then the `json` output type to see the formatting.

```
student@cp:~$ kubectl get configmap colors
```

NAME	DATA	AGE
colors	6	30s

```
student@cp:~$ kubectl get configmap colors -o yaml
```

```
apiVersion: v1
data:
  black: |
```

```
    k
    known as key
  cyan: |
    c
  favorite: |
    blue
  magenta: |
    m
  text: black
  yellow: |
    y
kind: ConfigMap
<output_omitted>
```

4. Now we can create a Pod to use the ConfigMap. In this case a particular parameter is being defined as an environment variable.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/simpleshell.yaml .
```

```
student@cp:~$ vim simpleshell.yaml
```

YA
ML

simpleshell.yaml

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: shell-demo
5  spec:
6    containers:
7      - name: nginx
```



```
8     image: nginx:1.29.6
9     env:
10    - name: ilike
11      valueFrom:
12        configMapKeyRef:
13          name: colors
14          key: favorite
```

5. Create the Pod and view the environmental variable. After you view the parameter, exit out and delete the pod.

```
student@cp:~$ kubectl create -f simpleshell.yaml
```

```
pod/shell-demo created
```

```
student@cp:~$ kubectl exec shell-demo -- /bin/bash -c 'echo $ilike'
```

```
blue
```

```
student@cp:~$ kubectl delete pod shell-demo
```

```
pod "shell-demo" deleted
```

6. All variables from a file can be included as environment variables as well. Comment out the previous `env:` stanza and add a slightly different `envFrom` to the file. Having new and old code at the same time can be helpful to see and understand the differences. Recreate the Pod, check all variables and delete the pod again. They can be found spread throughout the environment variable output.

```
student@cp:~$ vim simpleshell.yaml
```



simpleshell.yaml

```
1 <output_omitted>
2   image: nginx
3   #   env:
4   #   - name: ilike
5   #     valueFrom:
6   #       configMapKeyRef:
7   #         name: colors
8   #         key: favorite
9   envFrom:                                #<-- Same indent as image: line
10  - configMapRef:
11    name: colors
12
```

```
student@cp:~$ kubectl create -f simpleshell.yaml
```

```
pod/shell-demo created
```

```
student@cp:~$ kubectl exec shell-demo -- /bin/bash -c 'env'
```

```
black=k
known as key

KUBERNETES_SERVICE_PORT_HTTPS=443
cyan=c
<output_omitted>
```

```
student@cp:~$ kubectl delete pod shell-demo
```

```
pod "shell-demo" deleted
```

7. A ConfigMap can also be created from a YAML file. Create one with a few parameters to describe a car.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/car-map.yaml .
```

```
student@cp:~$ vim car-map.yaml
```

YAML

car-map.yaml

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: fast-car
5   namespace: default
6 data:
7   car.make: Ford
8   car.model: Mustang
9   car.trim: Shelby
```

8. Create the ConfigMap and verify the settings.

```
student@cp:~$ kubectl create -f car-map.yaml
```

```
configmap/fast-car created
```

```
student@cp:~$ kubectl get configmap fast-car -o yaml
```


YAML

```
1 apiVersion: v1
2 data:
3   car.make: Ford
4   car.model: Mustang
5   car.trim: Shelby
6 kind: ConfigMap
7 <output_omitted>
8
```

9. We will now make the ConfigMap available to a Pod as a mounted volume. You can again comment out the previous environmental settings and add the following new stanza. The containers: and volumes: entries are indented the same number of spaces.

```
student@cp:~$ vim simpleshell.yaml
```

YAML**simpleshell.yaml**

```
1 <output_omitted>
2 spec:
3   containers:
4     - name: nginx
5       image: nginx
6       volumeMounts:
7         - name: car-vol
8           mountPath: /etc/cars
9   volumes:
10    - name: car-vol
11      configMap:
12        name: fast-car
13 <comment out rest of file>
14
```

10. Create the Pod again. Verify the volume exists and the contents of a file within. Due to the lack of a carriage return in the file your next prompt may be on the same line as the output, Shelby.

```
student@cp:~$ kubectl create -f simpleshell.yaml
```

```
pod "shell-demo" created
```

```
student@cp:~$ kubectl exec shell-demo -- /bin/bash -c 'df -ha |grep car'
```

```
/dev/root      9.6G  3.2G   6.4G  34% /etc/cars
```

```
student@cp:~$ kubectl exec shell-demo -- /bin/bash -c 'cat /etc/cars/car.trim'
```

```
Shelby #<-- Then your prompt
```

11. Delete the Pod and ConfigMaps we were using.

```
student@cp:~$ kubectl delete pods shell-demo
```

```
pod "shell-demo" deleted
```

```
student@cp:~$ kubectl delete configmap fast-car colors
```

```
configmap "fast-car" deleted  
configmap "colors" deleted
```

Exercise 9.2: Creating a Persistent NFS Volume (PV)

We will first deploy an NFS server. Once tested we will create a persistent NFS volume for containers to claim.

1. Install the software on your cp node.

```
student@cp:~$ sudo apt-get update && sudo \  
    apt-get install -y nfs-kernel-server
```

```
<output_omitted>
```

2. Make and populate a directory to be shared. Also give it similar permissions to `/tmp/`

```
student@cp:~$ sudo mkdir /opt/sfw
```

```
student@cp:~$ sudo chmod 1777 /opt/sfw/
```

```
student@cp:~$ sudo bash -c 'echo software > /opt/sfw/hello.txt'
```

3. Edit the NFS server file to share out the newly created directory. In this case we will share the directory with all. You can always **snoop** to see the inbound request in a later step and update the file to be more narrow.

```
student@cp:~$ sudo vim /etc/exports
```

```
/opt/sfw/ *(rw, sync, no_root_squash, subtree_check)
```

4. Cause `/etc/exports` to be re-read:

```
student@cp:~$ sudo exportfs -ra
```

5. Test by mounting the resource from your **second** node.

```
student@worker:~$ sudo apt-get -y install nfs-common
```

```
<output_omitted>
```

```
student@worker:~$ showmount -e cp
```

```
Export list for cp:  
/opt/sfw *
```

```
student@worker:~$ sudo mount cp:/opt/sfw /mnt
```

```
student@worker:~$ ls -l /mnt
```

```
total 4  
-rw-r--r-- 1 root root 23 Aug 28 17:55 hello.txt
```

6. Return to the cp node and create a YAML file for the object with `kind: PersistentVolume`. Use the hostname of the cp server and the directory you created in the previous step. Only syntax is checked, an incorrect name or directory will not generate an error, but a Pod using the resource will not start. Note that the `accessModes` do not currently affect actual access and are typically used as labels instead.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/PV01.yaml .
```

```
student@cp:~$ vim PV01.yaml
```

YAML

PV01.yaml

```
1 apiVersion: v1  
2 kind: PersistentVolume
```



```

3 metadata:
4   name: pvvol-1
5 spec:
6   capacity:
7     storage: 1Gi
8   accessModes:
9     - ReadWriteMany
10  persistentVolumeReclaimPolicy: Retain
11  nfs:
12    path: /opt/sfw
13    server: cp #<-- Edit to match cp node
14    readOnly: false

```

7. Create the persistent volume, then verify its creation.

```
student@cp:~$ kubectl create -f PVol.yaml
```

```
persistentvolume/pvvol-1 created
```

```
student@cp:~$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
↪ VOLUMEATTRIBUTESCLASS		REASON	AGE			
pvvol-1	1Gi	RWX	Retain	Available		<unset>
↪ 6s						

Exercise 9.3: Creating a Persistent Volume Claim (PVC)

Before Pods can take advantage of the new PV we need to create a **Persistent Volume Claim (PVC)**.

1. Begin by determining if any currently exist.

```
student@cp:~$ kubectl get pvc
```

```
No resources found in default namespace.
```

2. Create a YAML file for the new pvc.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/pvc.yaml .
```

```
student@cp:~$ vim pvc.yaml
```

YAML

pvc.yaml

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: pvc-one
5  spec:
6    accessModes:
7    - ReadWriteMany
8    resources:
9      requests:
10       storage: 200Mi
```

3. Create and verify the new pvc is bound. Note that the size is 1Gi, even though 200Mi was suggested. Only a volume of at least that size could be used.

```
student@cp:~$ kubectl create -f pvc.yaml
```

```
persistentvolumeclaim/pvc-one created
```

```
student@cp:~$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
pvc-one	Bound	pvvol-1	1Gi	RWX		<unset>	7s

4. Look at the status of the pv again, to determine if it is in use. It should show a status of Bound.

```
student@cp:~$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
↪ VOLUMEATTRIBUTESCLASS		REASON	AGE			
pvvol-1	1Gi	RWX	Retain	Bound	default/pvc-one	
↪ <unset>			3m45s			

5. Create a new deployment to use the pvc. We will copy and edit an existing deployment yaml file. We will change the deployment name then add a volumeMounts section under containers and a volumes section to the general spec. The name used must match in both places, whatever name you use. The claimName must match an existing pvc. As shown in the following example. The volumes line is the same indent as containers and dnsPolicy.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/nfs-pod.yaml .
```

```
student@cp:~$ vim nfs-pod.yaml
```



nfs-pod.yaml

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    annotations:
5      deployment.kubernetes.io/revision: "1"
6    generation: 1
7    labels:
8      run: nginx
9    name: nginx-nfs                                #<-- Name has been changed
10   namespace: default
11  spec:
12    replicas: 1
13    selector:
14      matchLabels:
15        run: nginx
16    strategy:
17      rollingUpdate:
18        maxSurge: 1
19        maxUnavailable: 1
20      type: RollingUpdate
21    template:
22      metadata:
23        creationTimestamp: null
24        labels:
25          run: nginx
26      spec:
27        containers:
28          - image: nginx:1.29.6
29            name: nginx
30            volumeMounts:                          #<-- volume mounts has been added
31              - name: nfs-vol
```




```

32     mountPath: /opt
33     ports:
34     - containerPort: 80
35       protocol: TCP
36     resources: {}
37     terminationMessagePath: /dev/termination-log
38     terminationMessagePolicy: File
39 volumes:                                     #<--Volume has been added
40 - name: nfs-vol
41   persistentVolumeClaim:
42     claimName: pvc-one
43 dnsPolicy: ClusterFirst
44 restartPolicy: Always
45 schedulerName: default-scheduler
46 securityContext: {}
47 terminationGracePeriodSeconds: 30

```

6. Create the pod using the newly edited file.

```
student@cp:~$ kubectl create -f nfs-pod.yaml
```

```
deployment.apps/nginx-nfs created
```

7. Look at the details of the pod. You may see the daemonset pods running as well.

```
student@cp:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-nfs-1054709768-s8g28	1/1	Running	0	3m

```
student@cp:~$ kubectl describe pod nginx-nfs-1054709768-s8g28
```

```
Name:          nginx-nfs-1054709768-s8g28
Namespace:     default
Priority:       0
Node:          worker/10.128.0.5

<output_omitted>

Mounts:
  /opt from nfs-vol (rw)

<output_omitted>

Volumes:
  nfs-vol:
    Type:          PersistentVolumeClaim (a reference to a PersistentV...
    ClaimName:      pvc-one
    ReadOnly:       false
  <output_omitted>
```

Exercise 9.4: Using a ResourceQuota to Limit PVC Count and Usage

The flexibility of cloud-based storage often requires limiting consumption among users. We will use the `ResourceQuota` object to both limit the total consumption as well as the number of persistent volume claims.

1. Begin by deleting the deployment we had created to use NFS, the pv and the pvc.

```
student@cp:~$ kubectl delete deploy nginx-nfs
```

```
deployment.apps "nginx-nfs" deleted
```

```
student@cp:~$ kubectl delete pvc pvc-one
```

```
persistentvolumeclaim "pvc-one" deleted
```

```
student@cp:~$ kubectl delete pv pvvol-1
```

```
persistentvolume "pvvol-1" deleted
```

2. Create a yaml file for the ResourceQuota object. Set the storage limit to ten claims with a total usage of 500Mi.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/storage-quota.yaml .
```

```
student@cp:~$ vim storage-quota.yaml
```

YA
ML

storage-quota.yaml

```
1  apiVersion: v1
2  kind: ResourceQuota
3  metadata:
4    name: storagequota
5  spec:
6    hard:
7      persistentvolumeclaims: "10"
8      requests.storage: "500Mi"
```

3. Create a new namespace called `small`. View the namespace information prior to the new quota. Either the long name with double dashes `--namespace` or the nickname `ns` work for the resource.

```
student@cp:~$ kubectl create namespace small
```

```
namespace/small created
```

```
student@cp:~$ kubectl describe ns small
```

```
Name:          small
Labels:         <none>
Annotations:    <none>
Status:         Active
```

```
No resource quota.
```

```
No resource limits.
```

4. Create a new pv and pvc in the small namespace.

```
student@cp:~$ kubectl create -f PVol.yaml
```

```
persistentvolume/pvvol-1 created
```

```
student@cp:~$ kubectl -n small create -f pvc.yaml
```

```
persistentvolumeclaim/pvc-one created
```

5. Create the new resource quota, placing this object into the small namespace.

```
student@cp:~$ kubectl -n small create -f storage-quota.yaml
```

```
resourcequota/storagequota created
```

6. Verify the `small` namespace has quotas. Compare the output to the same command above.

```
student@cp:~$ kubectl describe ns small
```

```
Name:          small
Labels:        <none>
Annotations:   <none>
Status:        Active

Resource Quotas
Name:          storagequota
Resource       Used   Hard
-----
persistentvolumeclaims 1    10
requests.storage      200Mi 500Mi

No resource limits.
```

7. Remove the namespace line from the `nfs-pod.yaml` file. Should be around line 11 or so. This will allow us to pass other namespaces on the command line.

```
student@cp:~$ vim nfs-pod.yaml
```

8. Create the container again.

```
student@cp:~$ kubectl -n small create -f nfs-pod.yaml
```

```
deployment.apps/nginx-nfs created
```

9. Determine if the deployment has a running pod.

```
student@cp:~$ kubectl -n small get deploy
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-nfs	1/1	1	1	43s

```
student@cp:~$ kubectl -n small describe deploy nginx-nfs
```

<output_omitted>

10. Look to see if the pods are ready.

```
student@cp:~$ kubectl -n small get pod
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-nfs-2854978848-g3khf	1/1	Running	0	37s

11. Ensure the Pod is running and is using the NFS mounted volume. If you pass the namespace first Tab will auto-complete the pod name.

```
student@cp:~$ kubectl -n small describe pod \
    nginx-nfs-2854978848-g3khf
```

```
Name:          nginx-nfs-2854978848-g3khf
Namespace:     small
<output_omitted>
```

```
Mounts:
  /opt from nfs-vol (rw)
<output_omitted>
```

12. View the quota usage of the namespace

```
student@cp:~$ kubectl describe ns small
```

```
<output_omitted>
```

Resource Quotas

Name:	storagequota	
Resource	Used	Hard
-----	---	---
persistentvolumeclaims	1	10
requests.storage	200Mi	500Mi

```
No resource limits.
```

13. Create a 300M file inside of the `/opt/sfw` directory on the host and view the quota usage again. Note that with NFS the size of the share is not counted against the deployment.

```
student@cp:~$ sudo dd if=/dev/zero of=/opt/sfw/bigfile bs=1M count=300
```

```
300+0 records in
300+0 records out
314572800 bytes (315 MB, 300 MiB) copied, 0.196794 s, 1.6 GB/s
```

```
student@cp:~$ kubectl describe ns small
```

```
<output_omitted>
```

Resource Quotas

Name:	storagequota	
Resource	Used	Hard
-----	---	---
persistentvolumeclaims	1	10

```
requests.storage      200Mi      500Mi
<output_omitted>
```

```
student@cp:~$ du -h /opt/
```

```
301M      /opt/sfw
41M       /opt/cni/bin
41M       /opt/cni
341M      /opt/
```

14. Now let us illustrate what happens when a deployment requests more than the quota. Begin by shutting down the existing deployment.

```
student@cp:~$ kubectl -n small get deploy
```

```
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-nfs     1       1            1           11m
```

```
student@cp:~$ kubectl -n small delete deploy nginx-nfs
```

```
deployment.apps "nginx-nfs" deleted
```

15. Once the Pod has shut down view the resource usage of the namespace again. Note the storage did not get cleaned up when the pod was shut down.

```
student@cp:~$ kubectl describe ns small
```



```
<output_omitted>
Resource Quotas
Name:                storagequota
Resource             Used    Hard
-----
persistentvolumeclaims 1      10
requests.storage      200Mi 500Mi
```

16. Remove the pvc then view the pv it was using. Note the RECLAIM POLICY and STATUS.

```
student@cp:~$ kubectl -n small get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESSMODES	STORAGECLASS	AGE
pvc-one	Bound	pvvol-1	1Gi	RWX		19m

```
student@cp:~$ kubectl -n small delete pvc pvc-one
```

```
persistentvolumeclaim "pvc-one" deleted
```

```
student@cp:~$ kubectl -n small get pv
```

NAME	CAPACITY	ACCESSMODES	RECLAIMPOLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pvvol-1	1Gi	RWX	Retain	Released	small/pvc-one 44m

17. Dynamically provisioned storage uses the ReclaimPolicy of the StorageClass which could be Delete, Retain, or some types allow Recycle. Manually created persistent volumes default to Retain unless set otherwise at creation. The default storage policy is to retain the storage to allow recovery of any data. To change this begin by viewing the

yaml output.

```
student@cp:~$ kubectl get pv/pvvol-1 -o yaml
```

YAML

```
1  ....
2      path: /opt/sfw
3      server: cp
4      persistentVolumeReclaimPolicy: Retain
5  status:
6      phase: Released
7
```

18. Currently we will need to delete and re-create the object. Future development on a deleter plugin is planned. We will re-create the volume and allow it to use the Retain policy, then change it once running.

```
student@cp:~$ kubectl delete pv/pvvol-1
```

```
persistentvolume "pvvol-1" deleted
```

```
student@cp:~$ grep Retain PVol.yaml
```

```
persistentVolumeReclaimPolicy: Retain
```

```
student@cp:~$ kubectl create -f PVol.yaml
```

```
persistentvolume "pvvol-1" created
```

19. We will use `kubectl patch` to change the retention policy to `Delete`. The yaml output from before can be helpful in getting the correct syntax.

```
student@cp:~$ kubectl patch pv pvvol-1 -p \
'{"spec":{"persistentVolumeReclaimPolicy":"Delete"}}'
```

```
persistentvolume/pvvol-1 patched
```

```
student@cp:~$ kubectl get pv/pvvol-1
```

NAME	CAPACITY	ACCESSMODES	RECLAIMPOLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
pvvol-1	1Gi	RWX	Delete	Available	2m

20. View the current quota settings.

```
student@cp:~$ kubectl describe ns small
```

```
....
requests.storage      0      500Mi
```

21. Create the pvc again. Even with no pods running, note the resource usage.

```
student@cp:~$ kubectl -n small create -f pvc.yaml
```

```
persistentvolumeclaim/pvc-one created
```

```
student@cp:~$ kubectl describe ns small
```

```
....  
requests.storage      200Mi      500Mi
```

22. Remove the existing quota from the namespace.

```
student@cp:~$ kubectl -n small get resourcequota
```

```
NAME          CREATED AT  
storagequota  2026-03-19T04:10:02Z
```

```
student@cp:~$ kubectl -n small delete resourcequota storagequota
```

```
resourcequota "storagequota" deleted
```

23. Edit the storagequota.yaml file and lower the capacity to 100Mi.

```
student@cp:~$ vim storage-quota.yaml
```

YA
ML

```
1 ....  
2   requests.storage: "100Mi"  
3
```

24. Create and verify the new storage quota. Note the hard limit has already been exceeded.

```
student@cp:~$ kubectl -n small create -f storage-quota.yaml
```

```
resourcequota/storagequota created
```

```
student@cp:~$ kubectl describe ns small
```

```
....
persistentvolumeclaims      1          10
requests.storage            200Mi      100Mi

No resource limits.
```

25. Create the deployment again. View the deployment. Note there are no errors seen.

```
student@cp:~$ kubectl -n small create -f nfs-pod.yaml
```

```
deployment.apps/nginx-nfs created
```

```
student@cp:~$ kubectl -n small describe deploy/nginx-nfs
```

```
Name:                nginx-nfs
Namespace:            small
<output_omitted>
```

26. Examine the pods to see if they are actually running.

```
student@cp:~$ kubectl -n small get po
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-nfs-2854978848-vb6bh	1/1	Running	0	58s

27. As we were able to deploy more pods even with apparent hard quota set, let us test to see if the reclaim of storage takes place. Remove the deployment and the persistent volume claim.

```
student@cp:~$ kubectl -n small delete deploy nginx-nfs
```

```
deployment.apps "nginx-nfs" deleted
```

```
student@cp:~$ kubectl -n small delete pvc/pvc-one
```

```
persistentvolumeclaim "pvc-one" deleted
```

28. View if the persistent volume exists. You will see it attempted a removal, but failed. If you look closer you will find the error has to do with the lack of a deleter volume plugin for NFS. Other storage protocols have a plugin.

```
student@cp:~$ kubectl -n small get pv
```

NAME	CAPACITY	ACCESSMODES	RECLAIMPOLICY	STATUS	CLAIM
	STORAGECLASS	REASON	AGE		
pvvol-1	1Gi	RWX	Delete	Failed	small/pvc-one 20m

29. Ensure the deployment, pvc and pv are all removed.

```
student@cp:~$ kubectl delete pv/pvvol-1
```

```
persistentvolume "pvvol-1" deleted
```

30. Edit the persistent volume YAML file and change the `persistentVolumeReclaimPolicy`: to Recycle.

```
student@cp:~$ vim PVol.yaml
```



PVol.yaml

```

1 ....
2   persistentVolumeReclaimPolicy: Recycle
3 ....
4

```



Recycle reclaim policy

We are aware that the **Recycle** reclaim policy has been deprecated. The recommended approach is to use **dynamic provisioning** instead. This policy is included here for **educational purposes**, as it is still encountered in some **existing environments**.

31. Add a LimitRange to the namespace and attempt to create the persistent volume and persistent volume claim again. We can use the LimitRange we used earlier.

```
student@cp:~$ kubectl -n small create -f low-resource-range.yaml
```

```
limitrange/low-resource-range created
```

32. View the settings for the namespace. Both quotas and resource limits should be seen.

```
student@cp:~$ kubectl describe ns small
```

```

<output_omitted>
Resource Limits
  Type      Resource  Min  Max  Default Request  Default Limit  ...
  ---      -
Container  cpu       -    -    500m             1              -
Container  memory   -    -    100Mi            500Mi          -

```

33. Create the persistent volume again. View the resource. Note the Reclaim Policy is Recycle.

```
student@cp:~$ kubectl -n small create -f PVol.yaml
```

```
persistentvolume/pvvol-1 created
```

```
student@cp:~$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	...
pvvol-1	1Gi	RWX	Recycle	Available	...

34. Attempt to create the **PersistentVolumeClaim** again. The request is rejected because it exceeds the defined **storage quota**, which is enforced based on the specified **storage request**.

```
student@cp:~$ kubectl -n small create -f pvc.yaml
```

```
Error from server (Forbidden): error when creating "pvc.yaml":
persistentvolumeclaims "pvc-one" is forbidden: exceeded quota:
storagequota, requested: requests.storage=200Mi, used:
requests.storage=0, limited: requests.storage=100Mi
```

35. Edit the resourcequota to increase the requests.storage to 500Mi.

```
student@cp:~$ kubectl -n small edit resourcequota
```




```
1 ....
2 spec:
3   hard:
4     persistentvolumeclaims: "10"
5     requests.storage: 500Mi
6 status:
7   hard:
8     persistentvolumeclaims: "10"
9 ....
10
```

36. Create the pvc again. It should work this time. Then create the deployment again.

```
student@cp:~$ kubectl -n small create -f pvc.yaml
```

```
persistentvolumeclaim/pvc-one created
```

```
student@cp:~$ kubectl -n small create -f nfs-pod.yaml
```

```
deployment.apps/nginx-nfs created
```

37. View the namespace settings.

```
student@cp:~$ kubectl describe ns small
```

```
<output_omitted>
```

38. Delete the deployment. View the status of the pv and pvc.

```
student@cp:~$ kubectl -n small delete deploy nginx-nfs
```

```
deployment.apps "nginx-nfs" deleted
```

```
student@cp:~$ kubectl -n small get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
pvc-one	Bound	pvvol-1	1Gi	RWX		7m

```
student@cp:~$ kubectl -n small get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	...
pvvol-1	1Gi	RWX	Recycle	Bound	small/pvc-one	...

39. Delete the pvc and check the status of the pv. It should show as Available.

```
student@cp:~$ kubectl -n small delete pvc pvc-one
```

```
persistentvolumeclaim "pvc-one" deleted
```

```
student@cp:~$ kubectl -n small get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORA...
pvvol-1	1Gi	RWX	Recycle	Available		...

40. Remove the pv and any other resources created during this lab.

```
student@cp:~$ kubectl delete pv pvvol-1
```

```
persistentvolume "pvvol-1" deleted
```

Exercise 9.5: Using StorageClass to Dynamically provision a volume

StorageClasses in Kubernetes simplify and automate the process of provisioning and managing storage resources, provide users with the flexibility to choose appropriate storage types for their workloads, and help administrators enforce policies and manage storage infrastructure more effectively. StorageClasses enables dynamic provisioning of storage resources. Without StorageClasses, administrators have to manually create PersistentVolumes (PVs) for each PersistentVolumeClaim (PVC) made by users. With StorageClasses, this process is automated. When a user creates a PVC and specifies a StorageClasses, the system automatically creates a corresponding PV that meets the requirements.

1. Begin by listing to see if we have any storage class available on our cluster.

```
student@cp:~$ kubectl get sc
```

```
No resources found
```

2. We don't have any StorageClass created. Before we can create the sc, we need to deploy the provisioner. Kubernetes doesn't include an internal NFS provisioner. We need to use an external provisioner to create a StorageClass for NFS. Let us deploy a nfs provisioner.

```
student@cp:~$ helm repo add nfs-subdir-external-provisioner \  
https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/
```

```
"nfs-subdir-external-provisioner" has been added to your repositories
```

```
student@cp:~$ helm install nfs-subdir-external-provisioner \  
nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
```

```
--set nfs.server=cp \  
--set nfs.path=/opt/sfw/
```

```
NAME: nfs-subdir-external-provisioner  
LAST DEPLOYED: Mon Mar 19 12:11:39 2026  
NAMESPACE: default  
STATUS: deployed  
REVISION: 1  
TEST SUITE: None
```

3. The installation also created a StorageClass for us.

```
student@cp:~$ kubectl get sc
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
↪ ALLOWVOLUMEEXPANSION	AGE		
nfs-client	cluster.local/nfs-subdir-external-provisioner	Delete	Immediate
↪ true	12m		

4. List to see if there are any PV and PVC available. Clean up in previous lab should have removed all of them.

```
student@cp:~$ kubectl get pv,pvc
```

```
No resources found
```

5. Create a YAML file for the new pvc.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/pvc-sc.yaml .
```

```
student@cp:~$ vim pvc-sc.yaml
```



pvc-sc.yaml

```

1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: pvc-two
5  spec:
6    storageClassName: nfs-client
7    accessModes:
8      - ReadWriteMany
9    resources:
10     requests:
11       storage: 200Mi
12

```

6. Create and verify when the new pvc is created, a dynamic volume is provisioned.

```
student@cp:~$ kubectl create -f pvc-sc.yaml
```

```
persistentvolumeclaim/pvc-two created
```

```
student@cp:~$ kubectl get pv,pvc
```

NAME				CAPACITY	ACCESS	MODES	RECLAIM
↪ POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE		
persistentvolume/pvc-71149612-33f1-4b18-916d-c67f79aca797	200Mi	RWX	Delete				
↪ Bound	default/pvc-two	nfs-client	28s				

NAME		STATUS	VOLUME	CAPACITY	
↪ ACCESS	MODES	STORAGECLASS	AGE		
persistentvolumeclaim/pvc-two	Bound	pvc-71149612-33f1-4b18-916d-c67f79aca797	200Mi	RWX	
↪ nfs-client	28s				

7. Create a new pod to use the pvc.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_09/pod-sc.yaml .
```

```
student@cp:~$ vim pod-sc.yaml
```

YA
ML

pod-sc.yaml

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: web-server
5  spec:
6    containers:
7    - image: nginx:1.29.6
8      name: web-container
9      volumeMounts:
10     - name: nfs-volume
11       mountPath: /usr/share/nginx/html
12   volumes:
13   - name: nfs-volume
14     persistentVolumeClaim:
15       claimName: pvc-two
```

8. Create the pod using the file.

```
student@cp:~$ kubectl create -f pod-sc.yaml
```

```
pod/web-server created
```

9. Create a new file and copy it inside the pod.

```
student@cp:~$ echo "Welcome to the demo of storage class" > index.html
student@cp:~$ kubectl cp index.html web-server:/usr/share/nginx/html
```

10. The file was copied on to the default location of the nginx server. Instead of the ephemeral read-write layer of the container, the file is saved on the NFS server as we have made use of the PV.

```
student@cp:~$ ls -l /opt/sfw/default-pvc-two-pvc-<Hit the Tab key>
```

```
-rw-rw-r-- 1 student student 37 Jan  8 13:08 index.html
```

11. Cleanup by deleting the pod,volume claim.

```
student@cp:~$ kubectl delete pod/web-server pvc/pvc-two
```

```
pod "web-server" deleted
persistentvolumeclaim "pvc-two" deleted
```


Chapter 10

Services



10.1	Overview	302
10.2	Accessing Services	304
10.3	DNS	310
10.4	Labs	312

10.1 Overview

Overview

- Essential to micro-service architecture
- Connect Pods together, or outside cluster
- Service abstraction
- Load balancing
- Service types
- DNS resolution

Service Update Pattern

- Uses labels to match target Pods
- Rolling Deployments
- Traffic Shift

10.2 Accessing Services

Accessing an Application With A Service

```
$ kubectl expose deployment/nginx --port=80 --type=NodePort
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.0.0.1	<none>	443/TCP	18h
nginx	NodePort	10.0.0.112	<none>	80:31230/TCP	5s

```
$ kubectl get svc nginx -o yaml
```

```
apiVersion: v1
kind: Service
...
spec:
  clusterIP: 10.0.0.112
  ports:
    - nodePort: 31230
  ...
```

Open browser <http://Public-IP:31230>

Service Types

- ClusterIP
- NodePort
- LoadBalancer
- ExternalName

Service Types (cont)

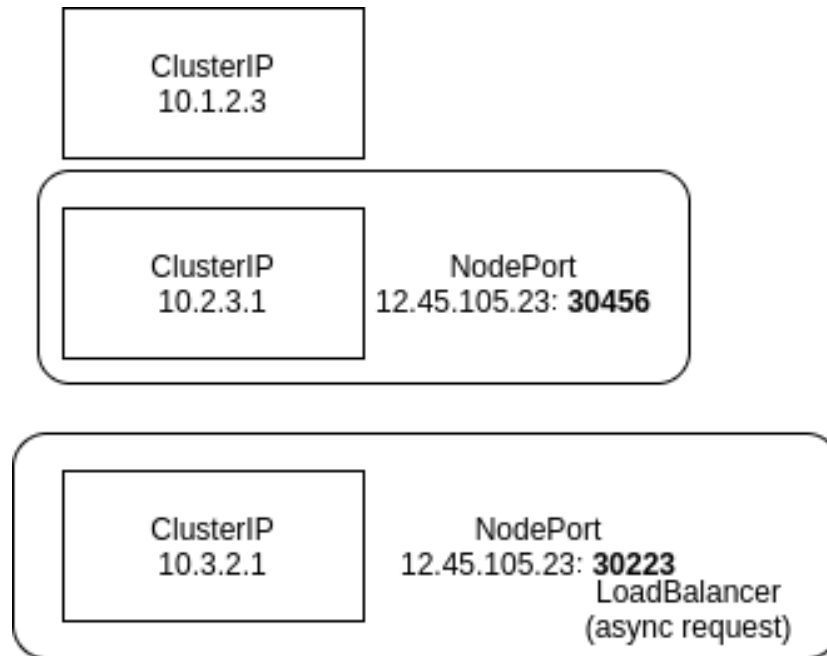


Figure 10.1: **Built in services**

Services Diagram

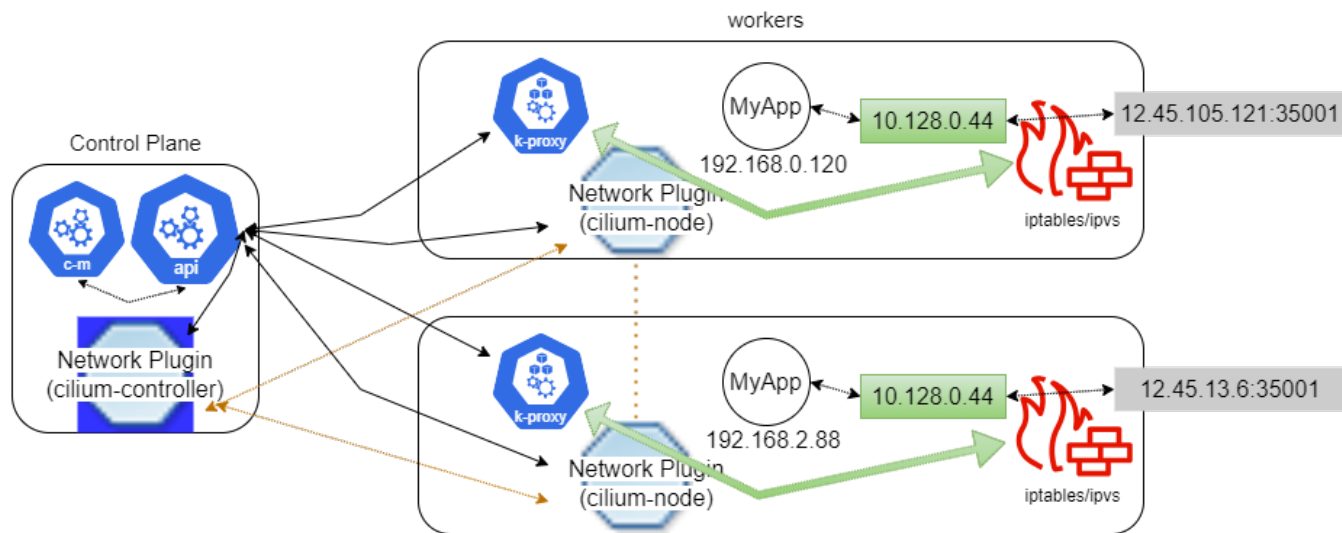


Figure 10.2: Service Traffic

Overall Network View

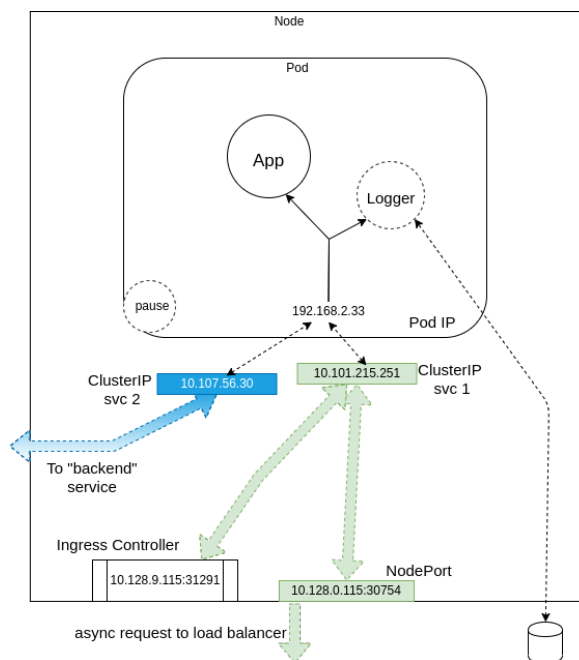


Figure 10.3: **Example of Cluster Networking**

Local Proxy For Development

- Quick way to check service
- Available on localhost
- Not exposed to Internet

```
$ kubectl proxy
```

```
Starting to serve on 127.0.0.1:8001
```

10.3 DNS

DNS

- DNS provided using **CoreDNS** by default as of v1.13
- Exposed via the **kube-dns** service.
- Server started for zones served
- Configured via a `configmap`
- Each loads plugin chains to provide services
 - `tls`
 - `prometheus`
 - `health`
 - `errors`

Verifying DNS Registration

- Verify the service and the pod
- **netstat**
- **dig**
- **nc**
- Network Policy

10.4 Labs

Exercise 10.1: Deploy A New Service

Overview

Services (also called **microservices**) are objects which declare a policy to access a logical set of Pods. They are typically assigned with `labels` to allow persistent access to a resource, when front or back end containers are terminated and replaced.

Native applications can use the `Endpoints` API for access. Non-native applications can use a Virtual IP-based bridge to access back end pods. `ServiceTypes` Type could be:

- **ClusterIP** default - exposes on a cluster-internal IP. Only reachable within cluster
- **NodePort** Exposes node IP at a static port. A ClusterIP is also automatically created.
- **LoadBalancer** Exposes service externally using cloud providers load balancer. NodePort and ClusterIP automatically created.
- **ExternalName** Maps service to contents of `externalName` using a CNAME record.

We use services as part of decoupling such that any agent or object can be replaced without interruption to access from client to back end application.

1. Deploy two **nginx** servers using **kubectl** and a new `.yaml` file. The kind should be `Deployment` and label it with `nginx`. Create two replicas and expose port 8080. What follows is a well documented file. There is no need to include the comments when you create the file. This file can also be found among the other examples in the tarball.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_10/nginx-one.yaml .
```

```
student@cp:~$ vim nginx-one.yaml
```



nginx-one.yaml

```
1  apiVersion: apps/v1
2  # Determines YAML versioned schema.
3  kind: Deployment
4  # Describes the resource defined in this file.
5  metadata:
6    name: nginx-one
7    labels:
8      system: secondary
9  # Required string which defines object within namespace.
10  namespace: accounting
11  # Existing namespace resource will be deployed into.
12  spec:
13    selector:
14      matchLabels:
15        system: secondary
16  # Declaration of the label for the deployment to manage
17  replicas: 2
18  # How many Pods of following containers to deploy
19  template:
20    metadata:
21      labels:
22        system: secondary
23  # Some string meaningful to users, not cluster. Keys
24  # must be unique for each object. Allows for mapping
25  # to customer needs.
26  spec:
27    containers:
28  # Array of objects describing containerized application with a Pod.
```



```
29 # Referenced with shorthand spec.template.spec.containers
30   - image: nginx:1.20.1
31 # The Docker image to deploy
32   imagePullPolicy: Always
33   name: nginx
34 # Unique name for each container, use local or Docker repo image
35   ports:
36     - containerPort: 8080
37     protocol: TCP
38 # Optional resources this container may need to function.
39   nodeSelector:
40     system: secondOne
41 # One method of node affinity.
```

2. View the existing labels on the nodes in the cluster.

```
student@cp:~$ kubectl get nodes --show-labels
```

```
<output_omitted>
```

3. Run the following command and look for the errors. Assuming there is no typo, you should have gotten an error about about the accounting namespace.

```
student@cp:~$ kubectl create -f nginx-one.yaml
```

```
Error from server (NotFound): error when creating
"nginx-one.yaml": namespaces "accounting" not found
```

4. Create the namespace and try to create the deployment again. There should be no errors this time.

```
student@cp:~$ kubectl create ns accounting
```

```
namespace/accounting" created
```

```
student@cp:~$ kubectl create -f nginx-one.yaml
```

```
deployment.apps/nginx-one created
```

5. View the status of the new pods. Note they do not show a Running status.

```
student@cp:~$ kubectl -n accounting get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-one-74dd9d578d-fcpmv	0/1	Pending	0	4m
nginx-one-74dd9d578d-r2d67	0/1	Pending	0	4m

6. View the node each has been assigned to (or not) and the reason, which shows under events at the end of the output.

```
student@cp:~$ kubectl -n accounting describe pod nginx-one-74dd9d578d-fcpmv
```

```
Name:          nginx-one-74dd9d578d-fcpmv
Namespace:     accounting
Node:          <none>

<output_omitted>

Events:
  Type    Reason             Age          From          ....
  ----    -
  Warning  FailedScheduling   <unknown>    default-scheduler
0/2 nodes are available: 2 node(s) didn't match node selector.
```

7. Label the secondary node. Note the value is case sensitive. Verify the labels.

```
student@cp:~$ kubectl label node <worker_node_name> system=secondOne
```

```
node/worker labeled
```

```
student@cp:~$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
cp	Ready	control-plane	14h	v1.35.2	beta.kubernetes.io/arch=amd64, beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=cp, kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=, node.kubernetes.io/exclude-from-external-load-balancers=
worker	Ready	<none>	14h	v1.35.2	beta.kubernetes.io/arch=amd64, beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker, kubernetes.io/os=linux

8. View the pods in the accounting namespace. They may still show as Pending. Depending on how long it has been since you attempted deployment the system may not have checked for the label. If the Pods show Pending after a minute delete one of the pods. They should both show as Running after a deletion. A change in state will cause the Deployment controller to check the status of both Pods.

```
student@cp:~$ kubectl -n accounting get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-one-74dd9d578d-fcpmv	1/1	Running	0	10m
nginx-one-74dd9d578d-sts5l	1/1	Running	0	3s

9. View Pods by the label we set in the YAML file. If you look back the Pods were given a label of `app=nginx`.

```
student@cp:~$ kubectl get pods -l system=secondary --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
accounting	nginx-one-74dd9d578d-fcpmv	1/1	Running	0	20m
accounting	nginx-one-74dd9d578d-sts5l	1/1	Running	0	9m

10. Recall that we exposed port 8080 in the YAML file. Expose the new deployment.

```
student@cp:~$ kubectl -n accounting expose deployment nginx-one
```

```
service/nginx-one exposed
```

11. View the newly exposed endpoints. Note that port 8080 has been exposed on each Pod.

```
student@cp:~$ kubectl -n accounting get ep nginx-one
```

NAME	ENDPOINTS	AGE
nginx-one	192.168.1.72:8080,192.168.1.73:8080	47s

12. Attempt to access the Pod on port 8080, then on port 80. Even though we exposed port 8080 of the container the application within has not been configured to listen on this port. The **nginx** server listens on port 80 by default. A `curl` command to that port should return the typical welcome page.

```
student@cp:~$ curl 192.168.1.72:8080
```

```
curl: (7) Failed to connect to 192.168.1.72 port 8080: Connection refused
```

```
student@cp:~$ curl 192.168.1.72:80
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

13. Delete the deployment. Edit the YAML file to expose port 80 and create the deployment again.

```
student@cp:~$ kubectl -n accounting delete deploy nginx-one
```

```
deployment.apps "nginx-one" deleted
```

```
student@cp:~$ vim nginx-one.yaml
```

YA
ML

nginx-one.yaml

```
1 ....
2     ports:
3     - containerPort: 8080    #<-- Edit this line
4       protocol: TCP
5     ....
6
```

```
student@cp:~$ kubectl create -f nginx-one.yaml
```

```
deployment.apps/nginx-one created
```

Exercise 10.2: Configure a NodePort

In a previous exercise we deployed a `LoadBalancer` which deployed a `ClusterIP` and `NodePort` automatically. In this exercise we will deploy a `NodePort`. While you can access a container from within the cluster, one can use a `NodePort` to NAT traffic from outside the cluster. One reason to deploy a `NodePort` instead, is that a `LoadBalancer` is also a load balancer resource from cloud providers like GKE and AWS.

1. In a previous step we were able to view the **nginx** page using the internal Pod IP address. Now expose the deployment using the `--type=NodePort`. We will also give it an easy to remember name and place it in the `accounting` namespace. We could pass the port as well, which could help with opening ports in the firewall.

```
student@cp:~$ kubectl -n accounting expose deployment nginx-one --type=NodePort --name=service-lab
```

```
service/service-lab exposed
```

2. View the details of the services in the `accounting` namespace. We are looking for the autogenerated port.

```
student@cp:~$ kubectl -n accounting describe services
```

```
....  
NodePort:                <unset> 32103/TCP  
....
```

3. Locate the exterior facing hostname or IP address of the cluster. The lab assumes use of GCP nodes, which we access via a `FloatingIP`, we will first check the internal only public IP address. Look for the Kubernetes cp URL. Whichever way you access check access using both the internal and possible external IP address

```
student@cp:~$ kubectl cluster-info
```

```
Kubernetes control plane is running at https://k8scp:6443  
CoreDNS is running at https://k8scp:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy  
  
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

4. Test access to the **nginx** web server using the combination of cp URL and NodePort.

```
student@cp:~$ curl http://k8scp:32103
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

5. Using the browser on your local system, use the public IP address you use to SSH into your node and the port. You should still see the **nginx** default page. You may be able to use **curl** to locate your public IP address.

```
student@cp:~$ curl ifconfig.io
```

```
104.198.192.84
```

Exercise 10.3: Working with CoreDNS

1. We can leverage **CoreDNS** and predictable hostnames instead of IP addresses. A few steps back we created the service-lab NodePort in the Accounting namespace. We will create a new pod for testing using Ubuntu. The pod name will be named ubuntu.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_10/nettool.yaml .
```

```
student@cp:~$ vim nettool.yaml
```



nettool.yaml

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: ubuntu
5  spec:
6    containers:
7      - name: ubuntu
8        image: ubuntu:latest
9        command: [ "sleep" ]
10       args: [ "infinity" ]
11
```

2. Create the pod and then log into it.

```
student@cp:~$ kubectl create -f nettool.yaml
```

```
pod/ubuntu created
```

```
student@cp:~$ kubectl exec -it ubuntu -- /bin/bash
```



On Container

- (a) Add some tools for investigating DNS and the network. The installation will ask you the geographic area and timezone information. Someone in Austin would first answer 2. America, then 37 for Chicago, which would be central time

```
root@ubuntu:/# apt-get update ; apt-get install curl dnsutils -y
```

- (b) Use the **dig** command with no options. You should see root name servers, and then information about the



DNS server responding, such as the IP address.

```
root@ubuntu:/# dig
```

```
; <<>> DiG 9.16.1-Ubuntu <<>>
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3394
;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1

<output_omitted>

;; Query time: 4 msec
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; WHEN: Thu Mar 19 22:06:18 CDT 2026
;; MSG SIZE rcvd: 431
```

- (c) Also take a look at the `/etc/resolv.conf` file, which will indicate nameservers and default domains to search if no using a Fully Qualified Distinguished Name (FQDN). From the output we can see the first entry is `default.svc.cluster.local..`

```
root@ubuntu:/# cat /etc/resolv.conf
```

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
c.endless-station-188822.internal google.internal
options ndots:5
```

- (d) Use the **dig** command to view more information about the DNS server. Use the **-x** argument to get the FQDN using the IP we know. Notice the domain name, which uses `.kube-system.svc.cluster.local.`,



to match the pod namespaces instead of default. Also note the name, kube-dns, is the name of a service not a pod.

```
root@ubuntu:/# dig @10.96.0.10 -x 10.96.0.10
```

```
...  
;; QUESTION SECTION:  
;10.0.96.10.in-addr.arpa.      IN      PTR  
  
;; ANSWER SECTION:  
10.0.96.10.in-addr.arpa.  
-> 30          IN      PTR          kube-dns.kube-system.svc.cluster.local.  
  
;; Query time: 0 msec  
;; SERVER: 10.96.0.10#53(10.96.0.10)  
;; WHEN: Thu Mar 19 23:39:14 CDT 2026  
;; MSG SIZE  rcvd: 139
```

- (e) Recall the name of the service-lab service we made and the namespaces it was created in. Use this information to create a FQDN and view the exposed pod.

```
root@ubuntu:/# curl service-lab.accounting.svc.cluster.local.
```



```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
...

```

- (f) Attempt to view the default page using just the service name. It should fail as `nettool` is in the default namespace.

```
root@ubuntu:/# curl service-lab
```

```
curl: (6) Could not resolve host: service-lab
```

- (g) Add the `accounting` namespaces to the name and try again. Traffic can access a service using a name, even across different namespaces.

```
root@ubuntu:/# curl service-lab.accounting
```




```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

- (h) Exit out of the container and look at the services running inside of the `kube-system` namespace. From the output we see that the `kube-dns` service has the DNS serverIP, and exposed ports DNS uses.

```
root@ubuntu:/# exit
```

```
student@cp:~$ kubectl -n kube-system get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP	42h

- Examine the service in detail. Among other information notice the selector in use to determine the pods the service communicates with.

```
student@cp:~$ kubectl -n kube-system get svc kube-dns -o yaml
```

```
...
  labels:
    k8s-app: kube-dns
    kubernetes.io/cluster-service: "true"
    kubernetes.io/name: CoreDNS
  ...
  selector:
    k8s-app: kube-dns
```

```

    sessionAffinity: None
    type: ClusterIP
...

```

4. Find pods with the same labels in all namespaces. We see that infrastructure pods all have this label, including **coredns**.

```
student@cp:~$ kubectl get pod -l k8s-app --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	cilium-5tv9d	1/1	Running	0	136m
kube-system	cilium-gzdk6	1/1	Running	0	54m
kube-system	coredns-5d78c9869d-44qvq	1/1	Running	0	31m
kube-system	coredns-5d78c9869d-j6tqx	1/1	Running	0	31m
kube-system	kube-proxy-lpsmq	1/1	Running	0	35m
kube-system	kube-proxy-pvl8w	1/1	Running	0	34m

5. Look at the details of one of the **coredns** pods. Read through the pod spec and find the image in use as well as any configuration information. You should find that configuration comes from a configmap.

```
student@cp:~$ kubectl -n kube-system get pod coredns-f9fd979d6-4dxpl -o yaml
```

```

...
spec:
  containers:
  - args:
    - -conf
    - /etc/coredns/Corefile
    image: k8s.gcr.io/coredns:1.7.0
...
  volumeMounts:
  - mountPath: /etc/coredns
    name: config-volume

```

```
        readOnly: true
    ...
    volumes:
    - configMap:
        defaultMode: 420
        items:
        - key: Corefile
          path: Corefile
        name: coredns
        name: config-volume
    ...
```

6. View the configmaps in the kube-system namespace.

```
student@cp:~$ kubectl -n kube-system get configmaps
```

NAME	DATA	AGE
cilium-config	4	43h
coredns	1	43h
extension-apiserver-authentication	6	43h
kube-proxy	2	43h
kubeadm-config	2	43h
kubelet-config	1	43h

7. View the details of the coredns configmap. Note the cluster.local domain is listed.

```
student@cp:~$ kubectl -n kube-system get configmaps coredns -o yaml
```

```
apiVersion: v1
data:
  Corefile: |
    .:53 {
```

```
    errors
    health {
        lameduck 5s
    }
    ready
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
    }
    prometheus :9153
    forward . /etc/resolv.conf {
        max_concurrent 1000
    }
    cache 30
    loop
    reload
    loadbalance
}
kind: ConfigMap
...
```

8. It is very important to backup our resources before we make changes to it.

```
student@cp:~$ kubectl -n kube-system get configmaps coredns -o yaml > coredns-backup.yaml
```

9. While there are many options and zone files we could configure, let's start with simple edit. Add a rewrite statement such that example.com will redirect to cluster.local. More about each line can be found at coredns.io.

```
student@cp:~$ kubectl -n kube-system edit configmaps coredns
```

```
apiVersion: v1
data:
  Corefile: |
```

```
.:53 {
    rewrite name regex (.*)\.example\.com {1}.default.svc.cluster.local    #<-- Add this line
    errors
    health {
        lameduck 5s
    }
    ready
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
    }
    prometheus :9153
    forward . /etc/resolv.conf {
        max_concurrent 1000
    }
    cache 30
    loop
    reload
    loadbalance
}
```

10. Delete the coredns pods causing them to re-read the updated configmap.

```
student@cp:~$ kubectl -n kube-system rollout restart deploy coredns
```

```
deployment.apps/coredns restarted
```

11. Create a new web server and create a ClusterIP service to verify the address works. Note the new service IP to start with a reverse lookup.

```
student@cp:~$ kubectl create deployment nginx --image=nginx
```

```
deployment.apps/nginx created
```

```
student@cp:~$ kubectl expose deployment nginx --type=ClusterIP --port=80
```

```
service/nginx expose
```

```
student@cp:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	3d15h
nginx	ClusterIP	10.104.248.141	<none>	80/TCP	7s

12. Log into the ubuntu container and test the URL rewrite starting with the reverse IP resolution.

```
student@cp:~$ kubectl exec -it ubuntu -- /bin/bash
```



On Container

- (a) Use the **dig** command. Note that the service name becomes part of the FQDN.

```
root@ubuntu:/# dig -x 10.104.248.141
```



```
....  
;; QUESTION SECTION:  
;141.248.104.10.in-addr.arpa.      IN      PTR  
  
;; ANSWER SECTION:  
141.248.104.10.in-addr.arpa.  
→ 30          IN          PTR      nginx.default.svc.cluster.local.  
....
```

- (b) Now that we have the reverse lookup test the forward lookup. The IP should match the one we used in the previous step.

```
root@ubuntu:/# dig nginx.default.svc.cluster.local.
```

```
....  
;; QUESTION SECTION:  
;nginx.default.svc.cluster.local. IN      A  
  
;; ANSWER SECTION:  
nginx.default.svc.cluster.local. 30 IN      A      10.104.248.141  
....
```

- (c) Now test to see if the rewrite rule for the `test.io` domain we added resolves the IP. Note the response uses the original name, not the requested FQDN.

```
root@ubuntu:/# dig nginx.test.io
```



```
....
;; QUESTION SECTION:
;nginx.test.io.                IN      A

;; ANSWER SECTION:
nginx.default.svc.cluster.local. 30 IN      A      10.104.248.141
....
```

13. Exit out of the container then edit the configmap to add an answer section.

```
student@cp:~$ kubectl -n kube-system edit configmaps coredns
```

```
....
data:
  Corefile: |
    .:53 {
      rewrite stop {                                #<-- Edit this and following two lines
        name regex (.*)\.test\.io {1}.default.svc.cluster.local
        answer name (.*)\.default\.svc\.cluster\.local {1}.test.io
      }
      errors
      health {
    ....
```

14. Delete the coredns pods again to ensure they re-read the updated configmap.

```
student@cp:~$ kubectl -n kube-system delete pod coredns-f9fd979d6-fv9qn coredns-f9fd979d6-lnxn5
```



```
pod "coredns-f9fd979d6-fv9qn" deleted
pod "coredns-f9fd979d6-lnxn5" deleted
```

15. Log into the ubuntu container again. This time the response should show the FQDN with the requested FQDN.

```
student@cp:~$ kubectl exec -it ubuntu -- /bin/bash
```



On Container

```
root@ubuntu:/# dig nginx.test.io
```

```
....
;; QUESTION SECTION:
nginx.test.io.                IN      A

;; ANSWER SECTION:
nginx.test.io.                30      IN      A      10.104.248.141
....
```

16. Exit then delete the DNS test tools container to recover the resources.

```
student@cp:~$ kubectl delete -f nettool.yaml
```

Exercise 10.4: Use Labels to Manage Resources

1. Try to delete all Pods with the `system=secondary` label, in all namespaces.

```
student@cp:~$ kubectl delete pods -l system=secondary \
--all-namespaces
```

```
pod "nginx-one-74dd9d578d-fcpmv" deleted
pod "nginx-one-74dd9d578d-sts5l" deleted
```

2. View the Pods again. New versions of the Pods should be running as the controller responsible for them continues.

```
student@cp:~$ kubectl -n accounting get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-one-74dd9d578d-ddt5r	1/1	Running	0	1m
nginx-one-74dd9d578d-hfzml	1/1	Running	0	1m

3. We also gave a label to the deployment. View the deployment in the accounting namespace.

```
student@cp:~$ kubectl -n accounting get deploy --show-labels
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	LABELS
nginx-one	2/2	2	2	10m	system=secondary

4. Delete the deployment using its label.

```
student@cp:~$ kubectl -n accounting delete deploy -l system=secondary
```

```
deployment.apps "nginx-one" deleted
```

5. Remove the label from the secondary node. Note that the syntax is a minus sign directly after the key you want to remove, or system in this case.

```
student@cp:~$ kubectl label node worker system-
```

```
node/worker unlabeled
```


Chapter 11

Advanced Traffic Management



11.1	Overview	338
11.2	Ingress Controller	339
11.3	Ingress Rules	344
11.4	Service Mesh	346
11.5	Limitations	347
11.6	Gateway API	348
11.7	Labs	353

11.1 Overview

Ingress Overview

- Single endpoint to cluster
- Manage external access to services in cluster
 - HTTP
 - Load-balancing
 - SSL termination
 - Name-based virtual hosting

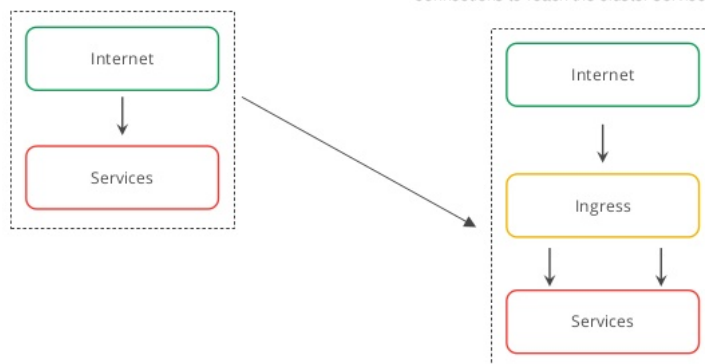
11.2 Ingress Controller

Ingress Controller

- Allow inbound traffic access to services
- Used instead of NodePort, or other services
- Proxy reconfigured according to rules

The Ingress

is collection of rules that allow inbound connections to reach the cluster services



Google Cloud Platform

Copyright 2016 Google Inc.

Figure 11.1: Ingress Controller for inbound connections

NGINX Ingress Controller

- Easy integration with Kubernetes APIs
- Uses **IngressClass** instead of annotations

```
spec:  
  ingressClassName: nginx
```

- Supports advanced **L7 routing** (host, path, TLS, rewrites)
- Client IP handling via `proxy-real-ip-cidr` and `use-forwarded-headers`
- Supports **session affinity** via cookies
- Integrates with **Service types** (NodePort, LoadBalancer)
- **TLS termination** supported using Secrets

Google Load Balancer Controller (GLBC)

- Controller called **glbc**, must be created and started first
- Also create
 - Replication Controller with single replica
 - Three services for application Pod
 - Ingress with two hostnames and three endpoints for each service.
- Backend is group of virtual machine instances, Instance Group
- Multi-pool path:
 - Global Forwarding Rule -> Target HTTP Proxy -> URL map
 - > Backend Service -> Instance Group
- Each pool checks next hop pool
- Not aware of GCE quota settings

Ingress API Resources

- Now part of networking.k8s.io API
- POST to API server
- Manage like other resources
- Able to expose low-number ports

```
$ kubectl get ingress  
$ kubectl delete ingress <ingress_name>  
$ kubectl edit ingress <ingress_name>
```

Deploy Ingress Controller

- Several options available
- Firewall rules may stop traffic
- Default backend serves 404 pages

```
$ kubectl create -f backend.yaml
```

11.3 Ingress Rules

Creating an Ingress Rule

- Run deployment and expose port
- POST rules entry to controller

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
rules:
  - host: ghost.192.168.99.100.nip.io
    http:
      paths:
        - backend:
            service:
              name: ghost
              port:
                number: 2368
            path: /
            pathType: ImplementationSpecific
```

Multiple Rules

- Multiple Services can use multiple rules

```
- host: ghost.192.168.99.100.nip.io
  http:
    paths:
      - backend:
          service
            name: external
            port:
              number: 80
    ....
- host: ghost.192.168.99.100.nip.io
  http:
    paths:
      - backend:
          service
            name: internal
            port:
              number: 8080
    ....
```

11.4 Service Mesh

Intelligent Connected Proxies

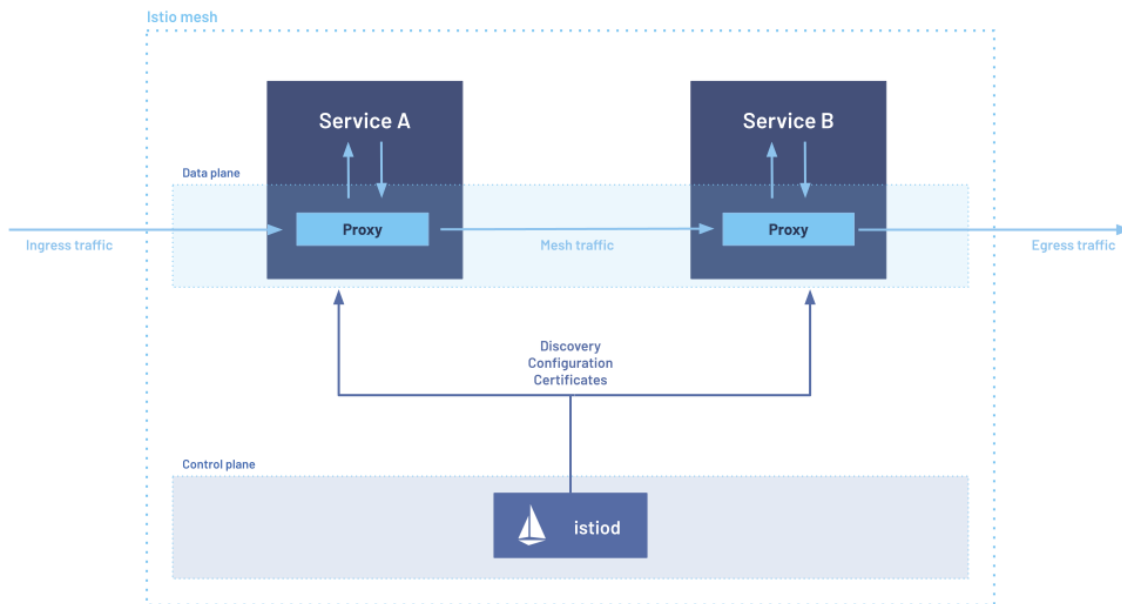


Figure 11.2: Istio Service Mesh

11.5 Limitations

Ingress Limitations

- Limited Feature Set - No Native Support to
 - TCP/UDP
 - gRPC
 - Traffic Splitting
- Heavy Reliance on Vendor Specific Annotations
 - Authentication
 - Rate limiting
 - Traffic management
- No True Role Separation
- Minimal Spec
- Frozen Development

11.6 Gateway API

Gateway API

- Modular Design
 - GatewayClass
 - Gateway
 - HTTPRoute
- Native Support to Advanced Routing
- Enhanced Security
- Extensible Framework
- Future-Proof and Portable

GatewayClass

- Acts as a template
- Implemented by different controllers
- Defines Controller Specification

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: example-class
spec:
  controllerName: example.com/gateway-controller
```

Gateway

- Network Entry Point
- Listener Configuration
- GatewayClass Association

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

HTTPRoute

- HTTP-Specific Routing
- Granular Matching
- Advanced Traffic Management

HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-httproute
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "www.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /login
    backendRefs:
    - name: example-svc
      port: 8080
```

11.7 Labs

Exercise 11.1: Service Mesh

If you have a large number of services to expose outside of the cluster, or to expose a low-number port on the host node you can deploy an ingress controller. While nginx and GCE have controllers mentioned a lot in Kubernetes.io, there are many to choose from. Even more functionality and metrics come from the use of a service mesh, such as Istio, Linkerd, Contour, Aspen, or several others.

1. We will install linkerd using their own scripts. There is quite a bit of output. Instead of showing all of it the output has been omitted. Look through the output and ensure that everything gets a green check mark. Some steps may take a few minutes to complete. Each command is listed here to make install easier. As well these steps are in the `setupLinkerd.txt` file.

```
student@cp:~$ curl -sL run.linkerd.io/install-edge | sh

student@cp:~$ export PATH=$PATH:/home/student/.linkerd2/bin

student@cp:~$ echo "export PATH=$PATH:/home/student/.linkerd2/bin" >> $HOME/.bashrc

student@cp:~$ linkerd check --pre

student@cp:~$ kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/\
releases/download/v1.2.1/standard-install.yaml

student@cp:~$ linkerd install --crds | kubectl apply -f -

student@cp:~$ linkerd install | kubectl apply -f -

student@cp:~$ linkerd check

student@cp:~$ linkerd viz install | kubectl apply -f -
```

```
student@cp:~$ linkerd viz check
```

```
student@cp:~$ linkerd viz dashboard &
```

2. By default the GUI is available on the localhost. We will need to edit the service and the deployment to allow outside access, in case you are using a cloud provider for the nodes. Edit to remove all characters after equal sign for `-enforced-host`, which is around line 59.

```
student@cp:~$ kubectl -n linkerd-viz edit deploy web
```

YAML

```
spec:
  containers:
  - args:
    - -linkerd-controller-api-addr=linkerd-controller-api.linkerd.svc.cluster.local:8085
    - -linkerd-metrics-api-addr=metrics-api.linkerd-viz.svc.cluster.local:8085
    - -cluster-domain=cluster.local
    - -grafana-addr=grafana.linkerd-viz.svc.cluster.local:3000
    - -controller-namespace=linkerd
    - -viz-namespace=linkerd-viz
    - -log-level=info
  #   - -enforced-host=                                #<-- Comment the line by adding #
  image: cr.l5d.io/linkerd/web:stable-2.11.1
  imagePullPolicy: IfNotPresent
```

3. Now edit the http nodePort and type to be a NodePort.

```
student@cp:~$ kubectl edit svc web -n linkerd-viz
```



```
1 ....
2 ports:
3   - name: http
4     nodePort: 31500                                #<-- Add line with an easy to remember port
5     port: 8084
6   ....
7   sessionAffinity: None
8   type: NodePort                                    #<-- Edit type to be NodePort
9 status:
10  loadBalancer: {}
11 ....
12
```

4. Test access using a local browser to your public IP. Your IP will be different than the one shown below.

```
student@cp:~$ curl ifconfig.io
```

```
104.197.159.20
```

5. From you local system open a browser and go to the public IP and the high-number nodePort. Be aware the look of the web page may look slightly different as the software is regularly updated, for example Grafana is not longer fully integrated.

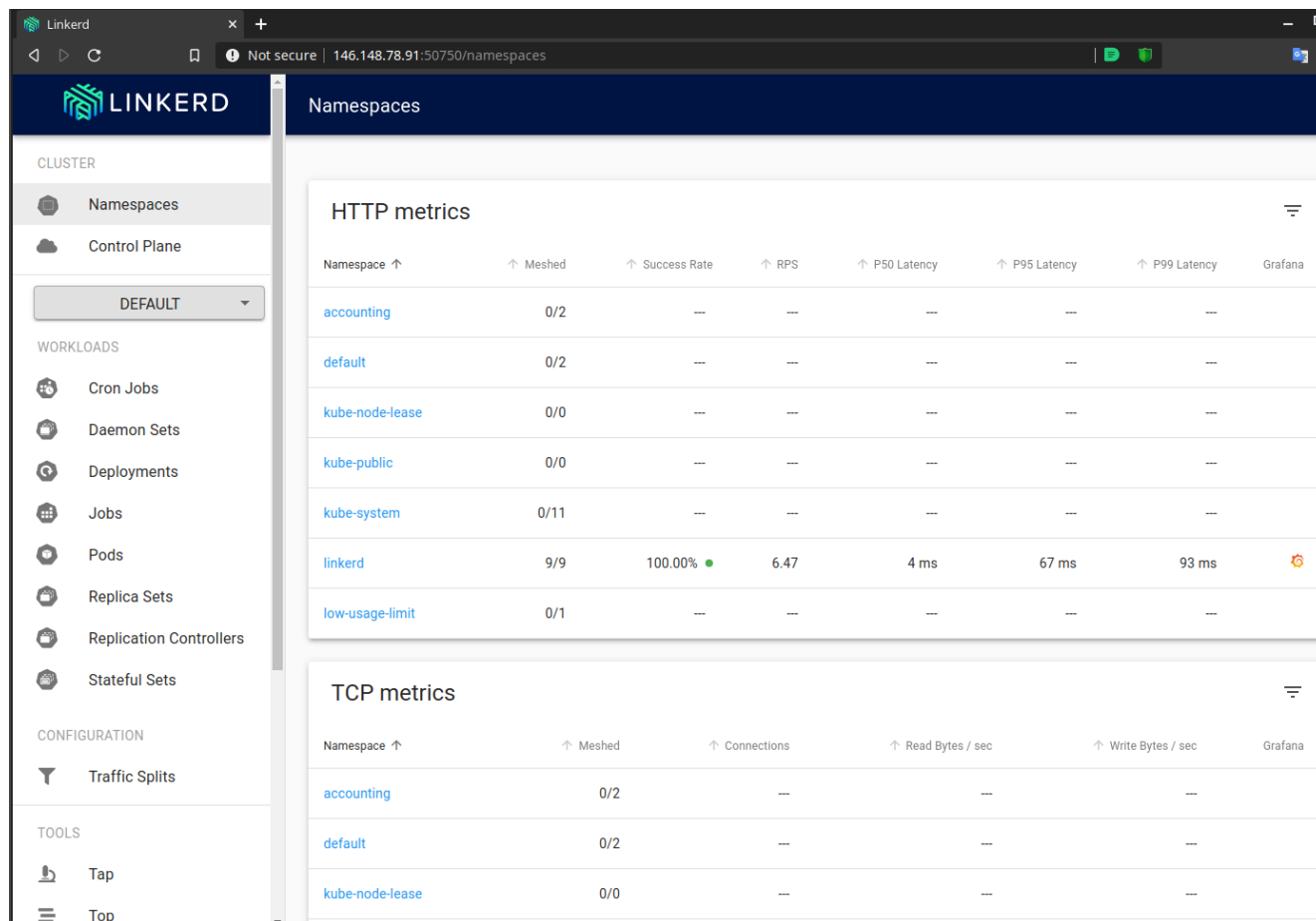


Figure 11.3: Main Linkerd Page

6. In order for linkerd to pay attention to an object we need to add an annotation. The **linkerd inject** command will do this for us. Generate YAML and pipe it to **linkerd** then pipe again to **kubectl**. Expect an error about how the object was created, but the process will work. The command can run on one line if you omit the back-slash. Recreate the `nginx-one` deployment we worked with in a previous lab exercise.

```
student@cp:~$ kubectl get ns accounting      ## Verify namespace exists

student@cp:~$ kubectl label node worker<TAB> system=secondOne      ## Re-label the node

student@cp:~$ vim nginx-one.yaml            ## Validate or correct containerPort: 80 (not 8080)

student@cp:~$ kubectl apply -f nginx-one.yaml      ## Re-deploy nginx-one application

student@cp:~$ kubectl -n accounting get deploy nginx-one -o yaml | \
    linkerd inject - | kubectl apply -f -
```

<output_omitted>

7. Check the GUI, you should see that the `accounting` namespaces and pods are now meshed, and the name is a link.
8. Generate some traffic to the pods, and watch the traffic via the GUI. Use the `service-lab` service.

```
student@cp:~$ kubectl -n accounting get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-one	ClusterIP	10.107.141.227	<none>	8080/TCP	5h15m
service-lab	NodePort	10.102.8.205	<none>	80:30759/TCP	5h14m

```
student@cp:~$ curl 10.102.8.205
```

```
<!DOCTYPE html>
<html>
```

```
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

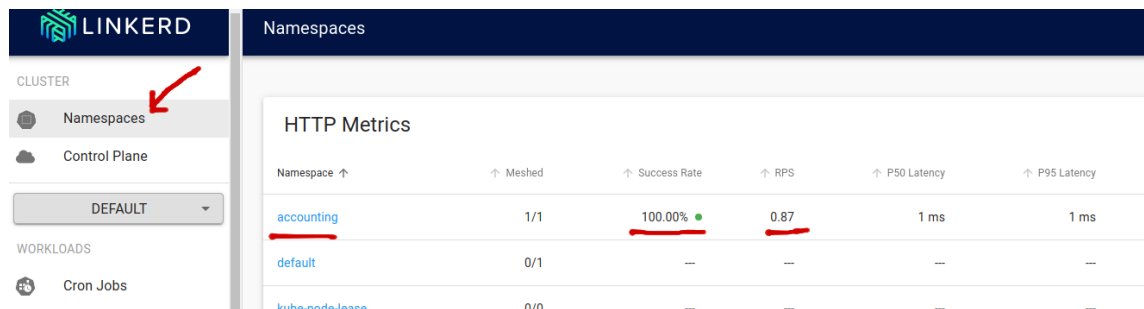


Figure 11.4: Now shows meshed

9. Scale up the `nginx-one` deployment. Generate traffic to get metrics for all the pods.

```
student@cp:~$ kubectl -n accounting scale deploy nginx-one --replicas=5
```

```
deployment.apps/nginx-one scaled
```

```
student@cp:~$ curl 10.102.8.205 #Several times
```

- Explore some of the other information provided by the GUI. Note that the initial view is of the default namespaces. Change to `accounting` to see details of the `nginx-one` deployment.

TCP Metrics

Namespace ↑	↑ Meshed	↑ Connections	↑ Read Bytes / sec	↑ Write Bytes / s
<code>accounting</code>	5/5	5	150B/s	1.401kB
<code>default</code>	0/1	---	---	---

Figure 11.5: Five meshed pods

Exercise 11.2: Ingress Controller

We will use the **Helm** tool we learned about earlier to install an ingress controller.

- Create two deployments in default namespace, `web-one` and `web-two`, each one running `nginx`. Expose both as ClusterIP services. Use previous content to determine the steps if you are unfamiliar. **Test that both ClusterIPs work before continuing to the next step.**
- Linkerd does not come with an ingress controller, so we will add one to help manage traffic. We will leverage a **Helm** chart to install an ingress controller. Search the hub to find that there are many available.

```
student@cp:~$ helm search hub ingress
```

```

URL                                CHART VERSION
APP VERSION      DESCRIPTION
https://artifacthub.io/packages/helm/k8s-as-hel...  1.0.2
v1.0.0           Helm Chart representing a single Ingress Kubern...
https://artifacthub.io/packages/helm/openstack-...  0.2.1
v0.32.0          OpenStack-Helm Ingress Controller
<output_omitted>
```

```
https://artifacthub.io/packages/helm/api/ingres...      3.29.1
0.45.0          Ingress controller for Kubernetes using NGINX a...
https://artifacthub.io/packages/helm/wener/ingr...      3.31.0
0.46.0          Ingress controller for Kubernetes using NGINX a...
https://artifacthub.io/packages/helm/nginx/ngin...      0.9.2
1.11.2          NGINX Ingress Controller
<output_omitted>
```

3. We will use a popular ingress controller provided by **NGINX**. Note: The Kubernetes community-maintained NGINX Ingress Controller (kubernetes/ingress-nginx) has been officially retired, with maintenance ending in March 2026. This change does not impact the NGINX web server itself. The controller will continue to be used for educational and demonstration purposes

```
student@cp:~$ helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
```

```
"ingress-nginx" has been added to your repositories
```

```
student@cp:~$ helm repo update
```

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "ingress-nginx" chart repository
Update Complete. -Happy Helming!-
```

4. Download and edit the `values.yaml` file and change it to use a DaemonSet instead of a Deployment. This way there will be a pod on every node to handle traffic.

```
student@cp:~$ helm fetch ingress-nginx/ingress-nginx --untar
```

```
student@cp:~$ cd ingress-nginx
```

```
student@cp:~/ingress-nginx$ ls
```

```
CHANGELOG.md Chart.yaml OWNERS README.md ci templates values.yaml
```

```
student@cp:~/ingress-nginx$ vim values.yaml
```

YAML

values.yaml

```
1 ....
2  ## DaemonSet or Deployment
3  ##
4  kind: DaemonSet                                #<-- Change to DaemonSet, around line 204
5
6  ## Annotations to be added to the controller Deployment or DaemonSet
7  ....
8
```

- Now install the controller using the chart. Note the use of the dot (.) to look in the current directory.

```
student@cp:~/ingress-nginx$ helm install myingress .
```

```
NAME: myingress
LAST DEPLOYED: Thu Mar 19 13:47:16 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The ingress-nginx controller has been installed.
It may take a few minutes for the LoadBalancer IP to be available.
You can watch the status by running
'kubectl --namespace default get services -o wide -w myingress-ingress-nginx-controller'
```

An example Ingress that makes use of the controller:
<output_omitted>

6. We now have an ingress controller running, but no rules yet. View the resources that exist. Use the **-w** option to watch the ingress controller service show up. After it is available use **ctrl-c** to quit and move to the next command.

```
student@cp:~$ kubectl get ingress --all-namespaces
```

No resources found

```
student@cp:~$ kubectl --namespace default get services -o wide myingress-ingress-nginx-controller
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S)	AGE	SELECTOR	
myingress-ingress-nginx-controller	LoadBalancer	10.104.227.79	<pending>
80:32558/TCP,443:30219/TCP	47s	app.kubernetes.io/component=controller, app.kubernetes.io/instance=myingress,app.kubernetes.io/name=ingress-nginx	

```
student@cp:~$ kubectl get pod --all-namespaces |grep nginx
```

default	myingress-ingress-nginx-controller-mrqt5	1/1	Running	0	20s
default	myingress-ingress-nginx-controller-pkdxm	1/1	Running	0	62s
default	nginx-b68dd9f75-h6ww7	1/1	Running	0	21h

7. Now we can add rules which match HTTP headers to services.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_11/ingress.yaml .
```

```
student@cp:~$ vim ingress.yaml
```

**YA
ML****ingress.yaml**

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: ingress-test
5    annotations:
6      nginx.ingress.kubernetes.io/service-upstream: "true"
7    namespace: default
8  spec:
9    ingressClassName: nginx
10   rules:
11   - host: www.example.com
12     http:
13       paths:
14       - backend:
15           service:
16             name: web-one
17             port:
18               number: 80
19         path: /
20         pathType: ImplementationSpecific
```

8. Create then verify the ingress is working. If you don't pass a matching header you should get a 404 error.

```
student@cp:~$ kubectl create -f ingress.yaml
```

```
ingress.networking.k8s.io/ingress-test created
```

```
student@cp:~$ kubectl get ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ingress-test	nginx	www.example.com		80	5s

```
student@cp:~$ kubectl get pod -o wide |grep myingress
```

myingress-ingress-nginx-controller-mrqt5	1/1	Running	0	8m9s	192.168.219.118
cp <none>	<none>				
myingress-ingress-nginx-controller-pkdxm	1/1	Running	0	8m9s	192.168.0.250
worker <none>	<none>				

```
student@cp:~/ingress-nginx$ curl 192.168.219.118
```

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

9. Check the ingress service and expect another 404 error, don't use the admission controller.

```
student@cp:~/ingress-nginx$ kubectl get svc |grep ingress
```

myingress-ingress-nginx-controller	LoadBalancer	10.104.227.79	<pending>
80:32558/TCP,443:30219/TCP		10m	
myingress-ingress-nginx-controller-admission	ClusterIP	10.97.132.127	<none>
443/TCP		10m	


```
student@cp:~/ingress-nginx$ curl 10.104.227.79
```

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

10. Now pass a header which matches a URL to one of the services we exposed in an earlier step. You should see the default nginx or httpd web server page.

```
student@cp:~/ingress-nginx$ curl -H "Host: www.example.com" http://10.104.227.79
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
<output_omitted>
```

11. We can add an annotation to the ingress pods for Linkerd. You will get some warnings, but the command will work.

```
student@cp:~/ingress-nginx$ kubectl get ds myingress-ingress-nginx-controller -o yaml | \
  linkerd inject --ingress - | kubectl apply -f -
```

```
daemonset "myingress-ingress-nginx-controller" injected
```

```
Warning: resource daemonsets/myingress-ingress-nginx-controller is missing the
kubectl.kubernetes.io/last-applied-configuration annotation which is required
by kubectl apply. kubectl apply should only be used on resources created
```

```
declaratively by either kubectl create --save-config or kubectl apply. The  
missing annotation will be patched automatically.  
daemonset.apps/myingress-ingress-nginx-controller configured
```

12. Go to the Top page, change the namespace to default and the resource to daemonset/myingress-ingress-nginx-controller. Press start then pass more traffic to the ingress controller and view traffic metrics via the GUI. Let top run so we can see another page added in an upcoming step.

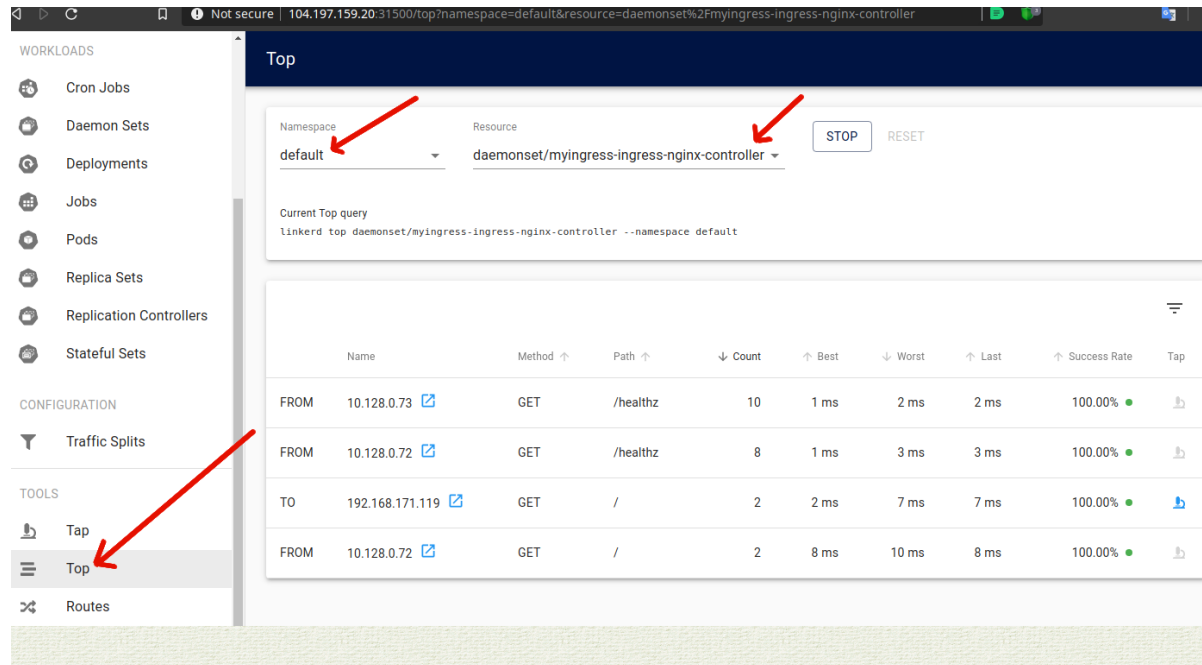


Figure 11.6: Ingress Traffic

13. At this point we would keep adding more and more servers. We'll configure one more, which would then could be a process continued as many times as desired.

Customize the web-two (or whichever deployment is running nginx) welcome page. Run a bash shell inside the web-two pod. Your pod name will end differently. Install **vim** or an editor inside the container then edit the `index.html` file of nginx so that the title of the web page will be Internal Welcome Page. Much of the command output is not shown below.

```
student@cp:~$ kubectl exec -it web-two-<Tab> -- /bin/bash
```



On Container

```
root@web-two-...-:/# apt-get update

root@web-two-...-:/# apt-get install vim -y

root@web-two-...-:/# vim /usr/share/nginx/html/index.html

<!DOCTYPE html>
<html>
<head>
<title>Internal Welcome Page</title>      #<-- Edit this line
<style>
<output_omitted>

root@thirdpage-:/# exit
```

Edit the ingress rules to point the thirdpage service. It may be easiest to copy the existing host stanza and edit the host and name.

14. `student@cp:~$ kubectl edit ingress ingress-test`



ingress-test

```
1  ....
2  spec:
3    rules:
4      - host: internal.org
5        http:
6          paths:
```



```

7     - backend:
8         service:
9             name: web-two
10            port:
11                number: 80
12            path: /
13            pathType: ImplementationSpecific
14 - host: www.example.com
15   http:
16     paths:
17       - backend:
18           service:
19               name: web-one
20              port:
21                  number: 80
22              path: /
23              pathType: ImplementationSpecific
24 status:
25 ....
26

```

15. Test the second Host: setting using **curl** locally as well as from a remote system, be sure the <title> shows the non-default page. Use the main IP of either node. The Linkerd GUI should show a new T0 line, if you select the small blue box with an arrow you will see the traffic is going to internal.org.

```
student@cp:~$ curl -H "Host: internal.org" http://10.128.0.7/
```

```

<!DOCTYPE html>
<html>
<head>
<title>Internal Welcome Page</title>
<style>

```

<output_omitted>

FROM	192.168.74.128		GET	/	3	4 ms	11 ms												
TO	192.168.74.152		GET	/	2	2 ms	2 ms												
TO	192.168.74.153	<table><thead><tr><th>Source</th><th></th><th>Destination</th></tr></thead><tbody><tr><td>ds/myingress-ingress-nginx-controller</td><td>→</td><td>deploy/web-two</td></tr><tr><td>po/myingress-ingress-nginx-controller-tgt7w</td><td>→</td><td>po/web-two-7bfc4687c5-rxd6g</td></tr><tr><td>192.168.171.115</td><td>→</td><td>192.168.74.153</td></tr></tbody></table>						Source		Destination	ds/myingress-ingress-nginx-controller	→	deploy/web-two	po/myingress-ingress-nginx-controller-tgt7w	→	po/web-two-7bfc4687c5-rxd6g	192.168.171.115	→	192.168.74.153
Source		Destination																	
ds/myingress-ingress-nginx-controller	→	deploy/web-two																	
po/myingress-ingress-nginx-controller-tgt7w	→	po/web-two-7bfc4687c5-rxd6g																	
192.168.171.115	→	192.168.74.153																	

Figure 11.7: Linkerd Top Metrics

Exercise 11.3: Gateway API

1. First step is to Deploy NGINX Gateway Fabric, To install the Gateway API resources, execute the following command

```
student@cp:~$ kubectl kustomize "https://github.com/nginx/nginx-gateway-fabric/\nconfig/crd/gateway-api/standard?ref=v1.6.1" | kubectl apply -f -
```

```
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io configured
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io configured
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
```

2. Deploy the NGINX Gateway Fabric CRDs by executing the following command

```
student@cp:~$ kubectl apply -f https://raw.githubusercontent.com/nginx/\
nginx-gateway-fabric/v1.6.1/deploy/crds.yaml
```

```
customresourcedefinition.apiextensions.k8s.io/clientsettingspolicies.gateway.nginx.org created
customresourcedefinition.apiextensions.k8s.io/nginxgateways.gateway.nginx.org created
customresourcedefinition.apiextensions.k8s.io/nginxproxies.gateway.nginx.org created
customresourcedefinition.apiextensions.k8s.io/observabilitypolicies.gateway.nginx.org created
customresourcedefinition.apiextensions.k8s.io/snippetsfilters.gateway.nginx.org created
customresourcedefinition.apiextensions.k8s.io/upstreamsettingspolicies.gateway.nginx.org created
```

3. Deploy the NGINX Gateway Fabric by executing the following command

```
student@cp:~$ kubectl apply -f https://raw.githubusercontent.com/nginx/\
nginx-gateway-fabric/v1.6.1/deploy/default/deploy.yaml
```

```
namespace/nginx-gateway created
serviceaccount/nginx-gateway created
clusterrole.rbac.authorization.k8s.io/nginx-gateway created
clusterrolebinding.rbac.authorization.k8s.io/nginx-gateway created
configmap/nginx-includes-bootstrap created
service/nginx-gateway created
deployment.apps/nginx-gateway created
gatewayclass.gateway.networking.k8s.io/nginx created
```

```
nginxgateway.gateway.nginx.org/nginx-gateway-config created
```

4. Verify the NGINX Gateway Fabric is running by executing the following command

```
student@cp:~$ kubectl get all -n nginx-gateway
```

```
NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-gateway-7c59cd4cc6-5m84b  2/2     Running   0           92s
```



```
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
↪ AGE
service/nginx-gateway               LoadBalancer        10.111.70.106    <pending>        80:32500/TCP,443:32730/TCP
↪ 92s
```



```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-gateway        1/1     1             1           92s
```



```
NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-gateway-7c59cd4cc6  1         1         1       92s
```

5. The NGINX Gateway Fabric service is of the type of LoadBalancer service. The external IP is pending, Kubernetes will randomly allocate two ports on every node of the cluster. To access the NGINX Gateway Fabric, use an IP address of any node of the cluster along with the two allocated ports. For simplicity sake, we can patch the service to be of NodePort.

```
student@cp:~$ kubectl patch service/nginx-gateway -n nginx-gateway -p '{"spec": {"type": "NodePort"}}'
student@cp:~$ kubectl get service/nginx-gateway -n nginx-gateway
```


service/nginx-gateway patched

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-gateway	NodePort	10.111.70.106	<none>	80:32500/TCP,443:32730/TCP	11m

6. Deploy an application and create a service named books. Then, define two Gateway API resources—a gateway and an HTTPRoute. These components work together to establish a routing rule that captures all HTTP requests for the hostname shop.example.com and directs them to the books service.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_11/books.yaml .
```

```
student@cp:~$ vim books.yaml
```



books.yaml

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: books
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: books
10   template:
11     metadata:
12       labels:
13         app: books
14     spec:
15       containers:
```



```
16     - name: books
17       image: nginx:1.29.6
18       ports:
19         - containerPort: 80
20     ---
21     apiVersion: v1
22     kind: Service
23     metadata:
24       name: books
25     spec:
26       ports:
27         - port: 80
28           targetPort: 80
29           protocol: TCP
30       name: http
31     selector:
32       app: books
```

```
student@cp:~$ kubectl create -f books.yaml
```

```
deployment.apps/books created
service/books created
```

7. Verify the books deployment and service are created and running fine

```
student@cp:~$ kubectl get svc/books deploy/books
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/books	ClusterIP	10.111.164.248	<none>	80/TCP	2m57s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/books	2/2	2	2	2m57s

8. To route traffic to the books application, we will create a gateway and HTTPRoute. We need a gateway to create an entry point for HTTP traffic coming into the cluster. The shop gateway we are going to create will open an entry point to the cluster on port 80 for HTTP traffic.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_11/gateway.yaml .
```

```
student@cp:~$ vim gateway.yaml
```



gateway.yaml

```
1 apiVersion: gateway.networking.k8s.io/v1
2 kind: Gateway
3 metadata:
4   name: shop
5 spec:
6   gatewayClassName: nginx
7   listeners:
8     - name: http
9       port: 80
10    protocol: HTTP
```

```
student@cp:~$ kubectl create -f gateway.yaml
```

```
gateway.gateway.networking.k8s.io/shop created
```

9. Verify the shop Gateway API resource has been deployed.

```
student@cp:~$ kubectl get gateway
```

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
shop	nginx		True	2m9s

10. To route HTTP traffic from the gateway to the books service, we need to create an HTTPRoute named books and attach it to the gateway. This HTTPRoute will have a single routing rule that routes all traffic to the hostname “shop.example.com” from the gateway to the books service. Once NGINX Gateway Fabric processes the shop gateway and books HTTPRoute, it will configure its data plane (NGINX) to route all HTTP requests sent to “shop.example.com” to the pods that the books service targets.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_11/httproute.yaml .
```

```
student@cp:~$ vim httproute.yaml
```

YAML

httproute.yaml

```
1 apiVersion: gateway.networking.k8s.io/v1
2 kind: HTTPRoute
3 metadata:
4   name: books
5 spec:
6   parentRefs:
7     - name: shop
```



```
8   hostnames:
9   - "shop.example.com"
10  rules:
11  - matches:
12    - path:
13      type: PathPrefix
14      value: /
15    backendRefs:
16    - name: books
17      port: 80
```

```
student@cp:~$ kubectl create -f httproute.yaml
```

```
httproute.gateway.networking.k8s.io/books created
```

11. Verify the httproute has been deployed.

```
student@cp:~$ kubectl get httproute
```

NAME	HOSTNAMES	AGE
books	["shop.example.com"]	35s

12. The Gateway API and HTTPRoute have been deployed successfully, test the configuration by sending a request to the **Node IP - i.e. private ip, which can be obtained by command `hostname -i`** and node port of the NGINX gateway Fabric. First, Lets send a request to the path `/`. Since the shop HTTPRoute routes all traffic on any path to the books application, the following requests should also be handled by the books pods.

```
student@cp:~$ curl -H "Host: shop.example.com" 10.2.0.30:32500
```

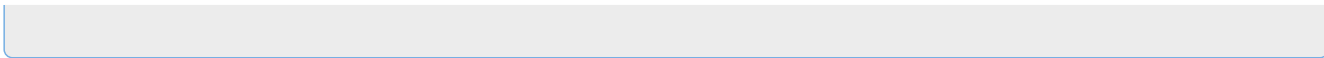
```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>

...
<output_omitted>
```

13. Requests to hostnames other than “shop.example.com” should not be routed to the books application, since the books HTTPRoute only matches requests with the “shop.example.com” hostname. To verify this, send a request to the host-name “test.example.com”:

```
student@cp:~$ curl -H "Host: test.example.com" 10.2.0.30:32500
```

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```



Chapter 12

Scheduling



12.1	Overview	382
12.2	Scheduler Settings	383
12.3	Pod Specification	388
12.4	Affinity Rules	390
12.5	Taints and Tolerations	395
12.6	Labs	398

12.1 Overview

kube-scheduler

- Topology-aware algorithm to determine Pod placement
- Pod priority and preemption
- Labels used for each method
- **podAffinity** label encourage Pod on specific nodes
- Avoid via **podAntiAffinity**
- Node **taints** to repel specific Pods
- Pod **tolerations** of node **taints**
- **nodeSelector** to force specific Pods
- Require, prefer and evict

12.2 Scheduler Settings

Node selection in kube-scheduler

- **Filtering**
- **Scoring**

Scheduling Configuration

- **Customize the behavior**
- **Scheduling Profiles**
 - Extension points
 - Scheduling plugins
 - Multiple profiles

Extension Points

- queueSort
- preFilter
- filter
- postFilter
- preScore
- score
- reserve
- permit
- preBind
- bind
- postBind
- multiPoint

Scheduling plugins

- ImageLocality
- TaintToleration
- NodeName
- NodePorts
- NodeAffinity
- PodTopologySpread
- NodeUnschedulable
- NodeResourcesFit
- NodeResourcesBalancedAllocation
- VolumeBinding
- VolumeRestrictions
- VolumeZone
- NodeVolumeLimits
- EBSLimits
- GCEPDLimits
- AzureDiskLimits
- InterPodAffinity
- PrioritySort
- DefaultBinder
- DefaultPreemption

Multiple profiles

- Configure more than one profile

- Example config

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
  - schedulerName: default-scheduler
  - schedulerName: custom-scheduler
  plugins:
    preFilter:
      disabled:
        - name: '*'
    filter:
      disabled:
        - name: '*'
    postFilter:
      disabled:
        - name: '*'
```

12.3 Pod Specification

Pod Specification

- Pod Specification fields
 - nodeName
 - nodeSelector
 - affinity
 - tolerations
 - schedulerName

Specifying Node Label

- Match a label with `nodeSelector`
- Pod remains in Pending state until a suitable Node is found

```
spec:  
  containers:  
  - name: redis  
    image: redis  
  nodeSelector:  
    net: fast
```

12.4 Affinity Rules

Pod Affinity Rules

- Uses In, NotIn, Exists, and DoesNotExist operators
- `requiredDuringSchedulingIgnoredDuringExecution`
- `preferredDuringSchedulingIgnoredDuringExecution`
- `affinity`
 - `podAffinity`
 - `podAntiAffinity`

podAffinity Example

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: security
                operator: In
                values:
                  - S1
```

podAntiAffinity Example

```
podAntiAffinity:
  preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 100
    podAffinityTerm:
      labelSelector:
        matchExpressions:
        - key: security
          operator: In
          values:
          - S2
```

Node Affinity Rules

- Similar to Pod affinity rules, declared with affinity
- Uses In, NotIn, Exists, and DoesNotExist operators
- `requiredDuringSchedulingIgnoredDuringExecution`
- `preferredDuringSchedulingIgnoredDuringExecution`
- Planned for future
 - `requiredDuringSchedulingRequiredDuringExecution`

Node Affinity Example

```
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
      preference:
        matchExpressions:
          - key: diskspeed
            operator: In
            values:
              - quick
              - fast
```

12.5 Taints and Tolerations

Taint

- Expressed as `key=value:effect`
- `key` and `value` created by admin
- Effect must be **NoSchedule**, **PreferNoSchedule**, or **NoExecute**
- Only nodes can be tainted currently
- Multiple taints possible on a node

Tolerations

- Pod setting to run on tainted nodes
- Same **effects** as node taints
- Two operators
 - Exists
 - Equal which requires value

tolerations:

```
- key: "server"
  operator: "Equal"
  value: "ap-east"
  effect: "NoExecute"
  tolerationSeconds: 3600
```


Custom Scheduler

- Create and deploy custom scheduler as a container
- Run multiple schedulers simultaneously
- Pods can declare which scheduler to use
- Scheduler must be running or Pod remains Pending
- View scheduler and other information with **kubectl get events**

12.6 Labs

Exercise 12.1: Assign Pods Using Labels

Overview

While allowing the system to distribute Pods on your behalf is typically the best route, you may want to determine which nodes a Pod will use. For example you may have particular hardware requirements to meet for the workload. You may want to assign VIP Pods to new, faster hardware and everyone else to older hardware.

In this exercise we will use `labels` to schedule Pods to a particular node. Then we will explore `taints` to have more flexible deployment in a large environment.

1. Begin by getting a list of the nodes. They should be in the ready state and without added labels or taints.

```
student@cp:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
cp	Ready	control-plane	44h	v1.35.2
worker	Ready	<none>	43h	v1.35.2

2. View the current labels and taints for the nodes.

```
student@cp:~$ kubectl describe nodes |grep -A5 -i label
```

```
Labels:          beta.kubernetes.io/arch=amd64
                 beta.kubernetes.io/os=linux
                 kubernetes.io/arch=amd64
                 kubernetes.io/hostname=scp
                 kubernetes.io/os=linux
                 node-role.kubernetes.io/control-plane=
--
```

```
Labels:          beta.kubernetes.io/arch=amd64
                beta.kubernetes.io/os=linux
                kubernetes.io/arch=amd64
                kubernetes.io/hostname=worker
                kubernetes.io/os=linux
                system=secondOne
```

```
student@cp:~$ kubectl describe nodes |grep -i taint
```

```
Taints:          <none>
Taints:          <none>
```

3. Get a count of how many containers are running on both the cp and worker nodes. There are about 24 containers running on the cp in the following example, and eight running on the worker. There are status lines which increase the **wc** count. You may have more or less, depending on previous labs and cleaning up of resources. Take note of the number of containers, and then notice the numbers change due to scheduling. The change between nodes is the important information, not the particular number. If you are using **cri-o** you can view containers using **crictl ps**.

```
student@cp:~$ kubectl get deployments --all-namespaces
```

NAMESPACE	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
accounting	nginx-one	1/1	1	1	19h
default	anotherweb-apache	1/1	1	1	8h
default	web-one	1/1	1	1	45m
default	web-two	1/1	1	1	45m
kube-system	cilium-operator	1/1	1	1	35h
<output_omitted>					

```
student@cp:~$ sudo crictl ps | wc -l
```

24

```
student@worker:~$ sudo crictl ps | wc -l
```

21

4. For the purpose of the exercise we will assign the cp node to be VIP hardware and the secondary node to be for others.

```
student@cp:~$ kubectl label nodes cp status=vip
```

```
node/cp labeled
```

```
student@cp:~$ kubectl label nodes worker status=other
```

```
node/worker labeled
```

5. Verify your settings. You will also find there are some built in labels such as hostname, os and architecture type. The output below appears on multiple lines for readability.

```
student@cp:~$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
cp	Ready	control-plane	35h	v1.35.2	beta.kubernetes.io/arch=amd64, beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=cp, kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node-role.kubernetes.io/master=, node.kubernetes.io/exclude-from-external-load-balancers=,status=vip
worker	Ready	<none>	35h	v1.35.2	beta.kubernetes.io/arch=amd64, beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker, kubernetes.io/os=linux,status=other,system=secondOne

6. Create `vip.yaml` to spawn four busybox containers which sleep the whole time. Include the `nodeSelector` entry.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_12/vip.yaml .
```

```
student@cp:~$ vim vip.yaml
```

**Y
A
ML****vip.yaml**

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: vip
5  spec:
6    containers:
7      - name: vip1
8        image: busybox
9        args:
10         - sleep
11         - "1000000"
12      - name: vip2
13        image: busybox
14        args:
15         - sleep
16         - "1000000"
17      - name: vip3
18        image: busybox
19        args:
20         - sleep
21         - "1000000"
```



```
22 - name: vip4
23   image: busybox
24   args:
25     - sleep
26     - "1000000"
27   nodeSelector:
28     status: vip
```

7. Deploy the new pod. Verify the containers have been created on the cp node. It may take a few seconds for all the containers to spawn. Check both the cp and the secondary nodes. From this point forward use **crictl** where the step lists **docker** if you have deployed your cluster with cri-o.

```
student@cp:~$ kubectl create -f vip.yaml
```

```
pod/vip created
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
28
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
21
```

8. Delete the pod then edit the file, commenting out the `nodeSelector` lines. It may take a while for the containers to fully terminate.

```
student@cp:~$ kubectl delete pod vip
```

```
pod "vip" deleted
```

```
student@cp:~$ vim vip.yaml
```

```
....  
#   nodeSelector:  
#       status: vip
```

9. Create the pod again. Containers can now be spawning on either of the node. You may see pods for the daemonsets as well.

```
student@cp:~$ kubectl get pods
```

```
<output_omitted>
```

```
student@cp:~$ kubectl create -f vip.yaml
```

```
pod/vip created
```

10. Determine where the new pods have been deployed. The pods should be more evenly spread this time. Again, the numbers may be different, the change in numbers is what we are looking for. Due to lack of `nodeSelector` they could go to either node.

```
student@cp:~$ sudo crictl ps |wc -l
```

```
24
```

```
student@worker:~$ sudo crictl ps |wc -l
```

25

11. Create another file for other users. Change the names from vip to others, and uncomment the nodeSelector lines.

```
student@cp:~$ cp vip.yaml other.yaml
student@cp:~$ sed -i s/vip/other/g other.yaml
student@cp:~$ vim other.yaml
```

YAML**other.yaml**

```
1 ....
2   nodeSelector:
3     status: other
4
```

12. Create the other containers. Determine where they deploy.

```
student@cp:~$ kubectl create -f other.yaml
```

```
pod/other created
```

```
student@cp:~$ sudo crictl ps |wc -l
```

24

```
student@worker:~$ sudo crictl ps |wc -l
```



```
25
```

13. Shut down both pods and verify they terminated. Only our previous pods should be found.

```
student@cp:~$ kubectl delete pods vip other
```

```
pod "vip" deleted
pod "other" deleted
```

```
student@cp:~$ kubectl get pods
```

```
<output_omitted>
```

Exercise 12.2: Using Taints to Control Pod Deployment

Use taints to manage where Pods are deployed or allowed to run. In addition to assigning a Pod to a group of nodes, you may also want to limit usage on a node or fully evacuate Pods. Using taints is one way to achieve this. You may remember that the cp node begins with a NoSchedule taint. We will work with three taints to limit or remove running pods.

1. Create a deployment which will deploy eight **nginx** containers. Begin by creating a YAML file.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_12/taint.yaml .
```

```
student@cp:~$ vim taint.yaml
```



taint.yaml

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: taint-deployment
5  spec:
6    replicas: 8
7    selector:
8      matchLabels:
9        app: nginx
10   template:
11     metadata:
12       labels:
13         app: nginx
14     spec:
15       containers:
16       - name: nginx
17         image: nginx:1.20.1
18         ports:
19         - containerPort: 80
```

2. Apply the file to create the deployment.

```
student@cp:~$ kubectl apply -f taint.yaml
```

```
deployment.apps/taint-deployment created
```

3. Determine where the containers are running. In the following example three have been deployed on the cp node and five on the secondary node. Remember there will be other housekeeping containers created as well. Your numbers may be different, the actual number is not important, we are tracking the change in numbers.

```
student@cp:~$ sudo crictl ps |grep nginx
```

```
00c1be5df1e7      nginx@sha256:e3456c851a152494c3e.....  
<output_omitted>
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
27
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
17
```

4. Delete the deployment. Verify the containers are gone.

```
student@cp:~$ kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
21
```

5. Now we will use a taint to affect the deployment of new containers. There are three taints, `NoSchedule`, `PreferNoSchedule` and `NoExecute`. The taints having to do with schedules will be used to determine newly deployed containers, but will not affect running containers. The use of `NoExecute` will cause running containers to move.

Taint the secondary node, verify it has the taint then create the deployment again. We will use the key of bubba to illustrate the key name is just some string an admin can use to track Pods.

```
student@cp:~$ kubectl taint nodes worker \
    bubba=value:PreferNoSchedule
```

```
node/worker tainted
```

```
student@cp:~$ kubectl describe node |grep Taint
```

```
Taints:                bubba=value:PreferNoSchedule
Taints:                <none>
```

```
student@cp:~$ kubectl apply -f taint.yaml
```

```
deployment.apps/taint-deployment created
```

6. Locate where the containers are running. We can see that more containers are on the cp, but there still were some created on the secondary. Delete the deployment when you have gathered the numbers.

```
student@cp:~$ sudo crictl ps |wc -l
```

```
21
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
23
```

```
student@cp:~$ kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

7. Remove the taint, verify it has been removed. Note that the key is used with a minus sign appended to the end.

```
student@cp:~$ kubectl taint nodes worker bubba-
```

```
node/worker untainted
```

```
student@cp:~$ kubectl describe node |grep Taint
```

```
Taints:                <none>
Taints:                <none>
```

8. This time use the NoSchedule taint, then create the deployment again. The secondary node should not have any new containers, with only daemonsets and other essential pods running.

```
student@cp:~$ kubectl taint nodes worker \
    bubba=value:NoSchedule
```

```
node/worker tainted
```

```
student@cp:~$ kubectl apply -f taint.yaml
```

```
deployment.apps/taint-deployment created
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
21
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
23
```

9. Remove the taint and delete the deployment. When you have determined that all the containers are terminated create the deployment again. Without any taint the containers should be spread across both nodes.

```
student@cp:~$ kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

```
student@cp:~$ kubectl taint nodes worker bubba-
```

```
node/worker untainted
```

```
student@cp:~$ kubectl apply -f taint.yaml
```

```
deployment.apps/taint-deployment created
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
27
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
17
```

10. Now use the `NoExecute` to taint the secondary (**worker**) node. Wait a minute then determine if the containers have moved. The DNS containers can take a while to shutdown. Some containers will remain on the worker node to continue communication from the cluster.

```
student@cp:~$ kubectl taint nodes worker \
    bubba=value:NoExecute
```

```
node "worker" tainted
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
37
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
5
```

11. Remove the taint. Wait a minute. Note that all of the containers did not return to their previous placement.

```
student@cp:~$ kubectl taint nodes worker bubba-
```

```
node/worker untainted
```

```
student@cp:~$ sudo crictl ps |wc -l
```

```
32
```

```
student@worker:~$ sudo crictl ps |wc -l
```

```
6
```

12. Remove the deployment a final time to free up resources.

```
student@cp:~$ kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

13. **CHALLENGE STEP** Use your knowledge of deployments and scaling items to deploy multiple `httpd` pods across the nodes and examine the typical spread and spread after using taints.

Chapter 13

Logging and Troubleshooting



13.1	Overview	414
13.2	Troubleshooting Flow	415
13.3	Basic Start Sequence	417
13.4	Monitoring	418
13.5	Plugins	419
13.6	Logging	422
13.7	Troubleshooting Resources	423
13.8	Labs	424

13.1 Overview

Overview

- **Linux** troubleshooting via shell
- Turn on basic monitoring
- Set up cluster-wide logging
- External products **Fluentd**, **Prometheus** helpful.
- Internal **Metrics Server** and API

13.2 Troubleshooting Flow

Basic Steps

- Errors from command line
- Pod logs and state of the Pod
- Use shell to troubleshoot Pod DNS and network
- Check node logs for errors. Enough resources
- RBAC, SELinux or AppArmor security settings
- API calls to and from controllers to **kube-apiserver**
- Enable auditing
- Inter-node network issues, DNS and firewall
- Control Plane server controllers.
 - Control Pods in pending or error state
 - Errors in log files
 - Enough resources

Ephemeral Containers

- Used for **debugging running Pods**
 - **Stable feature** (GA since Kubernetes v1.25)
 - Added without restarting the Pod
- ```
kubectl debug pod/buggypod -it --image=debian
```
- Shares **namespaces** (network, PID) with target container

## 13.3 Basic Start Sequence

# Cluster Start Sequence

- **systemd** starts `kubelet.service`
  - Uses `/etc/systemd/system/kubelet.service.d/10-kubeadm.conf`
  - Uses `/var/lib/kubelet/config.yaml` config file
  - **staticPodPath** set to `/etc/kubernetes/manifests/`
- **kubelet** creates all pods from `*.yaml` in directory
  - `kube-apiserver`
  - `etcd`
  - `kube-controller-manager`
  - `kube-scheduler`
- `kube-controller-manager` control loops use `etcd` data to start rest

## 13.4 Monitoring

# Monitoring

- Enable the add-ons
- **Metrics Server** and API
- **Prometheus**
- **Jaeger**
- **OpenTelemetry**
- API Server Tracing

## 13.5 Plugins

# Using Krew

- Install the software using steps at <https://krew.dev>

```
$ kubectl krew help
krew is the kubectl plugin manager.
You can invoke krew through kubectl: "kubectl krew [command]..."
```

Usage:

```
krew [command]
```

Available Commands:

|           |                                             |
|-----------|---------------------------------------------|
| help      | Help about any command                      |
| info      | Show information about a kubectl plugin     |
| install   | Install kubectl plugins                     |
| list      | List installed kubectl plugins              |
| search    | Discover kubectl plugins                    |
| uninstall | Uninstall plugins                           |
| update    | Update the local copy of the plugin index   |
| upgrade   | Upgrade installed plugins to newer versions |
| version   | Show krew version and diagnostics           |

# Managing Plugins

- Add paths to plugins to `$PATH` variable
- View current plugins with **`kubectl plugin list`**
- Find new plugins with **`kubectl krew search`**
- Installing using **`kubectl krew install new-plugin`**
- Once installed use as **`kubectl`** sub-command
- **`upgrade`** and **`uninstall`** also available



# Sniffing Traffic With Wireshark

- Read installation output for basic usage
- Note some require extra packages and configuration.
- **sniff** requires **Wireshark** and ability to export graphical display
- Pass the command the pod and container to use

```
$ kubectl krew install sniff nginx-123456-abcd -c webcont
```

## 13.6 Logging

### Logging Tools

- No cluster-wide logging in Kubernetes
- Often aggregated and digested by outside tools like **ElasticSearch**
- **Fluentd**
- **Kibana**

## 13.7 Troubleshooting Resources

### More Resources

- Official documentation
- Major vendor pages
- Github pages
- Kubernetes **Slack** channel

## 13.8 Labs

### Exercise 13.1: Review Log File Locations

#### Overview

In addition to various logs files and command output, you can use **journalctl** to view logs from the node perspective. We will view common locations of log files, then a command to view container logs. There are other logging options, such as the use of a **sidecar** container dedicated to loading the logs of another container in a pod.

Whole cluster logging is not yet available with Kubernetes. Outside software is typically used, such as **Fluentd**, part of <http://fluentd.org/>, which is another member project of **CNCF.io**, like Kubernetes.

Take a quick look at the following log files and web sites. As server processes move from node level to running in containers the logging also moves.

1. If using a **systemd**-based Kubernetes cluster, view the node level logs for **kubelet**, the local Kubernetes agent. Each node will have different contents as this is node specific.

```
student@cp:~$ journalctl -u kubelet |less
```

```
<output_omitted>
```

2. Major Kubernetes processes now run in containers. You can view them from the container or the pod perspective. Use the **find** command to locate the **kube-apiserver** log. Your output will be different, but will be very long.

```
student@cp:~$ sudo find / -name "*apiserver*log"
```

```
/var/log/containers/kube-apiserver-cp_kube-system_kube-apiserver-423
d25701998f68b503e64d41dd786e657fc09504f13278044934d79a4019e3c.log
```

3. Take a look at the log file.

```
student@cp:~$ sudo less /var/log/containers/kube-apiserver-cp_kube-system_kube-apiserver-423d25701998f68b503e64d41dd786e657fc09504f13278044934d79a4019e3c.log
```

<output\_omitted>

4. Search for and review other log files for `coredns`, `kube-proxy`, and other cluster agents.
5. If **not** on a Kubernetes cluster using **systemd** which collects logs via **journalctl** you can view the text files on the cp node.
  - (a) `/var/log/kube-apiserver.log`  
Responsible for serving the API
  - (b) `/var/log/kube-scheduler.log`  
Responsible for making scheduling decisions
  - (c) `/var/log/kube-controller-manager.log`  
Controller that manages replication controllers
6. `/var/log/containers`  
Various container logs
7. `/var/log/pods/`  
More log files for current Pods.
8. Worker Nodes Files (on non-**systemd** systems)
  - (a) `/var/log/kubelet.log`  
Responsible for running containers on the node
  - (b) `/var/log/kube-proxy.log`  
Responsible for service load balancing

9. More reading: <https://kubernetes.io/docs/tasks/debug-application-cluster/debug-service/> and <https://kubernetes.io/docs/tasks/debug-application-cluster/determine-reason-pod-failure/>

## Exercise 13.2: Viewing Logs Output

Container standard out can be seen via the **kubectl logs** command. If there is no standard out, you would not see any output. In addition, the logs would be destroyed if the container is destroyed.

1. View the current Pods in the cluster. Be sure to view Pods in all namespaces.

```
student@cp:~$ kubectl get po --all-namespaces
```

| NAMESPACE   | NAME                             | READY | STATUS  | RESTARTS | AGE  |
|-------------|----------------------------------|-------|---------|----------|------|
| kube-system | cilium-operator-788c7d7585-jgn6s | 1/1   | Running | 0        | 13m  |
| kube-system | cilium-5tv9d                     | 1/1   | Running | 0        | 6d1h |
| ....        |                                  |       |         |          |      |
| kube-system | etcd-cp                          | 1/1   | Running | 2        | 44h  |
| kube-system | kube-apiserver-cp                | 1/1   | Running | 2        | 44h  |
| kube-system | kube-controller-manager-cp       | 1/1   | Running | 2        | 44h  |
| kube-system | kube-scheduler-cp                | 1/1   | Running | 2        | 44h  |
| ....        |                                  |       |         |          |      |

2. View the logs associated with various infrastructure pods. Using the **Tab** key you can get a list and choose a container. Then you can start typing the name of a pod and use **Tab** to complete the name.

```
student@cp:~$ kubectl -n kube-system logs <Tab><Tab>
```

```
cilium-operator-788c7d7585-jgn6s
cilium-5tv9d
coredns-5644d7b6d9-k7kts
```

```
coredns-5644d7b6d9-rnr2v
etcd-cp
kube-apiserver-cp
kube-controller-manager-cp
kube-proxy-qhc4f
kube-proxy-s56hl
kube-scheduler-f-cp
traefik-ingress-controller-hw5tv
traefik-ingress-controller-mcn47
```

```
student@cp:~$ kubectl -n kube-system logs \
 kube-apiserver-cp
```

```
Flag --insecure-port has been deprecated, This flag will be removed in a future version.
I1119 02:31:14.933023 1 server.go:623] external host was not specified, using 10.128.0.3
I1119 02:31:14.933356 1 server.go:149] Version: v1.29.1
I1119 02:31:15.595131 1 plugins.go:158] Loaded 11 mutating admission controller(s)
successfully in the following order: NamespaceLifecycle,LimitRanger,ServiceAccount,
NodeRestriction,TaintNodesByCondition,Priority,DefaultTolerationSeconds,DefaultStorageClass,
StorageObjectInUseProtection,MutatingAdmissionWebhook,RuntimeClass.
I1119 02:31:15.595357 1 plugins.go:161] Loaded 7 validating admission controller(s)
successfully in the following order: LimitRanger,ServiceAccount,Priority,
PersistentVolumeClaimResize,ValidatingAdmissionWebhook,RuntimeClass,
ResourceQuota.
<output_omitted>
```

3. View the logs of other Pods in your cluster.

## Exercise 13.3: Installing Headlamp A Modern Kubernetes UI

The **Kubernetes Dashboard**, which was historically deployed as an add-on, is now **deprecated and no longer maintained**. The project has been officially **archived** and is not recommended for new deployments. For modern Kubernetes environments, it is advisable to use alternatives such as **Headlamp**, which provides an actively maintained and more feature-rich user interface.

We will deploy this in the following section.

1. Copy the dashboard yaml from the tarball and deploy the dashboard. The manifest was provided by Headlamp installation.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_13/kubernetes-headlamp.yaml .
```

```
student@cp:~$ kubectl create -f kubernetes-headlamp.yaml
```

```
service/headlamp created
deployment.apps/headlamp created
secret/headlamp-admin created
```

2. We will give the dashboard full admin rights, which may be more than one would in production. The dashboard is running in the default namespace. Create a service account and assign the cluster-admin rights.

There is more on service account in the Security chapter.

```
student@cp:~$ kubectl create sa headlamp-admin
```

```
serviceaccount/headlamp-admin created
```

3. Give admin rights to the account.

```
student@cp:~$ kubectl create clusterrolebinding headlamp-admin \
--serviceaccount=default:headlamp-admin --clusterrole=cluster-admin
```



```
clusterrolebinding.rbac.authorization.k8s.io/headlamp-admin created
```

4. View the service and record the NodePort, which will be used to access the dashboard.

```
kubectl get svc headlamp
```

| NAME     | TYPE     | CLUSTER-IP    | EXTERNAL-IP | PORT(S)      | AGE   |
|----------|----------|---------------|-------------|--------------|-------|
| headlamp | NodePort | 10.105.70.182 | <none>      | 80:30190/TCP | 9m47s |

5. On your local system open a browser and navigate to an HTTPS URL made of the Public IP and the high-numbered port. You will get a message about an insecure connection. Select the **Advanced** button, then **Add Exception...**, then **Confirm Security Exception**. Some browsers won't even give you to option. If nothing shows up try a different browser. The page should then show the Dashboard. You may be able to find the public IP address using **curl**.

```
student@cp:~$ curl ifconfig.io
```

```
34.121.147.3
```

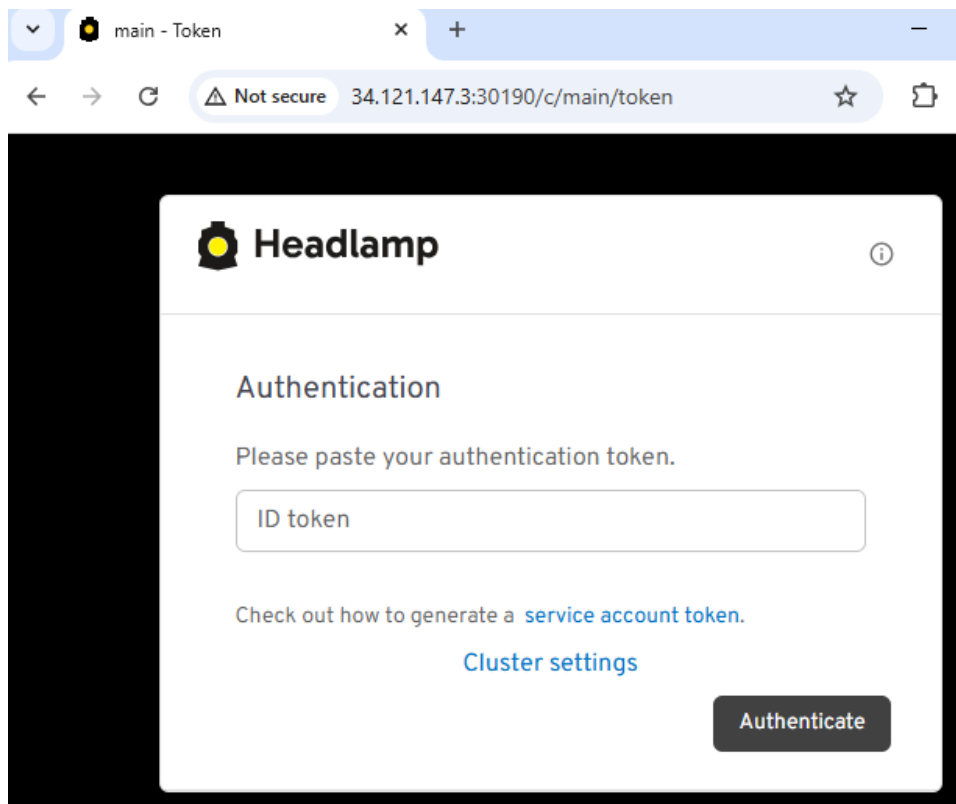


Figure 13.1: External Access via Browser

6. We will use the Token method to access the dashboard. With RBAC we need to use the proper token. Find the token, copy it then paste into the login page. The **Tab** key can be helpful to complete the secret name instead of finding the hash.

```
student@cp:~$ kubectl create token headlamp-admin
```

```
eyJxvezoLAilithbGciOiJSUzI1NiIsImtpZCI6IiJ9.eyJpc3MiOiJrdWJ1cm5ldGVzL3N1cnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWN1YWNjb3VudC9uYW1lc3BhY2UiOiJrdWJ1LXN5c3RlbSIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VjcmV0Lm5hbWUiOiJrdWJ1cm5ldGVzLWRhc2hib2FyZC10b2t1bi1wbW04NCIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VydmljZS1hY2NvdW50Lm5hbWUiOiJrdWJ1cm5ldGVzLWRhc2hib2FyZCIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VydmljZS1hY2NvdW50LnVpZCI6IjE5MDY4ZDIzLTE1MTctMTF1OS1hZmMyLTQyMDEwYThlMDAwMyIsInN1YiI6InN5c3RlbTpzZXJ2aWN1YWNjb3VudDprdWJ1LXN5c3RlbTprdWJ1cm5ldGVzLWRhc2hib2FyZCJ9.aYTUMWr290pjt5i32rb8qXpq4onn3hLhvz6yLSYexgRd6NysyVUyqnkRsFE1trg9i1ftNXKJdzkY5kQzN3AcPUtvyj_BvJgzNh3JM9p7QMjI8LHTz4TrRZrvwJVWitrEn4VnTQuFVcADFD_rKB9FyI_gvT_QiW5fQm24ygTIgfOYd44263oakG8sL64q7UfQNW2wt5S0orMUtybOmX4CXNUYM8G44ejEtv9GW50sVjEmLIGaoEMX7fctwUN_XCyPdzcCg2W0xRHahBJmbCuLz2SSWL52q4nXQmhTq_L8VDDpt6LjEqXW6LtDJZGjVCs2MnBLerQz-ZAgSvaubbQ
```

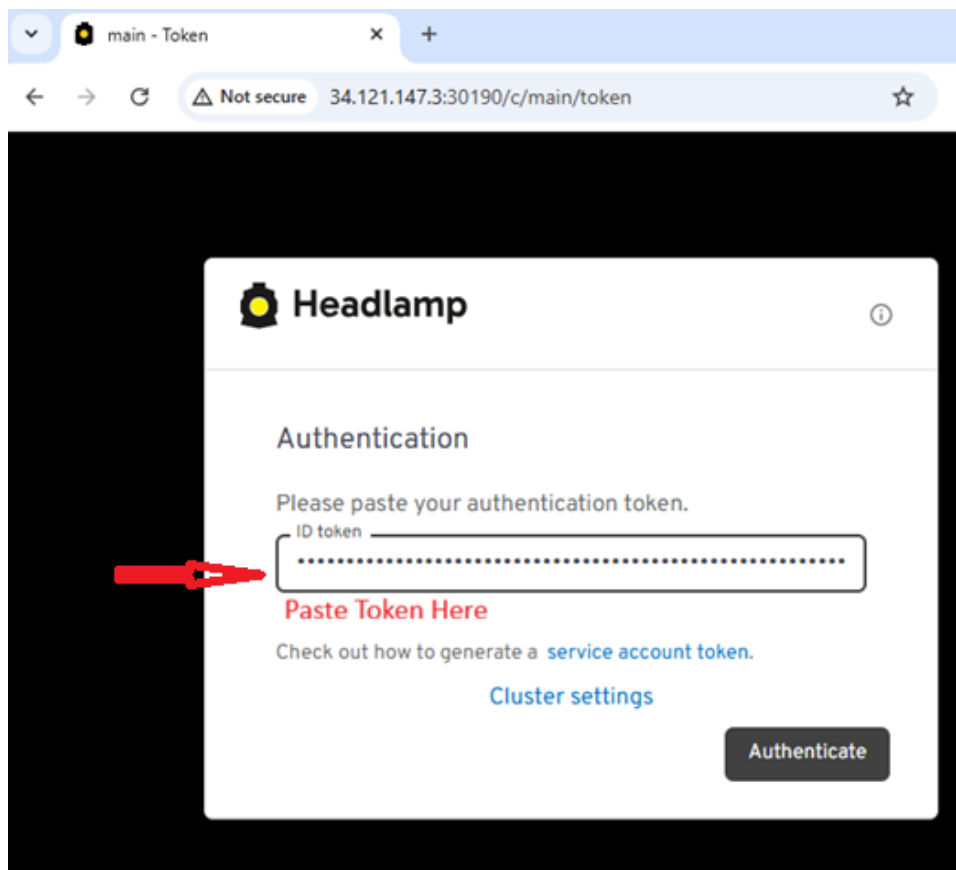


Figure 13.2: External Access via Browser

7. Navigate around the various sections and use the menu to the left as time allows. As the pod view is of the default namespace, you may want to switch over to the `kube-system` namespace or create a new deployment to view the

resources via the GUI. Scale the deployment up and down and watch the responsiveness of the GUI.

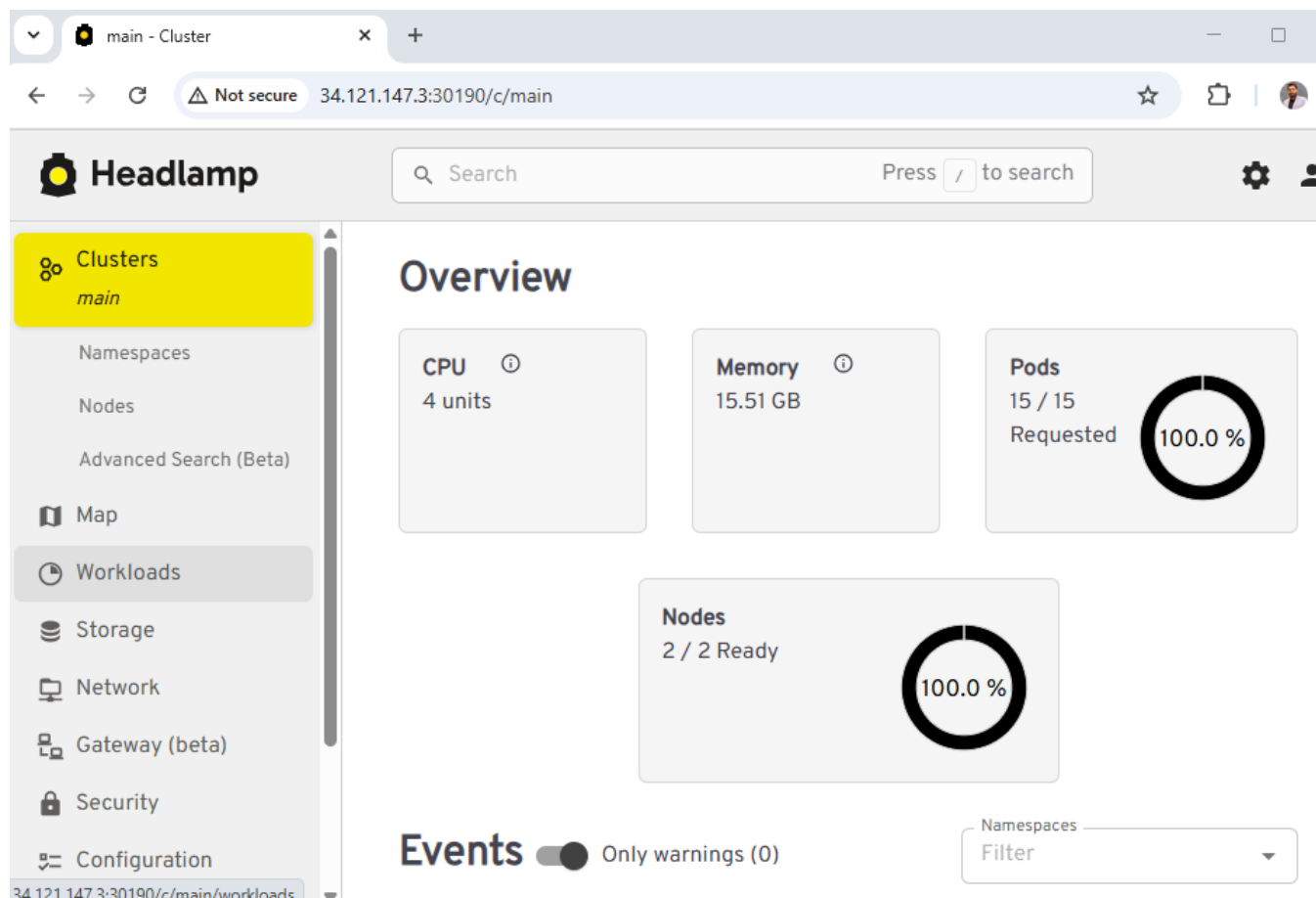


Figure 13.3: External Access via Browser

# Chapter 14

## Custom Resource Definition



|      |                                       |     |
|------|---------------------------------------|-----|
| 14.1 | Overview . . . . .                    | 436 |
| 14.2 | Custom Resource Definitions . . . . . | 437 |
| 14.3 | Aggregated APIs . . . . .             | 441 |
| 14.4 | Labs . . . . .                        | 442 |

## 14.1 Overview

# Custom Resources

- Flexible to meet changing needs
- Create your own API objects
- Manage your API objects via **kubectl**
- Custom Resource Definitions (CRD)
- Aggregated APIs (AA)



## 14.2 Custom Resource Definitions

# Custom Resource Definitions (CRDs)

- Extend the Kubernetes API with custom resources
- No need to run a separate API server (unlike Aggregated APIs)
- Schema-based validation using OpenAPI v3
- Supports Namespaced or Cluster-scoped resources
- Works with controllers/operators for automation

## Configuration Example

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
 name: backups.stable.linux.com
spec:
 group: stable.linux.com
 versions: v1
 scope: Namespaced
 names:
 plural: backups
 singular: backup
 shortNames:
 - bks
 kind: BackUp
```

## New Object Configuration

```
apiVersion: "stable.linux.com/v1"
kind: BackUp
metadata:
 name: a-backup-object
spec:
 timeSpec: "* * * * */5"
 image: linux-backup-image
 replicas: 5
```

# Optional Hooks

- Finalizer

```
metadata:
 finalizers:
 - finalizer.stable.linux.com
```

- Validation

```
validation:
 openAPIV3Schema:
 properties:
 spec:
 properties:
 timeSpec:
 type: string
 pattern: '^(\\d+|*)(/\\d+)?(\\s+(\\d+|*)(/\\d+)?){4}$'
 replicas:
 type: integer
 minimum: 1
 maximum: 10
```

## 14.3 Aggregated APIs

### Understanding Aggregated APIs (AA)

- Usually requires non-trivial programming
- More control over API behavior
- Subordinate API server behind primary
- Primary acts as proxy
- Leverages extension resource
- Mutual TLS auth between API servers
- RBAC rule to allow addition of API service objects

## 14.4 Labs

### Exercise 14.1: Create a Custom Resource Definition

#### Overview

The use of CustomResourceDefinitions (CRD), has become a common manner to deploy new objects and operators. Creation of a new operator is beyond the scope of this course, basically it is a watch-loop comparing a spec to the current status, and making changes until the states match. A good discussion of creating a operators can be found here: <https://operatorframework.io/>.

First we will examine an existing CRD, then make a simple CRD, but without any particular action. It will be enough to find the object ingested into the API and responding to commands.

1. View the existing CRDs.

```
student@cp:~$ kubectl get crd --all-namespaces
```

| NAME                                       | CREATED AT           |
|--------------------------------------------|----------------------|
| authorizationpolicies.policy.linkerd.io    | 2026-03-19T11:30:34Z |
| ciliumcidrgroups.cilium.io                 | 2026-03-19T08:58:54Z |
| ciliumclusterwidenetworkpolicies.cilium.io | 2026-03-19T08:58:57Z |

<output\_omitted>

2. We can see from the names that these CRDs are all working on Cilium, our network plugin. View the [cilium-cni.yaml](#) file we used when we initialized the cluster to see how these objects were created, and some CRD templates to review.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_03/cilium-cni.yaml .
```

```
student@cp:~$ less cilium-cni.yaml
student@cp:~$ kubectl describe crd ciliumcidrgroups.cilium.io
```

```
<output_omitted>

Name: ciliumcidrgroups.cilium.io
Namespace:
Labels: io.cilium.k8s.crd.schema.version=1.26.10
Annotations: <none>
API Version: apiextensions.k8s.io/v1
Kind: CustomResourceDefinition
Metadata:

<output_omitted>
```

3. Now that we have seen some examples, we will create a new YAML file.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_14/crd.yaml .
```

```
student@cp:~$ vim crd.yaml
```

**YAML**

crd.yaml

```
1 apiVersion: apiextensions.k8s.io/v1
2 kind: CustomResourceDefinition
3 metadata:
4 # name must match the spec fields below, and be in the form: <plural>.<group>
5 name: crontabs.stable.example.com
6 spec:
7 # group name to use for REST API: /apis/<group>/<version>
8 group: stable.example.com
```



```

 9 # list of versions supported by this CustomResourceDefinition
10 versions:
11 - name: v1
12 # Each version can be enabled/disabled by Served flag.
13 served: true
14 # One and only one version must be marked as the storage version.
15 storage: true
16 schema:
17 openAPIV3Schema:
18 type: object
19 properties:
20 spec:
21 type: object
22 properties:
23 cronSpec:
24 type: string
25 image:
26 type: string
27 replicas:
28 type: integer
29 # either Namespaced or Cluster
30 scope: Namespaced
31 names:
32 # plural name to be used in the URL: /apis/<group>/<version>/<plural>
33 plural: crontabs
34 # singular name to be used as an alias on the CLI and for display
35 singular: crontab
36 # kind is normally the CamelCased singular type. Your resource manifests use this.
37 kind: CronTab
38 # shortNames allow shorter string to match your resource on the CLI
39 shortNames:
40 - ct

```





41

4. Add the new resource to the cluster.

```
student@cp:~$ kubectl create -f crd.yaml
```

```
customresourcedefinition.apiextensions.k8s.io/crontabs.stable.example.com created
```

5. View and describe the resource. The new line may be in the middle of the output. You'll note the **describe** output is unlike other objects we have seen so far.

```
student@cp:~$ kubectl get crd
```

| NAME                        | CREATED AT           |
|-----------------------------|----------------------|
| <output_omitted>            |                      |
| crontabs.stable.example.com | 2026-03-19T03:18:07Z |
| <output_omitted>            |                      |

```
student@cp:~$ kubectl describe crd crontab<Tab>
```

```
Name: crontabs.stable.example.com
Namespace:
Labels: <none>
Annotations: <none>
API Version: apiextensions.k8s.io/v1
Kind: CustomResourceDefinition
<output_omitted>
```

6. Now that we have a new API resource we can create a new object of that type. In this case it will be a crontab-like image, which does not actually exist, but is being used for demonstration.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_14/new-crontab.yaml .
```

```
student@cp:~$ vim new-crontab.yaml
```

**YA  
ML****new-crontab.yaml**

```
1 apiVersion: "stable.example.com/v1"
2 # This is from the group and version of new CRD
3 kind: CronTab
4 # The kind from the new CRD
5 metadata:
6 name: new-cron-object
7 spec:
8 cronSpec: "* /5 * * * *"
9 image: some-cron-image
10 #Does not exist
```

7. Create the new object and view the resource using short and long name.

```
student@cp:~$ kubectl create -f new-crontab.yaml
```

```
crontab.example.com/new-cron-object created
```

```
student@cp:~$ kubectl get CronTab
```

| NAME            | AGE |
|-----------------|-----|
| new-cron-object | 22s |

```
student@cp:~$ kubectl get ct
```

```
NAME AGE
new-cron-object 29s
```

```
student@cp:~$ kubectl describe ct
```

```
Name: new-cron-object
Namespace: default
Labels: <none>
Annotations: <none>
API Version: stable.example.com/v1
Kind: CronTab
```

```
<output_omitted>
```

```
Spec:
 Cron Spec: */5 * * * *
 Image: some-cron-image
 Events: <none>
```

8. To clean up the resources we will delete the CRD. This should delete all of the endpoints and objects using it as well.

```
student@cp:~$ kubectl delete -f crd.yaml
```

```
customresourcedefinition.apiextensions.k8s.io "crontabs.stable.example.com" deleted
```



# Chapter 15

## Security



|      |                                            |     |
|------|--------------------------------------------|-----|
| 15.1 | Overview . . . . .                         | 450 |
| 15.2 | Accessing the API . . . . .                | 452 |
| 15.3 | Authentication and Authorization . . . . . | 453 |
| 15.4 | Admission Controller . . . . .             | 457 |
| 15.5 | Network Policies . . . . .                 | 459 |
| 15.6 | Labs . . . . .                             | 463 |

## 15.1 Overview

# Overview

- Explain how API requests go through the system.
- Compare different types of authentication.
- Configure authorization rules.
- Configure pod policies to control what containers are allowed to do.
- Restrict network traffic using network policies.

# Cloud Security Considerations

- Ongoing cycle of security
- Image and binary supply chain
- Hardening the operating system
- Securing the kube-apiserver
- Network considerations
- Static and runtime workload analysis
- Issue detection

## 15.2 Accessing the API

# Accessing the API

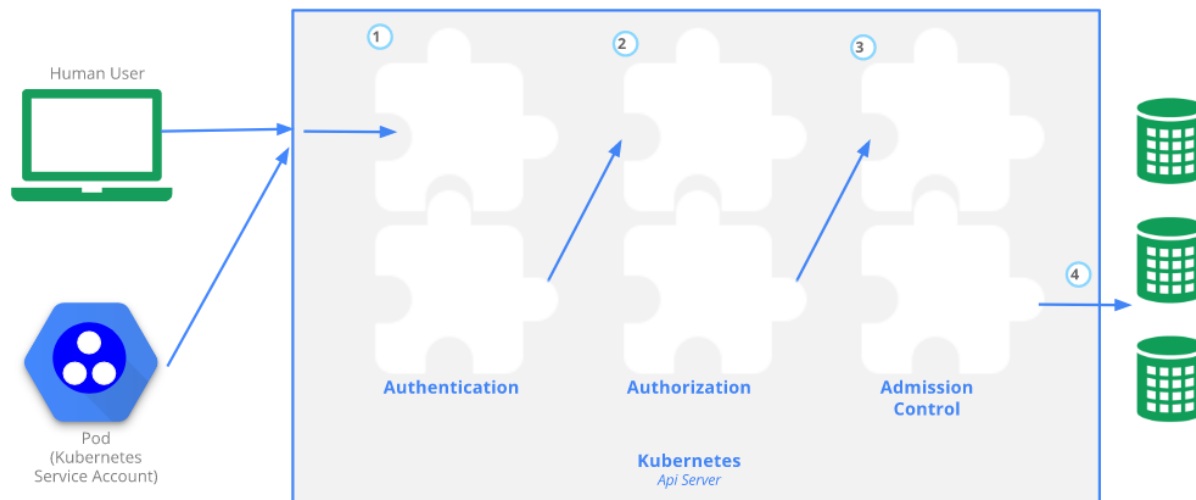


Figure 15.1: Accessing the API from **kubernet.es.io**



## 15.3 Authentication and Authorization

# Authentication

- One or more **Authenticator Modules** used
  - x509 Client Certs
  - Static Token, Bearer or Bootstrap Token
  - Static Password File
  - Service Account and OpenID Connect Tokens
- Each tried until success, order not guaranteed
- Anonymous access can be enabled. Otherwise 401 response.
- Users are not created by the API, should be managed by an external system.
- Use of proxy or webhook to LDAP, SAML, Kerberos, alternate x509 etc. instead.
- System accounts are used by processes to access the API

# Authorization

- **RBAC**
- **WebHook**

# RBAC

- Resources and Operations (verbs)
- Rules
- Roles and ClusterRoles
- Subjects
- RoleBindings and ClusterRoleBindings

# RBAC Process Overview

- Determine or create namespace
- Create certificate credentials for user
- Set the credentials for the user to the namespace using a **context**
- Create a role for the expected task set
- Bind the user to the role
- Verify the user has limited access

## 15.4 Admission Controller

# Admission Controller

- Access content of objects created
- Modify or validate content

# Security Contexts

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
spec:
 securityContext:
 runAsNonRoot: true
 containers:
 - image: nginx
 name: nginx
```

## 15.5 Network Policies

# Network Policies

- Requires a CNI plugin that enforces NetworkPolicies (e.g., Calico, Cilium)
- Supports both Ingress and Egress traffic control
- Policies are namespace-scoped
- Uses podSelector, namespaceSelector, and ipBlock
- Rules are additive (union of all matching policies)

# Network Policy Example

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: ingress-egress-policy
 namespace: default
spec:
 podSelector:
 matchLabels:
 role: db
 policyTypes:
 - Ingress
 - Egress
 ingress:
 - from:
 - ipBlock:
 cidr: 172.17.0.0/16
 except:
 - 172.17.1.0/24
<continued_on_next_slide>
```



## Network Policy Example Cont.

```
- namespaceSelector:
 matchLabels:
 project: myproject
- podSelector:
 matchLabels:
 role: frontend
ports:
- protocol: TCP
 port: 6379
egress:
- to:
 - ipBlock:
 cidr: 10.0.0.0/24
 ports:
 - protocol: TCP
 port: 5978
```

## Default Policy Example

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny
spec:
 podSelector: {}
 policyTypes:
 - Ingress
```

## 15.6 Labs

### Exercise 15.1: Working with TLS

#### Overview

We have learned that the flow of access to a cluster begins with TLS connectivity, then authentication followed by authorization, finally an admission control plug-in allows advanced features prior to the request being fulfilled. The use of `Initializers` allows the flexibility of a shell-script to dynamically modify the request. As security is an important, ongoing concern, there may be multiple configurations used depending on the needs of the cluster.

Every process making API requests to the cluster must authenticate or be treated as an anonymous user.

While one can have multiple cluster root Certificate Authorities (CA) by default each cluster uses their own, intended for intra-cluster communication. The CA certificate bundle is distributed to each node and as a secret to default service accounts. The **kubelet** is a local agent which ensures local containers are running and healthy.

1. View the **kubelet** on both the cp and secondary nodes. The **kube-apiserver** also shows security information such as certificates and authorization mode. As **kubelet** is a **systemd** service we will start looking at that output.

```
student@cp:~$ systemctl status kubelet.service
```

```
kubelet.service - kubelet: The Kubernetes Node Agent
Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: en
Drop-In: /etc/systemd/system/kubelet.service.d
 |_10-kubeadm.conf
<output_omitted>
```

2. Look at the status output. Follow the CGroup and kubelet information, which is a long line where configuration settings are drawn from, to find where the configuration file can be found.

```
CGroup: /system.slice/kubelet.service
|--19523 /usr/bin/kubelet --config=/var/lib/kubelet/config.yaml ..
```

3. Take a look at the settings in the `/var/lib/kubelet/config.yaml` file. Among other information we can see the `/etc/kubernetes/pki/` directory is used for accessing the **kube-apiserver**. Near the end of the output it also sets the directory to find other pod spec files.

```
student@cp:~$ sudo less /var/lib/kubelet/config.yaml
```

**YAML****config.yaml**

```
1 <output_omitted>
2 rotateCertificates: true
3 runtimeRequestTimeout: 0s
4 shutdownGracePeriod: 0s
5 shutdownGracePeriodCriticalPods: 0s
6 staticPodPath: /etc/kubernetes/manifests
7 streamingConnectionIdleTimeout: 0s
8 syncFrequency: 0s
9 volumeStatsAggPeriod: 0s
10
```

4. Other agents on the cp node interact with the **kube-apiserver**. View the configuration files where these settings are made. This was set in the previous YAML file. Look at one of the files for cert information.

```
student@cp:~$ sudo ls /etc/kubernetes/manifests/
```

```
etcd.yaml kube-controller-manager.yaml
kube-apiserver.yaml kube-scheduler.yaml
```

```
student@cp:~$ sudo less /etc/kubernetes/manifests/kube-controller-manager.yaml
```

```
<output_omitted>
```

5. The use of tokens has become central to authorizing component communication. The tokens are kept as **secrets**. Take a look at the current secrets in the `kube-system` namespace. Note: Token is valid only for 24 hours and then the secret is removed, If output does not show 'bootstrap-token-xxxxxx' secret, create a new token

```
student@cp:~$ sudo kubeadm token create
```

```
student@cp:~$ kubectl -n kube-system get secrets
```

```
NAME TYPE
DATA AGE
bootstrap-token-i3r13t bootstrap.kubernetes.io/token
7 5d
<output_omitted>
```

6. Take a closer look at one of the secrets and the token within. The `bootstrap-token` could be one to look at. The use of the Tab key can help with long names. Long lines have been truncated in the output below.

```
student@cp:~$ kubectl -n kube-system get secrets bootstrap-token<Tab> -o yaml
```

YAML

```
1 apiVersion: v1
2 data:
3 auth-extra-groups: c3lzdGVtOmJvb3RzdHJhcHB1cnM6a3ViZWFKbTpkZWZhdWx0LW5vZGUtdG9rZW4=
4 expiration: MjAyMyOWNiOWMlQxMjowOTowMlo=
5 token-id: NXBvMGo3
6 token-secret: MXhzMDgxOG1rcTFyeDQxbg==
```



```
7 usage-bootstrap-authentication: dHJ1ZQ==
8 usage-bootstrap-signing: dHJ1ZQ==
9 kind: Secret
10 metadata:
11 creationTimestamp: "2026-03-19T12:09:02Z"
12 name: bootstrap-token-5po0j7
13 namespace: kube-system
14 resourceVersion: "209"
15 uid: 98219199-4876-4cfa-a4be-586cda27cc6b
16 type: bootstrap.kubernetes.io/token
17
```

7. The **kubect**l **config** command can also be used to view and update parameters. When making updates this could avoid a typo removing access to the cluster. View the current configuration settings. The keys and certs are redacted from the output automatically.

```
student@cp:~$ kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
 certificate-authority-data: REDACTED
 <output_omitted>
```

8. View the options, such as setting a password for the admin instead of a key. Read through the examples and options.

```
student@cp:~$ kubectl config set-credentials -h
```

```
Sets a user entry in kubeconfig
<output_omitted>
```

9. Make a copy of your access configuration file. Later steps will update this file and we can view the differences.

```
student@cp:~$ cp $HOME/.kube/config $HOME/cluster-api-config
```

10. Explore working with cluster and security configurations both using **kubectl** and **kubeadm**. Among other values, find the name of your cluster. You will need to become **root** to work with **kubeadm**.

```
student@cp:~$ kubectl config <Tab><Tab>
```

```
current-context get-contexts set-context view
delete-cluster rename-context set-credentials
delete-context set unset
get-clusters set-cluster use-context
```

```
student@cp:~$ sudo kubeadm token -h
```

```
<output_omitted>
```

```
student@cp:~$ sudo kubeadm config -h
```

```
<output_omitted>
```

11. Review the cluster default configuration settings. There may be some interesting tidbits to the security and infrastructure of the cluster.

```
student@cp:~$ sudo kubeadm config print init-defaults
```

```
apiVersion: kubeadm.k8s.io/v1beta2
bootstrapTokens:
- groups:
 - system:bootstrappers:kubeadm:default-node-token
 token: abcdef.0123456789abcdef
```

```
ttl: 24h0m0s
usages:
<output_omitted>
```

## Exercise 15.2: Authentication and Authorization

Kubernetes clusters have two types of users `service accounts` and `normal users`, but `normal users` are assumed to be managed by an outside service. There are no objects to represent them and they cannot be added via an API call, but `service accounts` can be added.

We will use **RBAC** to configure access to actions within a namespace for a new contractor, `Developer Dan` who will be working on a new project.

1. Create two namespaces, one for production and the other for development.

```
student@cp:~$ kubectl create ns development
```

```
namespace/development created
```

```
student@cp:~$ kubectl create ns production
```

```
namespace/production created
```

2. View the current clusters and context available. The context allows you to configure the cluster to use, namespace and user for **kubectl** commands in an easy and consistent manner.

```
student@cp:~$ kubectl config get-contexts
```



| CURRENT | NAME                        | CLUSTER | AUTHINFO   | NAMESPACE        |
|---------|-----------------------------|---------|------------|------------------|
| *       | kubernetes-admin@kubernetes |         | kubernetes | kubernetes-admin |

3. Create a new user DevDan and assign a password of lftr@in.

```
student@cp:~$ sudo useradd -s /bin/bash DevDan
```

```
student@cp:~$ sudo passwd DevDan
```

```
Enter new UNIX password: lftr@in
Retype new UNIX password: lftr@in
passwd: password updated successfully
```

4. Generate a private key then Certificate Signing Request (CSR) for DevDan. On some Ubuntu 18.04 nodes a missing file may cause an error with random number generation. The **touch** command should ensure one way of success.

```
student@cp:~$ openssl genrsa -out DevDan.key 2048
```

```
Generating RSA private key, 2048 bit long modulus
.....+++
.....+++
e is 65537 (0x10001)
```

```
student@cp:~$ touch $HOME/.rnd
```

```
student@cp:~$ openssl req -new -key DevDan.key \
 -out DevDan.csr -subj "/CN=DevDan/O=development"
```

5. Using the newly created request generate a self-signed certificate using the x509 protocol. Use the CA keys for the Kubernetes cluster and set a 45 day expiration. You'll need to use **sudo** to access to the inbound files.

```
student@cp:~$ sudo openssl x509 -req -in DevDan.csr \
-CA /etc/kubernetes/pki/ca.crt \
-CAkey /etc/kubernetes/pki/ca.key \
-CAcreateserial \
-out DevDan.crt -days 45
```

```
Signature ok
subject=/CN=DevDan/O=development
Getting CA Private Key
```

6. Update the access config file to reference the new key and certificate. Normally we would move them to a safe directory instead of a non-root user's home.

```
student@cp:~$ kubectl config set-credentials DevDan \
--client-certificate=/home/student/DevDan.crt \
--client-key=/home/student/DevDan.key
```

```
User "DevDan" set.
```

7. View the update to your credentials file. Use **diff** to compare against the copy we made earlier.

```
student@cp:~$ diff cluster-api-config .kube/config

16a,19d15
> - name: DevDan
> user:
> as-user-extra: {}
> client-certificate: /home/student/DevDan.crt
> client-key: /home/student/DevDan.key
```

8. We will now create a context. For this we will need the name of the cluster, namespace and CN of the user we set or saw in previous steps.

```
student@cp:~$ kubectl config set-context DevDan-context \
 --cluster=kubernetes \
 --namespace=development \
 --user=DevDan
```

```
Context "DevDan-context" created.
```

9. Attempt to view the Pods inside the DevDan-context. Be aware you will get an error.

```
student@cp:~$ kubectl --context=DevDan-context get pods
```

```
Error from server (Forbidden): pods is forbidden: User "DevDan"
cannot list pods in the namespace "development"
```

10. Verify the context has been properly set.

```
student@cp:~$ kubectl config get-contexts
```

| CURRENT | NAME                        | CLUSTER    | AUTHINFO   | NAMESPACE        |
|---------|-----------------------------|------------|------------|------------------|
|         | DevDan-context              | kubernetes | DevDan     | development      |
| *       | kubernetes-admin@kubernetes | kubernetes | kubernetes | kubernetes-admin |

11. Again check the recent changes to the cluster access config file.

```
student@cp:~$ diff cluster-api-config .kube/config
```

```
<output_omitted>
```

12. We will now create a YAML file to associate RBAC rights to a particular namespace and Role.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_15/role-dev.yaml .
```

```
student@cp:~$ vim role-dev.yaml
```

**YAML**

role-dev.yaml

```
1 kind: Role
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
4 namespace: development
5 name: developer
6 rules:
7 - apiGroups: ["", "extensions", "apps"]
8 resources: ["deployments", "replicasets", "pods"]
9 verbs: ["list", "get", "watch", "create", "update", "patch", "delete"]
10 # You can use ["*"] for all verbs
11
```

13. Create the object. Check white space and for typos if you encounter errors.

```
student@cp:~$ kubectl create -f role-dev.yaml
```

```
role.rbac.authorization.k8s.io/developer created
```

14. Now we create a RoleBinding to associate the Role we just created with a user. Create the object when the file has been created.

```
student@cp:~$ cp /home/student/LFS458/SOLUTIONS/s_15/rolebind.yaml .
```

```
student@cp:~$ vim rolebind.yaml
```

**YA  
ML****rolebind.yaml**

```
1 kind: RoleBinding
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
4 name: developer-role-binding
5 namespace: development
6 subjects:
7 - kind: User
8 name: DevDan
9 apiGroup: ""
10 roleRef:
11 kind: Role
12 name: developer
13 apiGroup: ""
```

```
student@cp:~$ kubectl create -f rolebind.yaml
```

```
rolebinding.rbac.authorization.k8s.io/developer-role-binding created
```

15. Test the context again. This time it should work. There are no Pods running so you should get a response of No resources found.

```
student@cp:~$ kubectl --context=DevDan-context get pods
```

```
No resources found in development namespace.
```

16. Create a new pod, verify it exists, then delete it.

```
student@cp:~$ kubectl --context=DevDan-context \
create deployment nginx --image=nginx
```

```
deployment.apps/nginx created
```

```
student@cp:~$ kubectl --context=DevDan-context get pods
```

| NAME                  | READY | STATUS  | RESTARTS | AGE |
|-----------------------|-------|---------|----------|-----|
| nginx-7c87f569d-7gb9k | 1/1   | Running | 0        | 5s  |

```
student@cp:~$ kubectl --context=DevDan-context delete \
deploy nginx
```

```
deployment.apps "nginx" deleted
```

17. We will now create a different context for production systems. The Role will only have the ability to view, but not create or delete resources. Begin by copying and editing the Role and RoleBindings YAML files.

```
student@cp:~$ cp role-dev.yaml role-prod.yaml
```

```
student@cp:~$ vim role-prod.yaml
```

YAML

role-prod.yaml

```
1 kind: Role
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
```



```

4 namespace: production #<<- This line
5 name: dev-prod #<<- and this line
6 rules:
7 - apiGroups: ["", "extensions", "apps"]
8 resources: ["deployments", "replicasets", "pods"]
9 verbs: ["get", "list", "watch"] #<<- and this one
10

```

```
student@cp:~$ cp rolebind.yaml rolebindprod.yaml
```

```
student@cp:~$ vim rolebindprod.yaml
```



rolebindprod.yaml

```

1 kind: RoleBinding
2 apiVersion: rbac.authorization.k8s.io/v1
3 metadata:
4 name: production-role-binding #<-- Edit to production
5 namespace: production #<-- Also here
6 subjects:
7 - kind: User
8 name: DevDan
9 apiGroup: ""
10 roleRef:
11 kind: Role
12 name: dev-prod #<-- Also this
13 apiGroup: ""

```

18. Create both new objects.

```
student@cp:~$ kubectl create -f role-prod.yaml
```

```
role.rbac.authorization.k8s.io/dev-prod created
```

```
student@cp:~$ kubectl create -f rolebindprod.yaml
```

```
rolebinding.rbac.authorization.k8s.io/production-role-binding created
```

19. Create the new context for production use.

```
student@cp:~$ kubectl config set-context ProdDan-context \
--cluster=kubernetes \
--namespace=production \
--user=DevDan
```

```
Context "ProdDan-context" created.
```

20. Verify that user DevDan can view pods using the new context.

```
student@cp:~$ kubectl --context=ProdDan-context get pods
```

```
No resources found in production namespace.
```

21. Try to create a Pod in production. The developer should be Forbidden.

```
student@cp:~$ kubectl --context=ProdDan-context create \
deployment nginx --image=nginx
```

```
Error from server (Forbidden): deployments.apps is forbidden:
User "DevDan" cannot create deployments.apps in the
```



```
namespace "production"
```

22. View the details of a role.

```
student@cp:~$ kubectl -n production describe role dev-prod
```

```
Name: dev-prod
Labels: <none>
Annotations: kubectl.kubernetes.io/last-applied-configuration=
{"apiVersion":"rbac.authorization.k8s.io/v1","kind":"Role"
,"metadata":{"annotations":{},"name":"dev-prod","namespace":
"production"},"rules":[{"api...
PolicyRule:
 Resources Non-Resource URLs Resource Names Verbs

 deployments [] [] [get list watch]
 deployments.apps [] [] [get list watch]
<output_omitted>
```

23. Experiment with other subcommands in both contexts. They should match those listed in the respective roles.

24. **OPTIONAL CHALLENGE STEP:** Become the DevDan user. Solve any missing configuration errors. Try to create a deployment in the development and the production namespaces. Do the errors look the same? Configure as necessary to only have two contexts available to DevDan.

```
DevDan@cp:~$ kubectl config get-contexts
```

| CURRENT | NAME            | CLUSTER    | AUTHINFO | NAMESPACE   |
|---------|-----------------|------------|----------|-------------|
| *       | DevDan-context  | kubernetes | DevDan   | development |
|         | ProdDan-context | kubernetes | DevDan   | production  |

## Exercise 15.3: Admission Controllers

The last stop before a request is sent to the API server is an admission control plug-in. They interact with features such as setting parameters like a default storage class, checking resource quotas, or security settings. A newer feature (v1.7.x) is dynamic controllers which allow new controllers to be ingested or configured at runtime.

1. View the current admission controller settings. Unlike earlier versions of Kubernetes the controllers are now compiled into the server, instead of being passed at run-time. Instead of a list of which controllers to use we can enable and disable specific plugins.

```
student@cp:~$ sudo grep admission \
/etc/kubernetes/manifests/kube-apiserver.yaml
```

```
- --enable-admission-plugins=NodeRestriction
```

# Chapter 16

## High Availability



|      |                             |     |
|------|-----------------------------|-----|
| 16.1 | Overview . . . . .          | 480 |
| 16.2 | Stacked Database . . . . .  | 481 |
| 16.3 | External Database . . . . . | 482 |
| 16.4 | Labs . . . . .              | 483 |

## 16.1 Overview

# Cluster High Availability

- Uses load balancer
- Three nodes for database quorum
- Stacked control planes and **etcd** nodes
- External **etcd** nodes
- Use FQDN for SSL connections

## 16.2 Stacked Database

### Co-located Databases

- Default database location
- Simpler to deploy and manage
- Requires at least three nodes
- Instance failure would remove essential services and database

## 16.3 External Database

### Non-collocated Databases

- Configure HA **etcd** cluster
- Manually copy PKI certificates
- Configure first control plane
- Add more control planes
- Add worker nodes

## 16.4 Labs

### Exercise 16.1: High Availability Setup Guidelines

#### Overview

In this lab we will add two more control planes to our cluster, change taints and deploy an application to a particular node, and test that we can access it from outside the cluster. The nodes will handle various infrastructure services and the **etcd** database and should be sized accordingly.

The steps are presented in two ways. First the general steps for those interested in more of a challenge. Following that will be the detailed steps found in previous labs.

You will need three more nodes. One to act as a load balancer, the other two will act as cp nodes for quorum. Log into each and use the **ip** command to fill in the table with the IP addresses of the primary interface of each node. If using **GCE** nodes it would be `ens4`, yours may be different. You may need to install software such as an editor on the nodes.

|                      |  |
|----------------------|--|
| Proxy Node           |  |
| Second Control Plane |  |
| Third Control Plane  |  |

As the prompts may look similar you may want to change the terminal color or other characteristics to make it easier to keep them distinct. You can also change the prompt using something like: **PS1="ha-proxy\$ "**, which may help to keep the terminals distinct.

#### High level steps:

1. Deploy a load balancer configured to pass through traffic on your new proxy node. HAProxy is easy to deploy using online documentation. Start with forwarding traffic of the cp alias to just the working cp.
2. Install the Kubernetes software on the second and third cp nodes.

3. Use **kubeadm join** on the second cp, adding it to the cluster as another control plane using the node name.
4. Join the third cp as another control plane to the cluster using the node name.
5. Update the proxy to use all three cps backend IPs.
6. Temporarily shut down the first cp and monitor traffic.

## Detailed Steps:

### Deploy a Load Balancer

While there are many options, both software and hardware, we will be using an open source tool **HAProxy** to configure a load balancer.

1. Deploy HAProxy. Log into the proxy node. Update the repos then install a the HAProxy software. Answer yes, should you the installation ask if you will allow services to restart.

```
student@ha-proxy:~$ sudo apt-get update ; sudo apt-get install -y haproxy vim
```

```
<output_omitted>
```

2. Edit the configuration file and add sections for the front-end and back-end servers. We will comment out the second and third cp node until we are sure the proxy is forwarding traffic to the known working cp. **Note: A working template has been made available inside the Solutions directory**

```
student@ha-proxy:~$ sudo vim /etc/haproxy/haproxy.cfg
```

```
....
defaults
```

```
 log global
```

```
 #<-- Edit these three lines, starting around line 23
```



```

 option tcplog
 mode tcp

 errorfile 503 /etc/haproxy/errors/503.http
 errorfile 504 /etc/haproxy/errors/504.http

frontend proxynode #<-- Add the following lines to bottom of file
 bind *:80
 bind *:6443
 stats uri /proxystats
 default_backend k8sServers

backend k8sServers
 balance roundrobin
 server cp 10.128.0.24:6443 check #<-- Edit these with your IP addresses, port, and hostname
server secondcp 10.128.0.30:6443 check #<-- Comment out until ready
server thirdcp 10.128.0.66:6443 check #<-- Comment out until ready
listen stats
 bind :9999
 mode http
 stats enable
 stats hide-version
 stats uri /stats

```

- Restart the haproxy service and check the status. You should see the frontend and backend proxies report being started.

```

student@ha-proxy:~$ sudo systemctl restart haproxy.service
student@ha-proxy:~$ sudo systemctl status haproxy.service

```

```

<output_omitted>
Aug 08 18:43:08 ha-proxy systemd[1]: Starting HAProxy Load Balancer...
Aug 08 18:43:08 ha-proxy systemd[1]: Started HAProxy Load Balancer.
Aug 08 18:43:08 ha-proxy haproxy-systemd-wrapper[13602]: haproxy-systemd-wrapper:
Aug 08 18:43:08 ha-proxy haproxy[13603]: Proxy proxynode started.
Aug 08 18:43:08 ha-proxy haproxy[13603]: Proxy proxynode started.

```

```
Aug 08 18:43:08 ha-proxy haproxy[13603]: Proxy k8sServers started.
Aug 08 18:43:08 ha-proxy haproxy[13603]: Proxy k8sServers started.
```

4. **On the cp** Edit the `/etc/hosts` file and comment out the old and add a new `k8scp` alias to the IP address of the proxy server.

```
student@cp:~$ sudo vim /etc/hosts
```

```
10.128.0.64 k8scp #<-- Add alias to proxy IP
#10.128.0.24 k8scp #<-- Comment out the old alias, in case its needed
127.0.0.1 localhost
....
```

5. Use a local browser to navigate to the public IP of your proxy server. The <http://34.69.XX.YY:9999/stats> is an example your IP address would be different. Leave the browser up and refresh as you run following steps. You can find your public ip using **curl**. Your IP will be different than the one shown below.

```
ha-proxy$ curl ifconfig.io
```

```
34.69.73.159
```

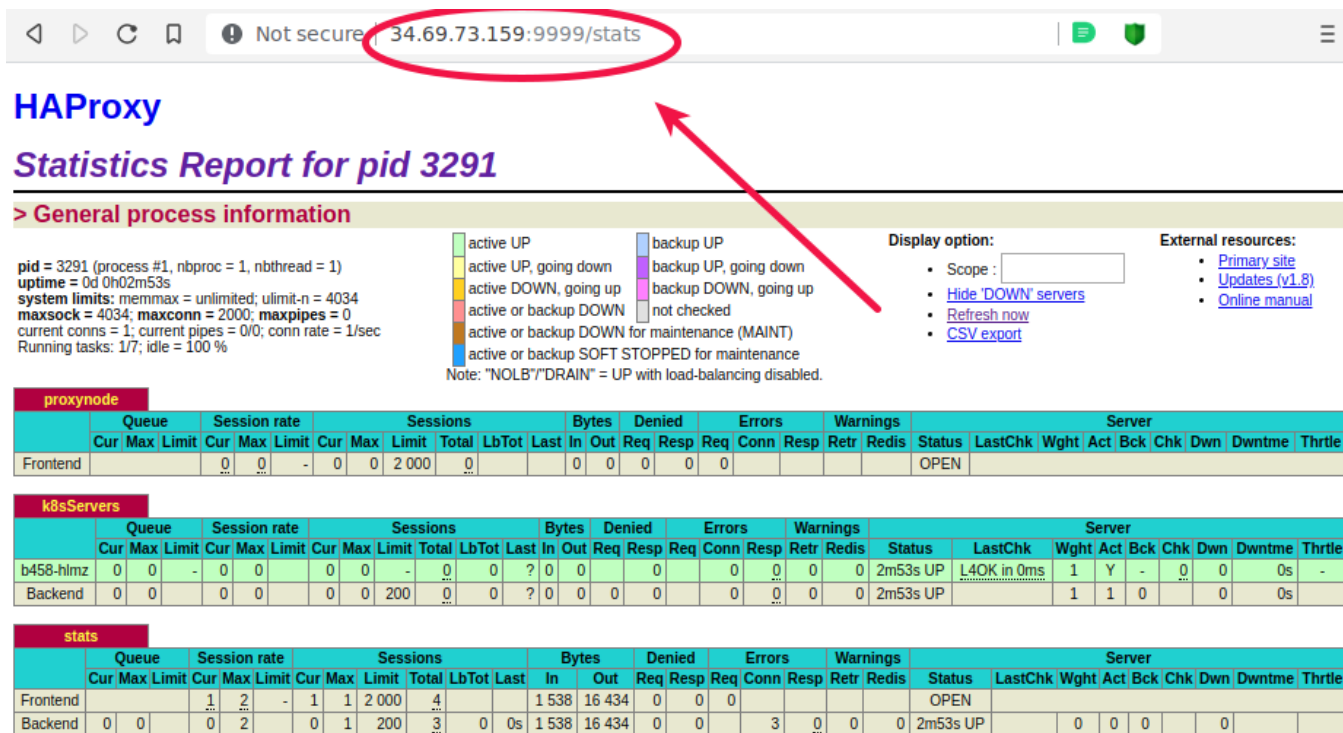


Figure 16.1: Initial HAProxy Status

6. Check the node status from the cp node then check the proxy statistics. You should see the byte traffic counter increase.

```
student@cp:~$ kubectl get nodes
```

| NAME   | STATUS | ROLES         | AGE  | VERSION |
|--------|--------|---------------|------|---------|
| cp     | Ready  | control-plane | 2d6h | v1.35.2 |
| worker | Ready  | <none>        | 2d3h | v1.35.2 |

## Install Software

We will add two more control planes with stacked **etcd** databases for cluster quorum. You may want to open up two more PuTTY or SSH sessions and color code the terminals to keep track of the nodes.

Initialize the second cp before adding the third cp

1. Configure and install the kubernetes software on the **second cp**. Use the same steps as when we first set up the cluster, earlier in the course. You may want to copy and paste from earlier commands in your **history** to make these steps easier. **All the steps up to but not including `kubeadm init` or `kubeadm join`** A script `k8sWorker.sh` has been included in the course tarball to make this process go faster, if you would like. View and edit the script to be the correct version before running it.
2. Install the software on the **third cp** using the same commands.

## Join Control Plane Nodes

1. Edit the `/etc/hosts` file **ON ALL NODES** to ensure the alias of `k8scp` is set on each node to the proxy IP address. Your IP address may be different.

```
student@cp:~$ sudo vim /etc/hosts
```

```
10.128.0.64 k8scp
#10.128.0.24 k8scp
127.0.0.1 localhost
....
```

2. On the **first cp** create the tokens and hashes necessary to join the cluster. These commands may be in your **history** and easier to copy and paste.
3. Create a new token.

```
student@cp:~$ sudo kubeadm token create
```

```
jasg79.fdh4p279l320cz1g
```

4. Create a new SSL hash.

```
student@cp:~$ openssl x509 -pubkey \
-in /etc/kubernetes/pki/ca.crt | openssl rsa \
-pubin -outform der 2>/dev/null | openssl dgst \
-sha256 -hex | sed 's/^.* //'
```

```
f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd
```

5. Create a new cp certificate to join as a cp instead of as a worker.

```
student@cp:~$ sudo kubeadm init phase upload-certs --upload-certs
```

```
[upload-certs] Storing the certificates in Secret "kubeadm-certs" in the "kube-system" Namespace
[upload-certs] Using certificate key:
5610b6f73593049acddee6b59994360aa4441be0c0d9277c76705d129ba18d65
```

6. On the **second cp** use the previous output to build a **kubeadm join** command. Please be aware that multi-line copy and paste from Windows and some MacOS has paste issues. If you get unexpected output copy one line at a time.

```
student@$secondcp:~$ sudo kubeadm join k8scp:6443 \
--token jasg79.fdh4p279l320cz1g \

```

```
--discovery-token-ca-cert-hash sha256:f62bf97d4fba6876e4c3ff645df3fca969c06169dee3865aab9d0bca8ec9f8cd \
--control-plane --node-name=secondcp --certificate-key \
5610b6f73593049acddee6b59994360aa4441be0c0d9277c76705d129ba18d65
```

```
[preflight] Running pre-flight checks
[WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the Docker cgroup driver. The recommended
↪ driver \
 is "systemd". Please follow the guide at https://kubernetes.io/docs/setup/cri/
<output_omitted>
```

- Return to the first cp node and check to see if the node has been added and is listed as a cp.

```
student@cp:~$ kubectl get nodes
```

| NAME     | STATUS | ROLES         | AGE  | VERSION |
|----------|--------|---------------|------|---------|
| cp       | Ready  | control-plane | 2d6h | v1.35.2 |
| secondcp | Ready  | control-plane | 10m  | v1.35.2 |
| worker   | Ready  | <none>        | 2d3h | v1.35.2 |

- Copy and paste the **kubeadm join** command to the third cp. Remember to change the node name to **thirdcp** Then check that the third cp has been added.

```
student@cp:~$ kubectl get nodes
```

| NAME     | STATUS | ROLES         | AGE  | VERSION |
|----------|--------|---------------|------|---------|
| cp       | Ready  | control-plane | 2d6h | v1.35.2 |
| secondcp | Ready  | control-plane | 13m  | v1.35.2 |
| thirdcp  | Ready  | control-plane | 3m   | v1.35.2 |
| worker   | Ready  | <none>        | 2d3h | v1.35.2 |

9. Copy over the configuration file as suggested in the output at the end of the join command. Do this on both newly added cp nodes.

```
student@secondcp:~$ mkdir -p $HOME/.kube
student@secondcp:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
student@secondcp:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

10. On the **Proxy node**. Edit the proxy to include all three cp nodes then restart the proxy. **Note: A working template has been made available inside the Solutions directory**

```
student@ha-proxy:~$ sudo vim /etc/haproxy/haproxy.cfg
```

```
....
backend k8sServers
 balance roundrobin
 server cp 10.128.0.24:6443 check
 server secondcp 10.128.0.30:6443 check #<-- Edit/Uncomment these lines
 server thirdcp 10.128.0.66:6443 check #<--
....
```

```
student@ha-proxy:~$ sudo systemctl restart haproxy.service
```

11. View the proxy statistics. When it refreshes you should see three new back-ends. As you check the status of the nodes using **kubectl get nodes** you should see the byte count increase on each node indicating each is handling some of the requests.

| proxynode |       |     |       |              |     |       |          |     |         |       |       |      |        |         |        |      |  |
|-----------|-------|-----|-------|--------------|-----|-------|----------|-----|---------|-------|-------|------|--------|---------|--------|------|--|
|           | Queue |     |       | Session rate |     |       | Sessions |     |         |       |       |      | Bytes  |         | Denied |      |  |
|           | Cur   | Max | Limit | Cur          | Max | Limit | Cur      | Max | Limit   | Total | LbTot | Last | In     | Out     | Req    | Resp |  |
| Frontend  |       |     |       | 0            | 5   | -     | 4        | 6   | 262 121 | 34    |       |      | 57 679 | 333 984 | 0      | 0    |  |

| k8s Servers |       |     |       |              |     |       |          |     |        |       |       |      |        |         |        |      |  |
|-------------|-------|-----|-------|--------------|-----|-------|----------|-----|--------|-------|-------|------|--------|---------|--------|------|--|
|             | Queue |     |       | Session rate |     |       | Sessions |     |        |       |       |      | Bytes  |         | Denied |      |  |
|             | Cur   | Max | Limit | Cur          | Max | Limit | Cur      | Max | Limit  | Total | LbTot | Last | In     | Out     | Req    | Resp |  |
| cp          | 0     | 0   | -     | 0            | 2   |       | 2        | 3   | -      | 12    | 12    | 2s   | 19 280 | 89 042  | 0      | 0    |  |
| secondcp    | 0     | 0   | -     | 0            | 2   |       | 1        | 2   | -      | 11    | 11    | 3s   | 19 147 | 115 928 | 0      | 0    |  |
| thirdcp     | 0     | 0   | -     | 0            | 2   |       | 1        | 2   | -      | 11    | 11    | 3s   | 19 252 | 129 014 | 0      | 0    |  |
| Backend     | 0     | 0   |       | 0            | 5   |       | 4        | 5   | 26 213 | 34    | 34    | 2s   | 57 679 | 333 984 | 0      | 0    |  |

| stats    |       |     |       |              |     |       |          |     |         |       |       |      |       |         |        |      |   |
|----------|-------|-----|-------|--------------|-----|-------|----------|-----|---------|-------|-------|------|-------|---------|--------|------|---|
|          | Queue |     |       | Session rate |     |       | Sessions |     |         |       |       |      | Bytes |         | Denied |      |   |
|          | Cur   | Max | Limit | Cur          | Max | Limit | Cur      | Max | Limit   | Total | LbTot | Last | In    | Out     | Req    | Resp |   |
| Frontend |       |     |       | 0            | 3   | -     | 1        | 4   | 262 121 | 11    |       |      | 4 916 | 119 887 | 0      | 0    | 3 |
| Backend  | 0     | 0   |       | 0            | 1   |       | 0        | 1   | 26 213  | 7     | 0     | 0s   | 4 916 | 119 887 | 0      | 0    |   |

Figure 16.2: Multiple HAProxy Status

12. View the logs of the newest **etcd** pod. Leave it running, using the **-f** option in one terminal while running the following commands in a different terminal. As you have copied over the cluster admin file you can run **kubectl** on any cp.

```
student@cp:~$ kubectl -n kube-system get pods |grep etcd
```

```
etcd-cp 1/1 Running 0 2d12h
etcd-secondcp 1/1 Running 0 22m
etcd-thirdcp 1/1 Running 0 18m
```

```
student@cp:~$ kubectl -n kube-system logs -f etcd-thirdcp
```



```

....
2026-03-09 01:58:03.768858 I | mvcc: store.index: compact 300473
2026-03-09 01:58:03.770773 I | mvcc: finished scheduled compaction at 300473 (took 1.286565ms)
2026-03-09 02:03:03.766253 I | mvcc: store.index: compact 301003
2026-03-09 02:03:03.767582 I | mvcc: finished scheduled compaction at 301003 (took 995.775µs)
2026-03-09 02:08:03.785807 I | mvcc: store.index: compact 301533
2026-03-09 02:08:03.787058 I | mvcc: finished scheduled compaction at 301533 (took 913.185µs)

```

13. Log into one of the **etcd** pods and check the cluster status, using the IP address of each server and port 2379. Your IP addresses may be different. Exit back to the node when done.

```

student@cp:~$ kubectl -n kube-system exec -it etcd-cp -- \
etcdctl \
--endpoints=https://10.128.0.66:2379,https://10.128.0.24:2379,https://10.128.0.30:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
-w table endpoint status

```

| ENDPOINT         | ID               | VERSION | DB SIZE | IS LEADER | RAFT TERM | RAFT INDEX |
|------------------|------------------|---------|---------|-----------|-----------|------------|
| 10.128.0.66:2379 | 2331065cd4fb02ff | 3.5.7   | 24 MB   | true      | 11        | 392573     |
| 10.128.0.24:2379 | d2620a7d27a9b449 | 3.5.7   | 24 MB   | false     | 11        | 392573     |
| 10.128.0.30:2379 | ef44cc541c5f37c7 | 3.5.7   | 24 MB   | false     | 11        | 392573     |

## Test Failover

Now that the cluster is running and has chosen a leader we will shut down containerd, which will stop all containers on that node. This will emulate an entire node failure. We will then view the change in leadership and logs of the events.

1. Shut down the node which shows IS LEADER set to true.

```
student@cp:~$ sudo reboot
```

If you chose cri-o as the container engine then the cri-o service and common processes are distinct. It may be easier to reboot the node and refresh the HAProxy web page until it shows the node is down. It may take a while for the node to finish the boot process. The second and third cp should work the entire time.

2. You will probably note the **logs** command exited when the service shut down. Run the same command and, among other output, you'll find errors similar to the following. Note the messages about losing the leader and electing a new one, with an eventual message that a peer has become inactive.

```
student@cp:~$ kubectl -n kube-system logs -f etcd-thirdcp
```

```
....
2026-03-09 02:11:39.569827 I | raft: 2331065cd4fb02ff [term: 9] received a MsgVote message with
↳ higher \
 term from ef44cc541c5f37c7 [term: 10]
2026-03-09 02:11:39.570130 I | raft: 2331065cd4fb02ff became follower at term 10
2026-03-09 02:11:39.570148 I | raft: 2331065cd4fb02ff [logterm: 9, index: 355240, vote: 0] cast
↳ MsgVote \
 for ef44cc541c5f37c7 [logterm: 9, index: 355240] at term 10
2026-03-09 02:11:39.570155 I | raft: raft.node: 2331065cd4fb02ff lost leader d2620a7d27a9b449 at
↳ term 10
2026-03-09 02:11:39.572242 I | raft: raft.node: 2331065cd4fb02ff elected leader ef44cc541c5f37c7
↳ at \
 term 10
2026-03-09 02:11:39.682319 W | rafthttp: lost the TCP streaming connection with peer
↳ d2620a7d27a9b449 \
 (stream Message reader)
```

```

2026-03-09 02:11:39.682635 W | rafthttp: lost the TCP streaming connection with peer
↳ d2620a7d27a9b449 \
 (stream MsgApp v2 reader)
2026-03-09 02:11:39.706068 E | rafthttp: failed to dial d2620a7d27a9b449 on stream MsgApp v2 \
 (peer d2620a7d27a9b449 failed to find local node 2331065cd4fb02ff)
2026-03-09 02:11:39.706328 I | rafthttp: peer d2620a7d27a9b449 became inactive (message send to
↳ peer failed)
....

```

3. View the proxy statistics. The proxy should show the first cp as down, but the other cp nodes remain up.

| k8sServers |       |     |       |              |     |       |          |     |        |       |       |       |         |           |        |      |        |      |      |          |       |           |                    |          |  |  |
|------------|-------|-----|-------|--------------|-----|-------|----------|-----|--------|-------|-------|-------|---------|-----------|--------|------|--------|------|------|----------|-------|-----------|--------------------|----------|--|--|
|            | Queue |     |       | Session rate |     |       | Sessions |     |        |       |       |       | Bytes   |           | Denied |      | Errors |      |      | Warnings |       | Status    | LastChk            | Serv Wgt |  |  |
|            | Cur   | Max | Limit | Cur          | Max | Limit | Cur      | Max | Limit  | Total | LbTot | Last  | In      | Out       | Req    | Resp | Req    | Conn | Resp | Retr     | Redis |           |                    |          |  |  |
| cp         | 0     | 0   | -     | 0            | 2   | -     | 2        | 3   | -      | 23    | 23    | 1m50s | 40 091  | 518 686   | 0      | 0    | 0      | 0    | 0    | 0        | 0     | 2s DOWN   | * L4TOUT in 2001ms | 1        |  |  |
| secondcp   | 0     | 0   | -     | 0            | 2   | -     | 1        | 2   | -      | 23    | 23    | 1m50s | 48 861  | 302 977   | 0      | 0    | 0      | 0    | 0    | 0        | 0     | 41m18s UP | L4OK in 0ms        | 1        |  |  |
| thirdcp    | 0     | 0   | -     | 0            | 2   | -     | 1        | 2   | -      | 22    | 22    | 1m54s | 38 446  | 257 768   | 0      | 0    | 0      | 0    | 0    | 0        | 0     | 41m18s UP | L4OK in 0ms        | 1        |  |  |
| Backend    | 0     | 0   | -     | 0            | 5   | -     | 4        | 7   | 26 213 | 68    | 68    | 1m50s | 127 398 | 1 079 431 | 0      | 0    | 0      | 0    | 0    | 0        | 0     | 41m18s UP |                    | 2        |  |  |

| stats    |       |     |       |              |     |       |          |     |         |       |       |      |        |         |        |      |        |      |      |          |       |           |         |      |  |  |
|----------|-------|-----|-------|--------------|-----|-------|----------|-----|---------|-------|-------|------|--------|---------|--------|------|--------|------|------|----------|-------|-----------|---------|------|--|--|
|          | Queue |     |       | Session rate |     |       | Sessions |     |         |       |       |      | Bytes  |         | Denied |      | Errors |      |      | Warnings |       | Status    | LastChk | Wght |  |  |
|          | Cur   | Max | Limit | Cur          | Max | Limit | Cur      | Max | Limit   | Total | LbTot | Last | In     | Out     | Req    | Resp | Req    | Conn | Resp | Retr     | Redis |           |         |      |  |  |
| Frontend |       |     |       | 0            | 3   | -     | 1        | 4   | 262 121 | 24    |       |      | 11 228 | 282 167 | 0      | 0    | 8      |      |      |          |       | OPEN      |         |      |  |  |
| Backend  | 0     | 0   |       | 0            | 1   |       | 0        | 1   | 26 213  | 15    | 0     | 0s   | 11 228 | 282 167 | 0      | 0    |        | 15   |      | 0        | 0     | 41m18s UP |         | 0    |  |  |

Figure 16.3: HAProxy Down Status

4. View the status using **etcdctl** from within one of the running **etcd** pods. You should get an error for the endpoint you shut down and a new leader of the cluster.

```

student@secondcp:~$ kubectl -n kube-system exec -it etcd-secondcp -- \
etcdctl \
--endpoints=https://10.128.0.66:2379,https://10.128.0.24:2379,https://10.128.0.30:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \

```

```
--key=/etc/kubernetes/pki/etcd/server.key \
-w table endpoint status
```

```
Failed to get the status of endpoint 10.128.0.66:2379 (context deadline exceeded)
```

| ENDPOINT         | ID               | VERSION | DB SIZE | IS LEADER | RAFT TERM | RAFT INDEX |
|------------------|------------------|---------|---------|-----------|-----------|------------|
| 10.128.0.24:2379 | d2620a7d27a9b449 | 3.5.7   | 24 MB   | true      | 12        | 395729     |
| 10.128.0.30:2379 | ef44cc541c5f37c7 | 3.5.7   | 24 MB   | false     | 12        | 395729     |

5. Turn the containerd service back on. You should see the peer become active and establish a connection.

```
student@cp:~$ sudo systemctl start containerd.service
```

```
student@cp:~$ kubectl -n kube-system logs -f etcd-thirdcp
```

```
....
2026-03-09 02:45:11.337669 I | rafthttp: peer d2620a7d27a9b449 became active
2026-03-09 02:45:11.337710 I | rafthttp: established a TCP streaming connection with peer\
 d2620a7d27a9b449 (stream MsgApp v2 reader)
....
```

6. View the **etcd** cluster status again. Experiment with how long it takes for the **etcd** cluster to notice failure and choose a new leader with the time you have left.

## Chapter 17

# Closing and Evaluation Survey



|      |                             |     |
|------|-----------------------------|-----|
| 17.1 | Evaluation Survey . . . . . | 498 |
|------|-----------------------------|-----|

## 17.1 Evaluation Survey

# Evaluation Survey

Thank you for taking this course brought to you by **The Linux Foundation**. Your comments are important to us and we take them seriously, both to measure how well we fulfilled your needs and to help us improve future sessions.

- Please Evaluate your training using the link your instructor will provide.
- Please be sure to check the spelling of your name and use correct capitalization as this is how your name will appear on the certificate.
- This information will be used to generate your **Certificate of Completion**, which will be sent to the email address you have supplied, so make sure it is correct.



Figure 17.1: Course Survey

# Appendices





# Appendix A

## Domain Review



|     |                              |     |
|-----|------------------------------|-----|
| A.1 | CKA Exam . . . . .           | 502 |
| A.2 | Exam Domain Review . . . . . | 503 |

## A.1 CKA Exam

# Domain Review

- Review the Candidate Handbook
- Find the proper version of the Curriculum Overview
- Write out each bullet point and steps
- Recognize good YAML examples in documentation
- Practice at speed

## A.2 Exam Domain Review

### Exercise A.1: Are you Ready?

1. If you have not searched for working and tested YAML examples, and can easily search for them again for all of the subjects the domain review mentions for the exam, you may:
  - a. Run out of time while searching for good YAML examples.
  - b. Make more mistakes.
  - c. Use YAML that isn't proper for the Kubernetes version of the exam and waste time trying to troubleshoot the issue.
  - d. All of the above.
2. Answer all that apply. In the context of the Curriculum Overview the term **Understand** when stating what a candidate should be able to do means:
  - a. You only have a general idea what the object does.
  - b. You can create the object.
  - c. You can configure and integrate the object with other objects.
  - d. You can properly update and test the object.
  - e. You can troubleshoot the object.
3. Have you practiced creating, integrating, and troubleshooting all of the domain review items **at speed**?
  - a. No. I kind of did the exercises. So I'm good.
  - b. No. I did the labs twice over a two week period.
  - c. Yes. I know the exam is intense and practiced with a clock running to make sure I can get everything done and also check my work.

### Solution A.1

## Are You Ready?

1. d.
2. b, c, d, e.
3. Hopefully c.

## Exercise A.2: Preparing for the CKA Exam



### Very Important

The source pages and content in this review could change at any time. **IT IS YOUR RESPONSIBILITY TO CHECK THE CURRENT INFORMATION.**

## Before Taking Exam

Use this exercise as a resource after you complete the course but before you take the exam. Review the resources, know what good YAML looks like, and practice creating and working with objects at exam speed to assist with review and preparation.

1. Using a browser go to <https://www.cncf.io/certification/cka/> and read through the program description.
2. In the **Exam Resources** section open the [Curriculum Overview](#) and [Candidate-handbook](#) in new tabs. Both of these should be read and understood prior to sitting for the exam.
3. Navigate to the [Curriculum Overview](#) tab. You should see links for domain information for various versions of the exam. Select the latest version, such as **CKA\_Curriculum\_V1.25.pdf**. The versions you see may be different. You should see a new page showing a PDF.
4. Read through the document. Be aware that the term `Understand`, such as `Understand Services`, is more than just knowing they exist. In this case expect it to also mean create, configure, update, and troubleshoot.

5. Using only the exam-allowed URLs and sub-domains search for YAML examples for each domain or skill item. Ensure it works for the version of the exam you are taking, as the YAML may not have been re-tested after a new release. Become familiar with out to find each good example again, so you can find the page again during the exam.
6. Using a timer see how long it takes you to create and verify the objects listed below. Write down the time. Try it again and see how much faster you can complete and test each step.

"Practice until you get it right. Then practice until you can't get it wrong" -Unknown

## Domain Review Items

This list is copied from competency domains found on the PDF. Again, **it remains your responsibility to check the web page for any changes to this list.**

- **Cluster Architecture, Installation & Configuration**

- Manage role based access control (RBAC)
- Use Kubeadm to install a basic cluster
- Manage a highly-available Kubernetes cluster
- Provision underlying infrastructure to deploy a Kubernetes cluster
- Perform a version upgrade on a Kubernetes cluster using Kubeadm
- Implement etcd backup and restore

- **Workloads & Scheduling**

- Understand deployments and how to perform rolling updates and rollbacks
- Use ConfigMaps and Secrets to configure applications
- Know how to scale applications

- Understand the primitives used to create robust, self-healing, application deployments
- Understand how resource limits can affect Pod scheduling
- Awareness of manifest management and common templating tools

- **Services & Networking**

- Understand host networking configuration on the cluster nodes
- Understand connectivity between Pods
- Understand ClusterIP, NodePort, LoadBalancer service types and endpoints
- Know how to use Ingress controllers and Ingress resources
- Know how to configure and use CoreDNS
- Choose an appropriate container network interface plugin

- **Storage**

- Understand storage classes, persistent volumes
- Understand volume mode, access modes and reclaim policies for volumes
- Understand persistent volume claims primitive
- Know how to configure applications with persistent storage

- **Troubleshooting**

- Evaluate cluster and node logging
- Understand how to monitor applications
- Manage container stdout & stderr logs
- Troubleshoot application failure
- Troubleshoot cluster component failure
- Troubleshoot networking

## Exercise A.3: Practicing Skills

This exercise is to help you practice your skills. It does not cover all the items listed in the domain review guide. You should develop your own steps to build a full list of skill tests and steps.

Also note that all the detailed steps are not included. You should be able to complete these steps without being told what to type.

In a work or exam environment you may not be told exactly what to do or how to do it. The following steps are meant to get you used to thinking about solutions when the exact need isn't clear.

1. Find and use the [review1.yaml](#) file included in the course tarball. Use the **find** output and copy the YAML file to your home directory. Use **kubectl create** to create the object. Determine if the pod is running. Fix any errors you may encounter. The use of **kubectl describe** may be helpful.

```
student@cp:~$ find ~ -name review1.yaml
```

```
student@cp:~$ cp <copy-paste-from-above> .
```

```
student@cp:~$ kubectl create -f review1.yaml
```

2. After you get the pod running remove any pods or services you may have created as part of the review before moving on to the next section. For example:

```
student@cp:~$ kubectl delete -f review1.yaml
```

3. Use the [review2.yaml](#) file to create a non-working deployment. Fix the deployment such that both containers are running and in a READY state. The web server listens on port 80, and the proxy listens on port 8080.
4. View the default page of the web server. When successful verify the GET activity logs in the container log. The message should look something like the following. Your time and IP may be different.

```
192.168.124.0 - - [3/Dec/2025:03:30:31 +0000] "GET / HTTP/1.1" 200 612 "-"
"curl/7.58.0" "-"
```

5. Find and use the `review4.yaml` file to create a pod, and verify it's running
6. Edit the pod such that it only runs on your worker node using the `nodeSelector` label.
7. Determine the CPU and memory resource requirements of `design-pod1`.
8. Edit the pod resource requirements such that the CPU limit is exactly twice the amount requested by the container. (Hint: subtract .22)
9. Increase the memory resource limit of the pod until the pod shows a `Running` status. This may require multiple edits and attempts. Determine the minimum amount necessary for the `Running` status to persist at least a minute.
10. Use the `review5.yaml` file to create several pods with various labels.
11. Using **only** the `--selector` value `tux` to delete only those pods. This should be half of the pods. Hint, you will need to view pod settings to determine the key value as well.
12. Create a new cronjob which runs `busybox` and the `sleep 30` command. Have the cronjob run every three minutes. View the job status to check your work. Change the settings so the pod runs 10 minutes from the current time, every week. For example, if the current time was 2:14PM, I would configure the job to run at 2:24PM, every Monday.
13. Delete any objects created during this review. You may want to delete all but the `cronjob` if you'd like to see if it runs in 10 minutes. Then delete that object as well.
14. Create a new secret called `specialofday` using the key `entree` and the value `meatloaf`.
15. Create a new deployment called `foodie` running the `nginx` image.
16. Add the `specialofday` secret to pod mounted as a volume under the `/food/` directory.
17. Execute a bash shell inside a `foodie` pod and verify the secret has been properly mounted.
18. Update the deployment to use the `nginx:1.12.1-alpine` image and verify the new image is in use.
19. Roll back the deployment and verify the typical, current stable version of `nginx` is in use again.
20. Create a new 200M NFS volume called `reviewvol` using the NFS server configured earlier in the lab.



21. Create a new PVC called `reviewpvc` which will use the `reviewvol` volume.
22. Edit the deployment to use the PVC and mount the volume under `/newvol`
23. Execute a bash shell into the `nginx` container and verify the volume has been mounted.
24. Delete any resources created during this review.
25. Create a new deployment which uses the `nginx` image.
26. Create a new `LoadBalancer` service to expose the newly created deployment. Test that it works.
27. Create a new `NetworkPolicy` called `netblock` which blocks all traffic to pods in this deployment only. Test that all traffic is blocked to deployment.
28. Create a pod running `nginx` and ensure traffic can reach that deployment.
29. Update the `netblock` policy to allow traffic to the pod on port 80 only. Test that you can now access the default `nginx` web page.
30. Find and use the `review6.yaml` file to create a pod.  

```
student@cp:~$ kubectl create -f review6.yaml
```
31. View the status of the pod.
32. Use the following commands to figure out why the pod has issues.  

```
student@cp:~$ kubectl get pod securityreview

student@cp:~$ kubectl describe pod securityreview

student@cp:~$ kubectl logs securityreview
```
33. After finding the errors, log into the container and find the proper id of the `nginx` user.
34. Edit the pod such that the `securityContext` is in place and allows the web server to read the proper configuration files.

35. Create a new `serviceAccount` called `securityaccount`.
36. Create a `ClusterRole` named `secrole` which only allows create, delete, and list of pods in all `apiGroups`.
37. Bind the new `clusterRole` to the new `serviceAccount`.
38. Locate the token of the `securityaccount`. Create a file called `/tmp/securitytoken`. Put only the value of `token:` is equal to, a long string that may start with `eyJH` and be several lines long. Careful that only that string exists in the file.
39. Remove any resources you have added during this review
40. Create a new pod called `webone`, running the `nginx` service. Expose port 80.
41. Create a new service named `webone-svc`. The service should be accessible from outside the cluster.
42. Update both the pod and the service with selectors so that traffic for to the service IP shows the web server content.
43. Change the type of the service such that it is only accessible from within the cluster. Test that exterior access no longer works, but access from within the node works.
44. Deploy another pod, called `webtwo`, this time running the `wlniao/website` image. Create another service, called `webtwo-svc` such that only requests from within the cluster work. Note the default page for each server is distinct.
45. Test DNS names and verify CoreDNS is properly functioning.
46. Install and configure an ingress controller such that requests for `webone.com` see the `nginx` default page, and requests for `webtwo.org` see the `wlniao/website` default page. It does not matter which ingress controller you use.
47. Remove any resources created in this review.
48. Install a new cluster using an recent, previous version of Kubernetes. Backup etcd, then properly upgrade the entire cluster.
49. Create a pod running `busybox` without the scheduler being consulted.
50. Continue to create objects, integrate them with other objects and troubleshoot until each domain item has been covered.